

科学计算与系统建模仿真平台MWORKS

基础培训签到



请各位老师、同学扫码签到

MoHub技术交流专区/高校专区

MWORKS技术答疑和交流社区



直接搜索“MoHub”

<https://mohub.net>

基于MWORKS.Sysplorer的 系统建模仿真应用

徐德军

苏州同元软控信息技术有限公司

2025年5月23日



传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，
版权所有，侵权必究

目录

1. MWOKRS.Sysplorer软件介绍

2. 软件界面简介

3. 图形化建模

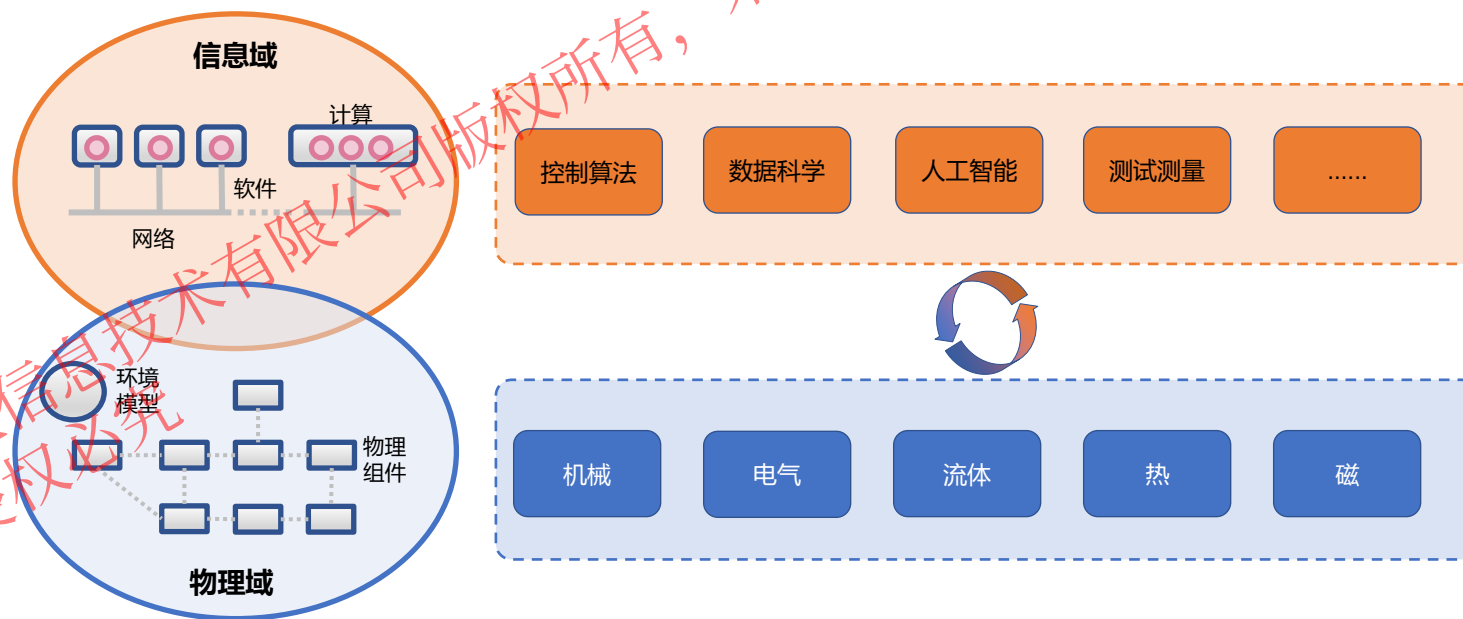
4. Modelica文本建模

5. MWORKS.Sysplorer与Simulink之间的差异

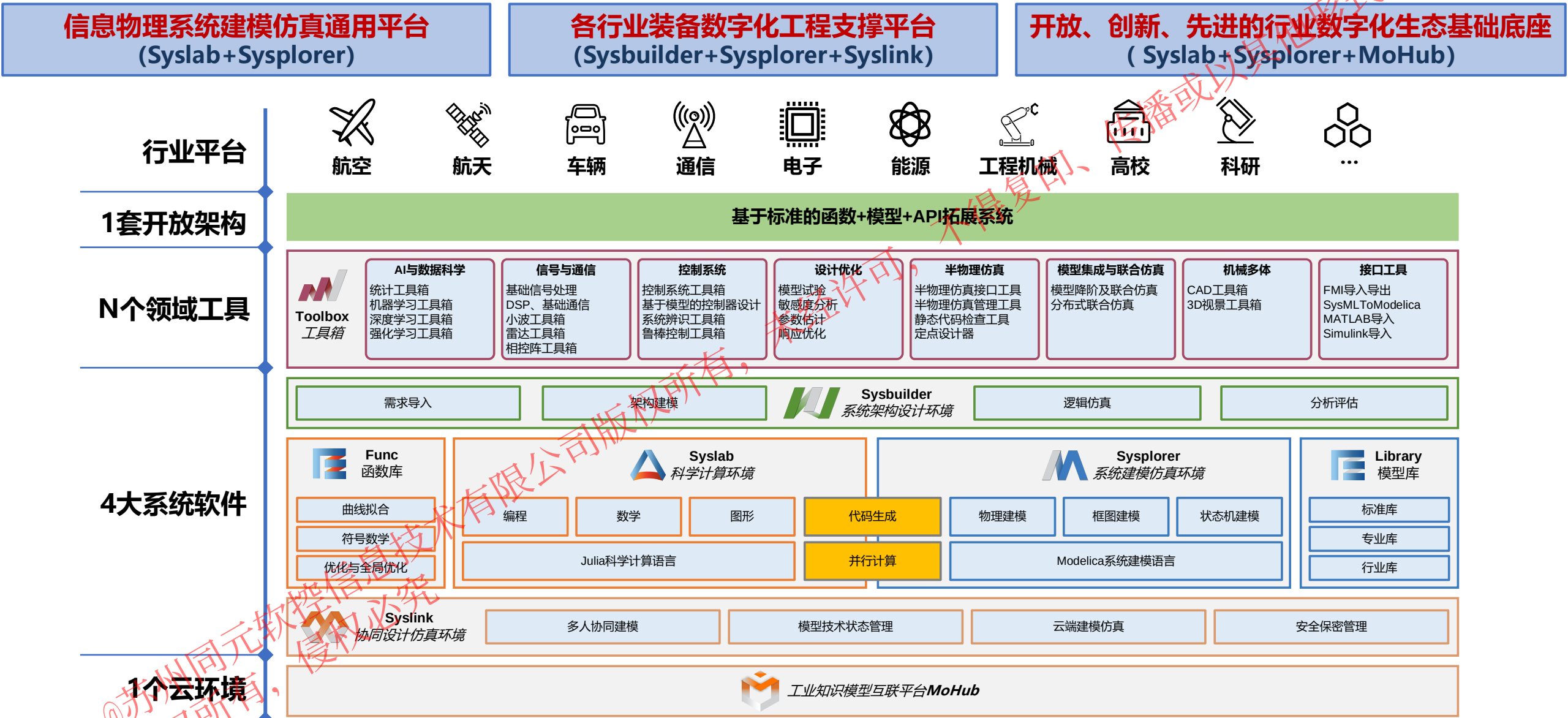
6. MWORKS.Sysplorer与Syslab双向融合



现代装备都是**信息物理融合系统（CPS）** CPS系统的设计、建模与仿真是数字化的核心



1.1 新一代科学计算与系统建模仿真平台MWOKRS



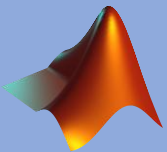
1.1 新一代科学计算与系统建模仿真平台MWOKRS

MWORKS正在成为**中国的MATLAB**，是全球**第四**个科学计算与建模仿真一体化软件

MWORKS经过 **20 余年**的持续研发和技术积累，在功能层面达到与MATLAB/Simulink相当水平

MWORKS.Syslab

MATLAB



科学计算环境

- 交互式脚本编程环境
- 基础语言功能
- 基础数学功能
- 解释与调试模块
- 图形可视化模块
- 性能分析模块
- 外部语言接口模块

MWORKS.Sysplorer

Simulink



系统建模仿真环境

- 可视化框图建模环境
- 编译与分析模块
- 调试模块
- 仿真求解模块
- 可视化后处理环境
- 异构模型集成与联合仿真
- 状态机事件建模

MWORKS.Toolbox

MATLAB.Toolbox



专业工具箱

- 数学优化类工具箱
- AI与数据科学类工具箱
- 信号处理类工具箱
- 通信类工具箱
- 射频与混合信号类工具箱
- 控制系统类工具箱
- 模型驱动的代码生成与实时仿真类工具箱
- 机械多体类工具箱
- 基于模型的设计优化类工具箱
- 模型库

MWORKS是**国际先进**、**亚洲唯一**、**完全自主**的系统仿真软件，具有国际排名**前三**的自主内核，也是中国**首例向欧美出口**求解内核引擎的高端设计仿真工业软件。

1.1 新一代科学计算与系统建模仿真平台MWOKRS

——是继CAD、CAE之后新一代系统级数字化设计仿真工业软件，是**数字化变革制高点**，已被中国航天指定为**装备数字化核心支撑软件**。

重大型号工程	应用对象
空间站	空间站总体：GNC、能源、推进、控制、环热控、通信
嫦娥工程	嫦娥3号、嫦娥5号：能源供配电系统
大推力运载火箭	重型火箭：增压输送系统、电气系统
战略导弹	导弹发射系统
火星探测	火星探测器：能源供配电系统
卫星互联网	星座链路数字化设计验证
国际月球科研站	多学科协同论证与仿真评价系统
飞航导弹	飞行器多领域统一建模仿真
高超飞行器	多专业联合仿真
大飞机	ARJ21、C919、CR929
两机专项	航空发动机 CJ1000、CJ2000
核能动力	新型核动力系统

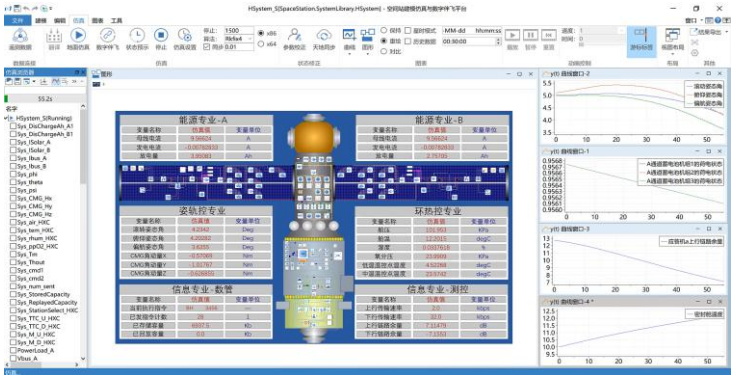
“嫦娥五号”型号应用



“载人登月”型号应用

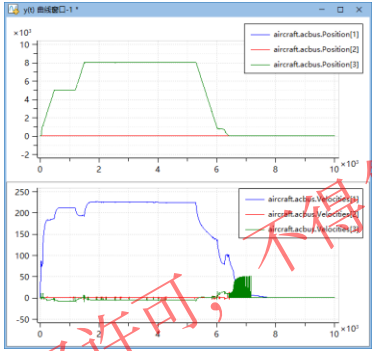
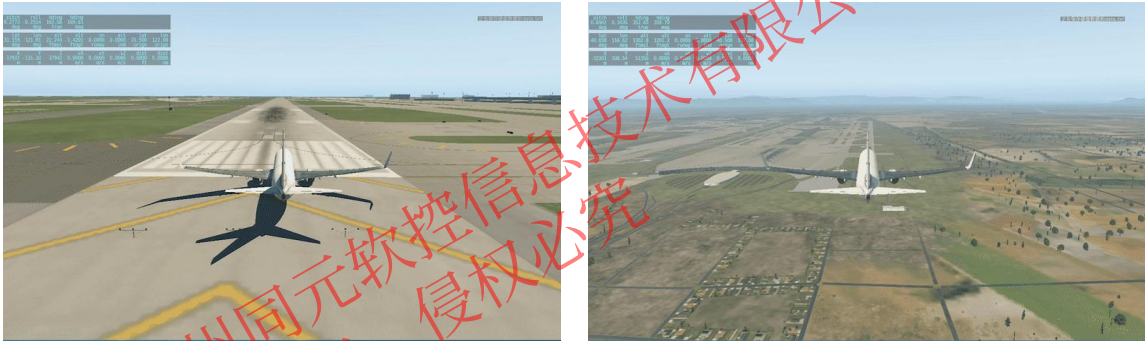
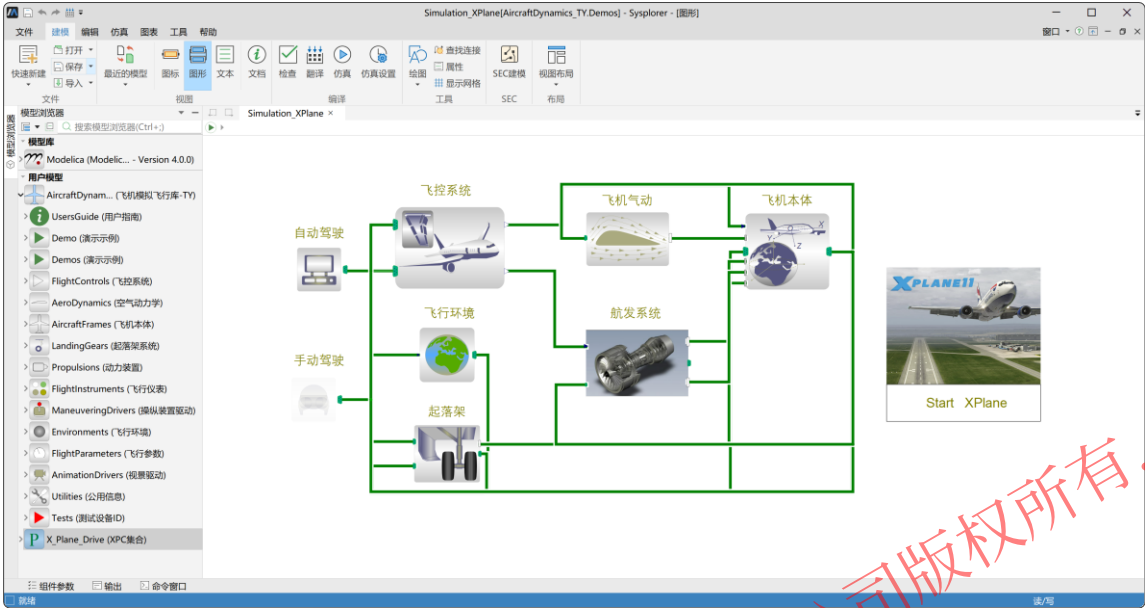


“空间站”型号应用

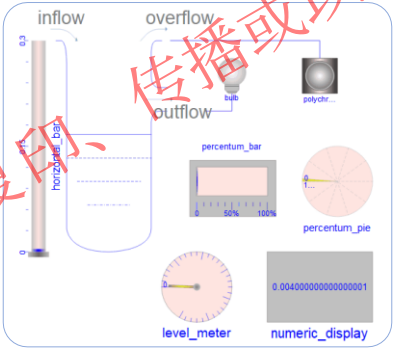


1.2 系统建模仿真环境 Sysplorer

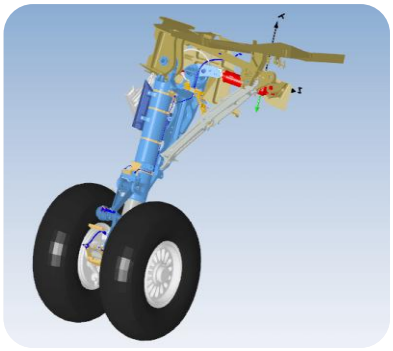
Sysplorer是面向多领域工业产品的系统建模与仿真环境，支持多领域统一建模规范Modelica，是数字化时代国际领先的建模仿真通用软件



曲线显示



二维动画



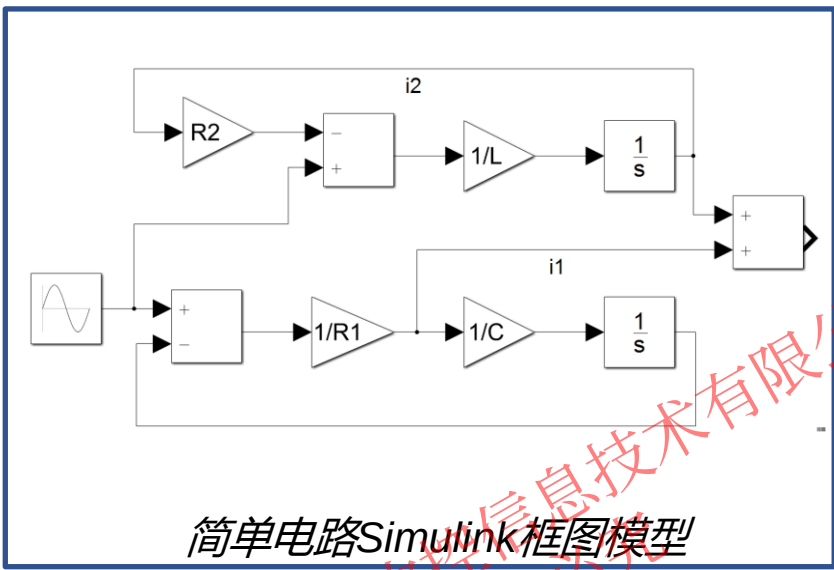
三维动画

- **系统建模：** 提供图形/文本混合建模，支持物理、框图、状态机等多范式系统建模
- **编译求解：** 内置变步长和定步长多种求解算法，并支持用户扩展
- **仿真后处理：** 提供曲线、二维/三维动画等丰富易用的的可视化后处理能力
- **实时代码生成：** 支持Windows、Linux、VxWorks、NI VeriStand、ETAS等多种操作系统的实时代码生成
- **扩展接口：** 支持API、FMI、三维CAD、CAE软件等接口

1.2 系统建模仿真环境 Sysplorer

MWORKS.Sysplorer建模语言：Modelica语言

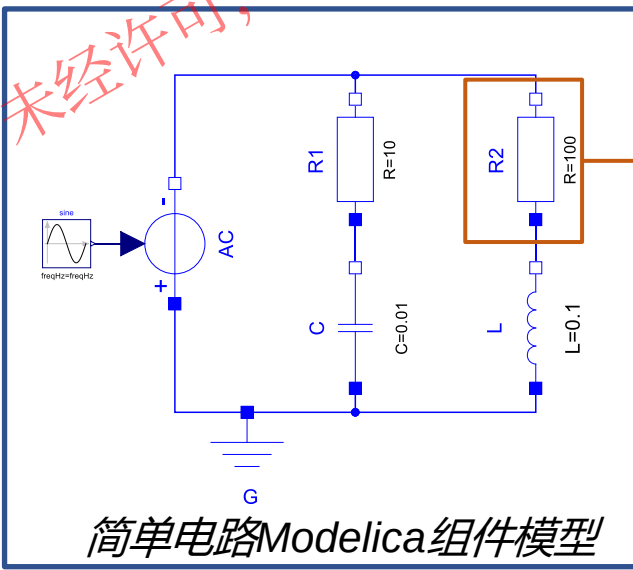
多领域系统统一建模仿真语言Modelica已经成为世界公认的MBSE和CPS的核心关键支撑技术。



同一电路对象

Sysplorer特点:

- 1. 直观
- 2. 无需因果推导
- 3. 可直接替换组件
- 4. 陈述式语言, 建模效率更高



```
model Resistance "电阻"
  parameter SI.Resistance R = 10;
  Modelica.SIunits.Voltage v;
  Modelica.SIunits.Current i;
  Connector.PositivePin positivePin
    annotation (...);
  Connector.NegativePin negativePin
    annotation (...);
  annotation (...);
equation
  v = positivePin.v - negativePin.v;
  i = positivePin.i;
  positivePin.i + negativePin.i = 0;
  v = R * i; //欧姆定律, 直接描述
end Resistance;
```

可直接基于已有模型库和物理拓扑关系拖拽式建模，
当模型库不能满足时，还可基于模型机理方程直接使用Modelica语言进行陈述式建模



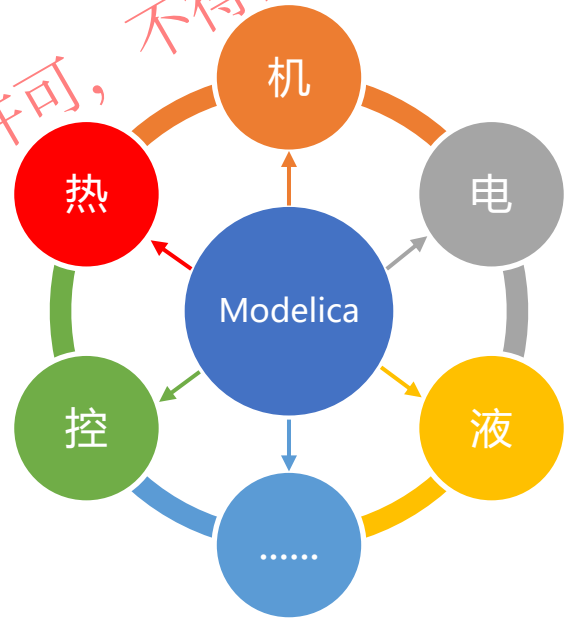
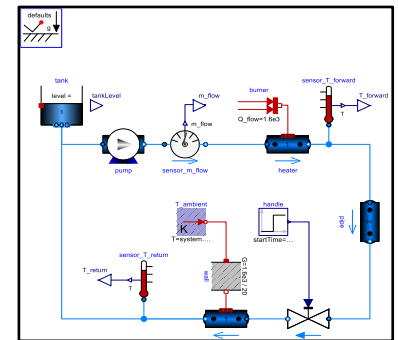
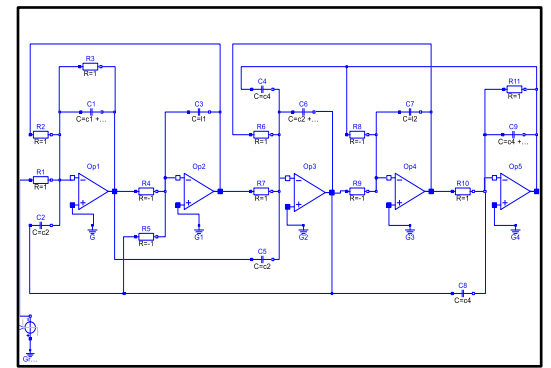
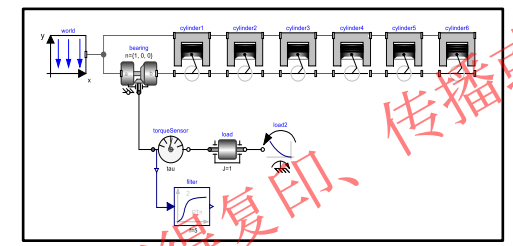
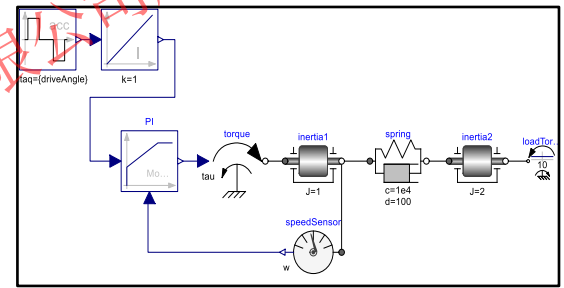
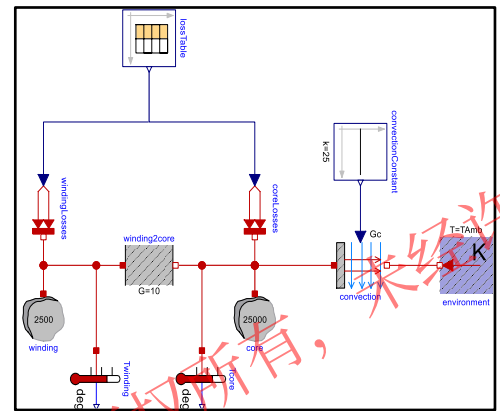
1.2 系统建模仿真环境 Sysplorer



面向**系统(物理)建模仿真**的工程师:

- 机械系统建模
- 控制系统建模
- 热力系统建模
- 电力系统建模
- 流体系统建模
- 多领域耦合系统建模
(包含机、电、液、热、控等多专业的复杂装备)

.....



@苏州同元软控信息技术有限公司
版权所有，侵权必究



1.2 系统建模仿真环境 Sysplorer



Sysplorer
系统建模仿真软件

航天领域:

- 航天器姿态和轨道动力学分析
- 航天器GNC系统分析
- 航天器能源和信息系统分析
- 航天器多领域集成仿真分析
- 液体火箭发动机静态/动态性能分析
- 武器系统设计验证
- ...

航空领域:

- 飞行器总体性能评估与验证
- 飞行任务模拟与评估
- 子系统功能性能评估与验证
- 基于模型的半物理仿真试验
- ...

汽车领域:

- 混合动力汽车快速原型设计
- 混合动力汽车能量管理系统设计与仿真
- 涡轮增压发动机动态性能仿真与设计
- 双离合传动系统与控制系统设计
- 底盘与传动系统的实时硬件在环仿真
- 多种路面接触场景下的轮胎行为仿真分析
- ...

工程机械领域:

- 挖掘机液压和传动系统仿真分析
- 起重机伸缩臂动态特性分析
- 皮带传输设备仿真分析
- 装载机系统仿真与优化分析
- ...

能源动力领域:

- 核电轻水反应堆系统性能分析
- 太阳能发电设备系统设计与分析
- 热电设备动态性能分析
- 制冷设备设计与分析
- 空调系统的稳态及瞬态仿真分析
- 底盘与传动系统的实时硬件
- 建筑物及其暖通空调系统热响应动态分析
- 聚电解质膜燃料电池动态性能建模仿真
- 整体煤气化联合循环(IGCC)发电系统设计和动态仿真分析
- ...

船舶、机器人、数字工厂

1.2 系统建模仿真环境 Sysplorer - 统一环境下的多范式建模

单一建模范式难以支撑复杂系统建模



算法编程

```
model BouncingBall
parameter Modelica.SIunits.Height H0 = 1 "初始高度";
Modelica.SIunits.Height H "小球高度";
constant Real g = Modelica.Constants.g_n "重力加速度";
parameter Real e = 0.8 "反弹恢复系数";
Modelica.SIunits.Velocity v "小球速度";
initial equation
H = H0;
equation
der(H) = v;
der(v) = -g;
when H < 0 then
reinit(v, -e * v);
end when;
end BouncingBall;
```

Modelica建模

```
function EXTHALMAN(meas, deltat, time)
# 声明全局变量
global g_P
global g_xhat
# Initialization
residual = []
xhatOut = []
# 1. Compute Phi, Q, and R
Phi = [1 deltat 0 0; 0 1 0 0; 0 0 1 deltat; 0 0 0 1]
Q = Diagonal([0, 0.005, 0, 0.005])
R = Diagonal([300*2, 0.001*2])
# 2. Propagate the covariance matrix:
g_P = Phi * g_P * Phi' + Q
# 3. Propagate the track estimate:
g_xhat = Phi * g_xhat
```

Julia算法

```
// C 梯度下降算法
do {
double gradient = df(x);
x = x - learning_rate * gradient;
step_size = learning_rate * gradient;
iteration++;
} while (fabs(step_size) > precision && iteration < max_iteration);
```

C代码算法

框图建模

Sysblock

物理建模

Sysplorer基础建模环境

参数估计

参数估计APP

模型降阶

模型降阶与融合仿真APP

神经网络

机器学习工具箱
深度学习工具箱
强化学习工具箱

系统辨识

系统辨识工具箱

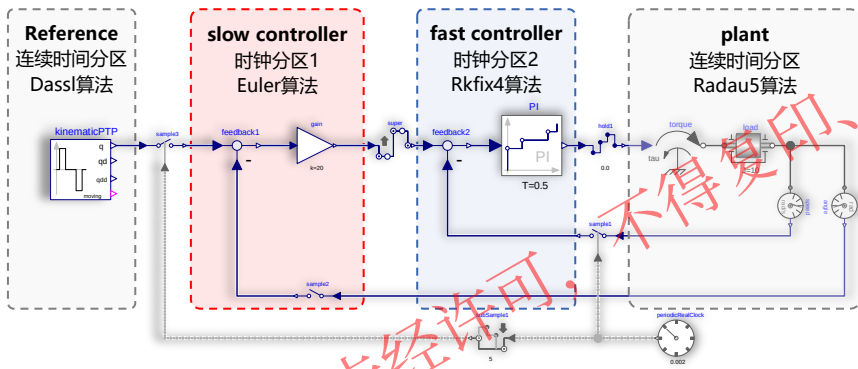
Sysplorer+Syslab基础建模环境 Sysblock Sysplorer基础建模环境

1.2 系统建模仿真环境 Sysplorer - 国际先进的高性能系统仿真内核

- 丰富、高效的积分求解算法
- 同步时钟机制支持连续、离散混合的多速率系统模型更灵活、高效的求解
- 在求解规模、效率、鲁棒性上国际领先

提供变步长、定步长合计22种求解算法；同时支持用户自定义求解算法的集成

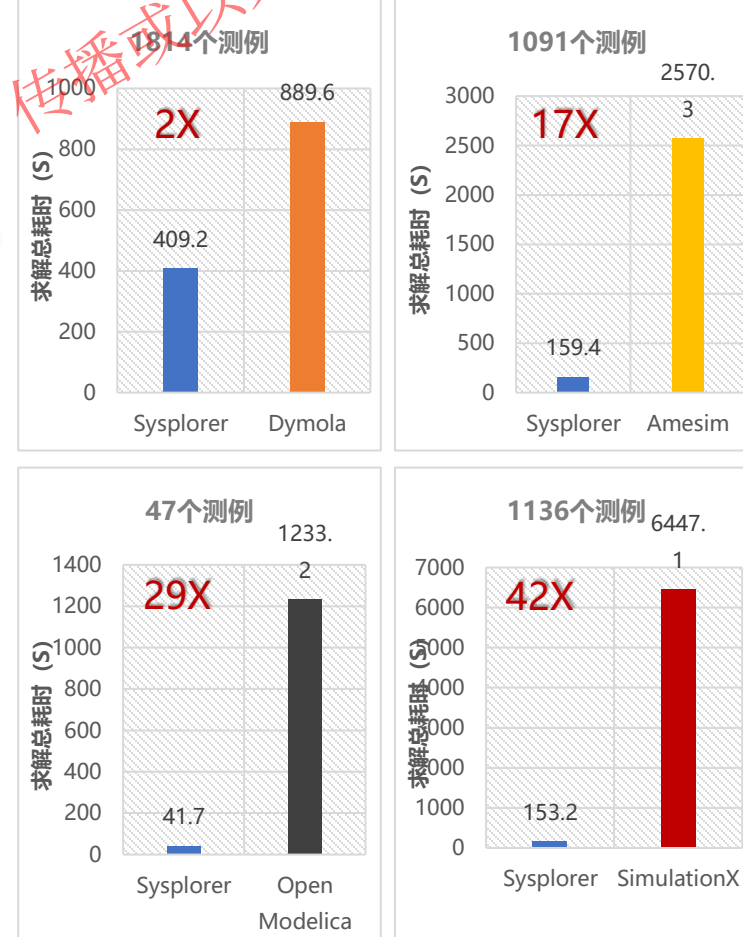
算法类型	序号	求解算法	说明
变步长积分算法 Variable Step	1	Dassl	变步长和变阶算法, 适用范围广
	2	Ida	隐式变步长方法, DAE、初值问题 (IVP) 的通用求解器, 支持准确的事件定位
	3	Mebdfi	是 Mebdf 的扩展方法, 能有效处理刚性问题
	4	Lsodar	是 LSODA 算法的变种, 支持事件定位
	5	Lsode	ODE通用求解, 可处理刚性/非刚性问题
	6	Cvode	变步长变阶的多步方法
	7	Mebdf	扩展改进 BDF 方法, 用于求解具有刚性初值的大型稀疏系统, 具有更好的稳定性和高阶收敛性
	8	Radau5	变步长定阶算法, 适用于求解刚性问题
	9	Sdirk34	单对角隐式龙格-库塔 (4 阶) 算法
	10	Esdirk23	显式单对角隐式龙格-库塔 (3阶)
	11	Esdirk34	显式单对角隐式龙格-库塔 (4阶)
	12	Esdirk45	显式单对角隐式龙格-库塔 (5阶)
	13	Dopri5	基于 Dormand-Prince 的5(4) 阶显式龙格-库塔方法, 适用于非刚性问题, 检测到类刚性问题报出警告
	14	Dop853	基于 Dormand-Prince 的8(5, 3) 阶显式龙格-库塔方法
定步长积分算法 Fixed Step	15	Euler	适合实时仿真的固定步长的算法
	16	Rkfix2	龙格-库塔 (2阶) 算法
	17	Rkfix3	龙格-库塔 (3阶) 算法
	18	Rkfix4	龙格-库塔 (4阶) 算法
	19	Rkfix6	龙格-库塔 (6阶) 算法
	20	Rkfix8	龙格-库塔 (8阶) 算法
	21	ImplicitEuler	隐式欧拉法, 后向欧拉方法, 算法稳定强但求解效率低, 适合刚性较强、精度要求低的问题
	22	ImplicitTrap ezoid	隐式梯形法, 基于隐式梯形规则的隐式二阶龙格库塔方法; 针对刚性较弱的问题
自定义	23	Custom	支持用户自定义求解算法



支持稳态查找与稳态初始化方法



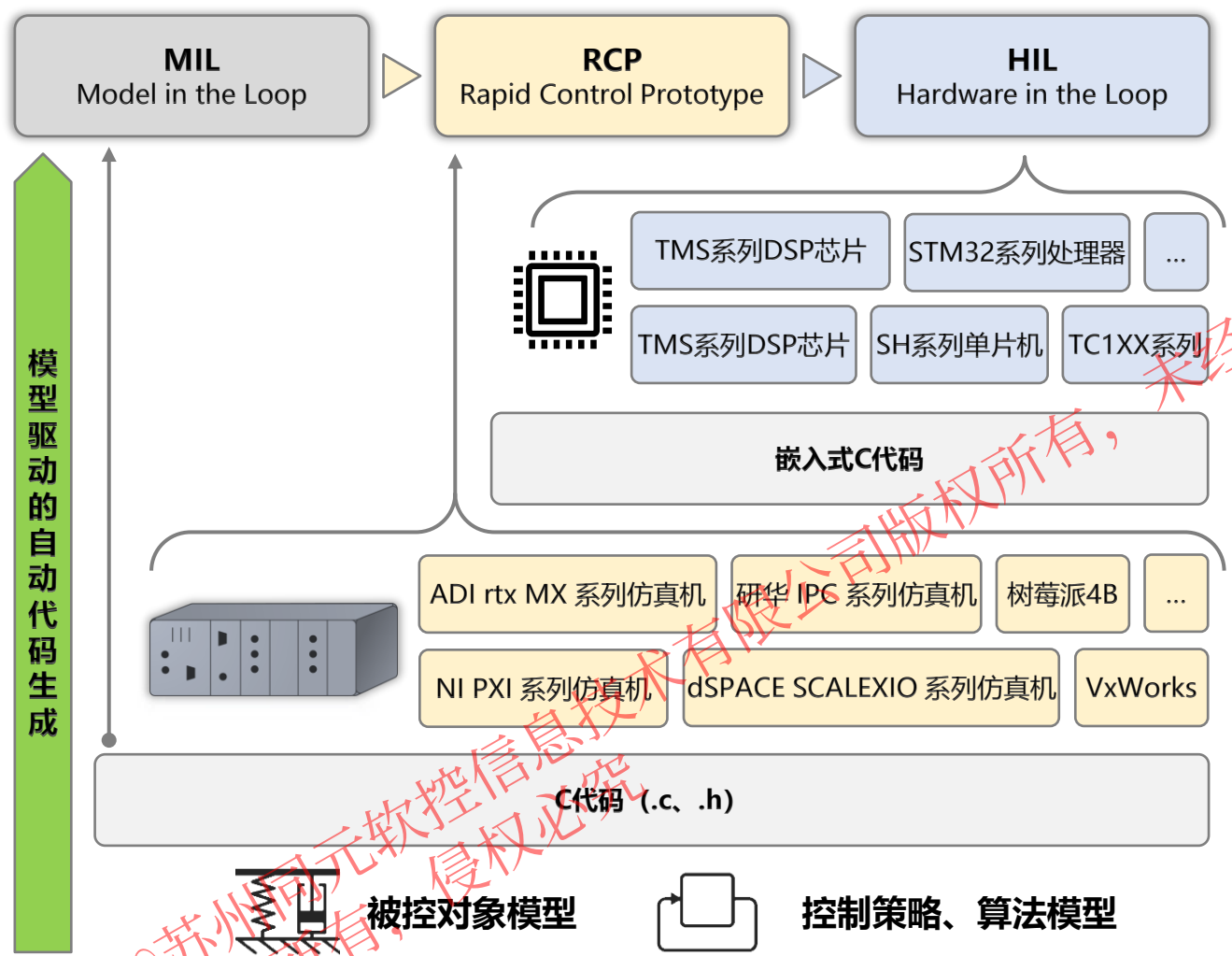
稳态查找: 在动态仿真过程中, 自动查找稳态, 并终止仿真
稳态初始化: 自动计算初始稳态, 作为初始值开展动态仿真



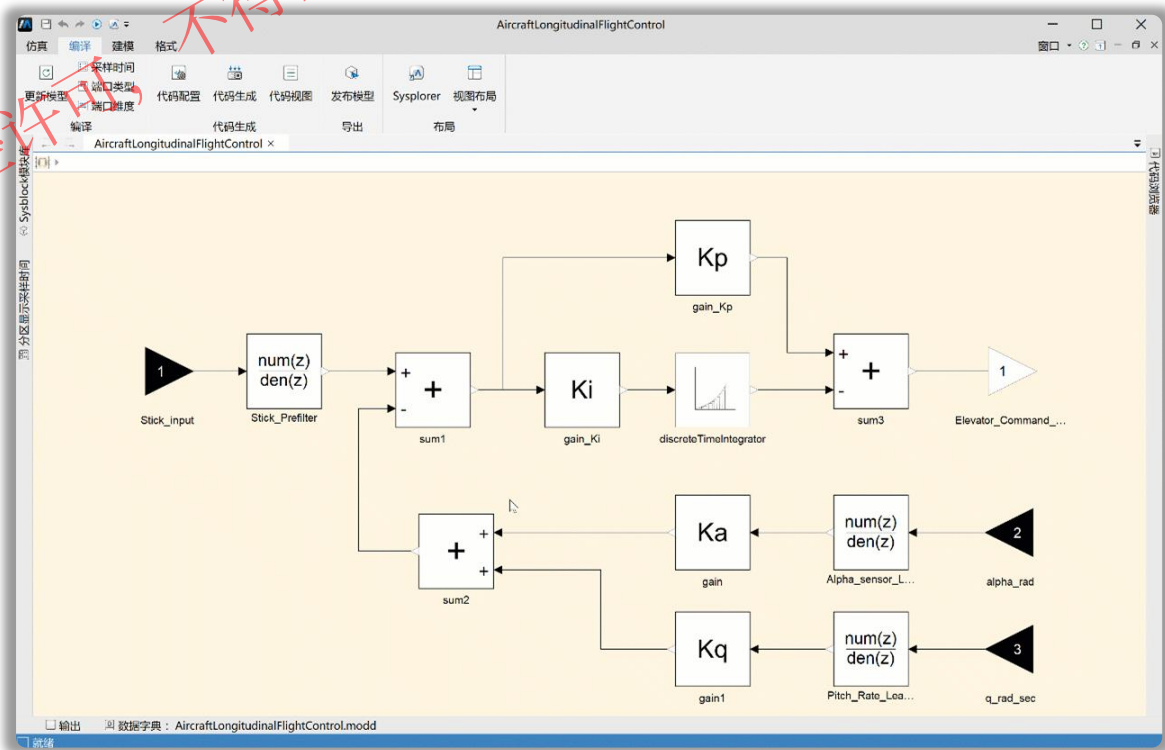
以Modelica 4.0 标准库和39个开源库中选取1814个实例模型为基准。
仿真步数和时间采用模型默认设置, 方程规模从10到10W+

1.2 系统建模仿真环境 Sysplorer - 模型驱动的多目标代码自动生成

控制策略与被控对象均支持代码生成与硬件部署，基于模型生成的嵌入式代码可在MCU中us级运行



嵌入式代码生成器



飞机飞控系统纵向控制律示例

1.2 系统建模仿真环境 Sysplorer - 大量经工程验证的模型库

- 提供丰富的模型库，涵盖 3 个通用基础库、19 个专业模型库、多个行业系统库与应用库
- 19 个专业模型库，涵盖 1300+ 模型、6 个产品级汽车行业应用库，涵盖 2400+ 个模型，与业界多款商用模型库对标情况优异

行业应用库

发动机库

电池库

车辆电子库

动力性经济性库

热管理库

车辆动力学库

行业系统库

GNC库

飞控系统库

汽车动力库

反应堆库

航天电气库

飞机液压库

刹车系统库

蒸汽给水库

液体动力库

起落架库

车辆控制库

燃烧系统库

固体动力库

航空发动机库

变速箱库

发电机库

航天环控库

飞行动力学库

车辆动力学库

雷达设备库

燃油系统库

项目复用级模型库

专业组件库

基础机械库

传动组件库

柔性体库

平面机械库

液压元件库

热液压元件库

气动元件库

基础电气模型库

电力系统库

基础热流体模型库

多体模型库

三维传动库

接触模型库

液压组件库

热液压组件库

气动组件库

电机模型库

燃料电池模型库

热模型库

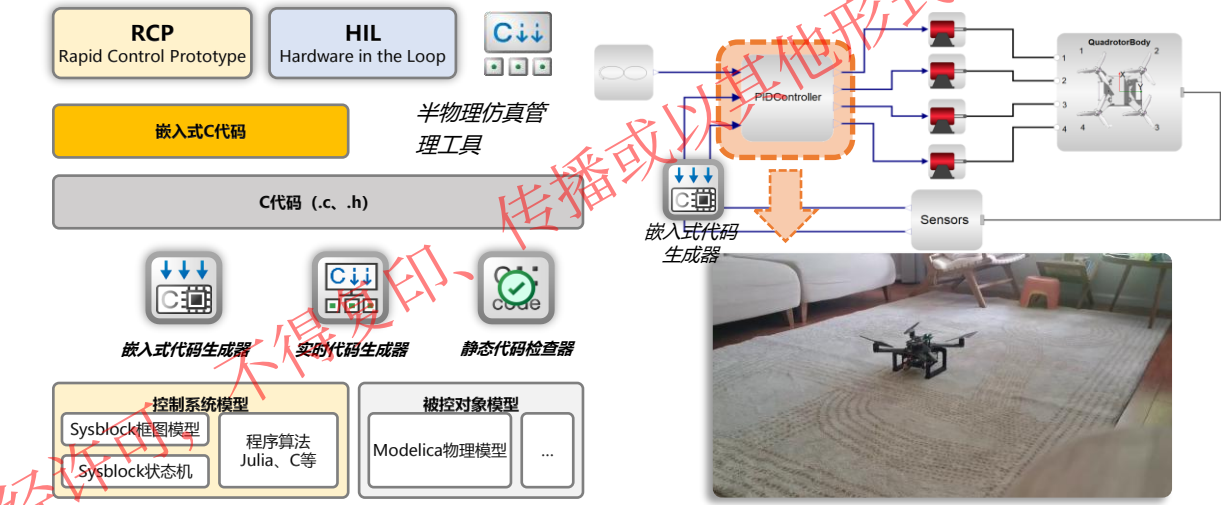
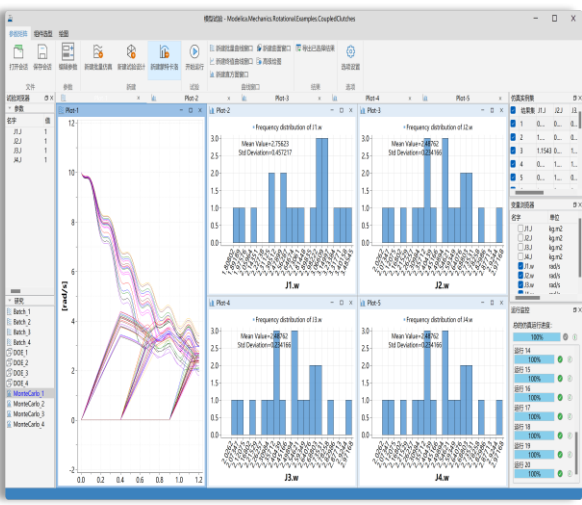
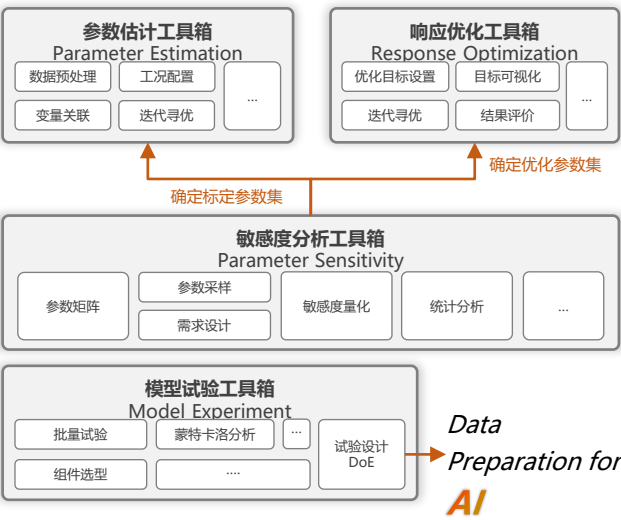
通用基础库

液压介质模型库

热流介质模型库

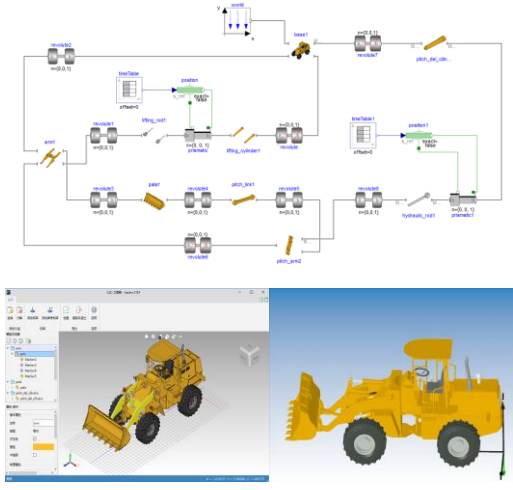
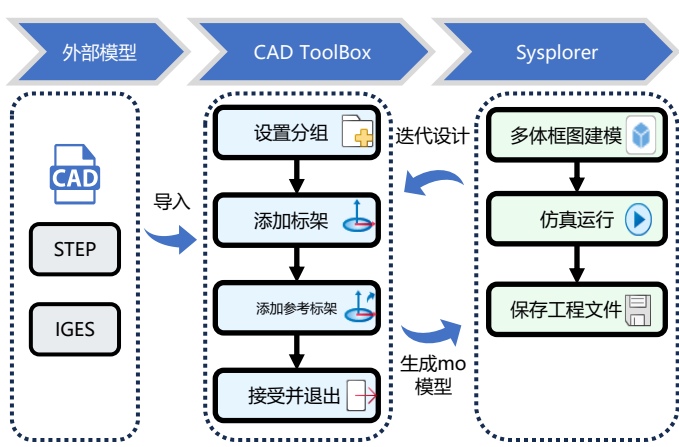
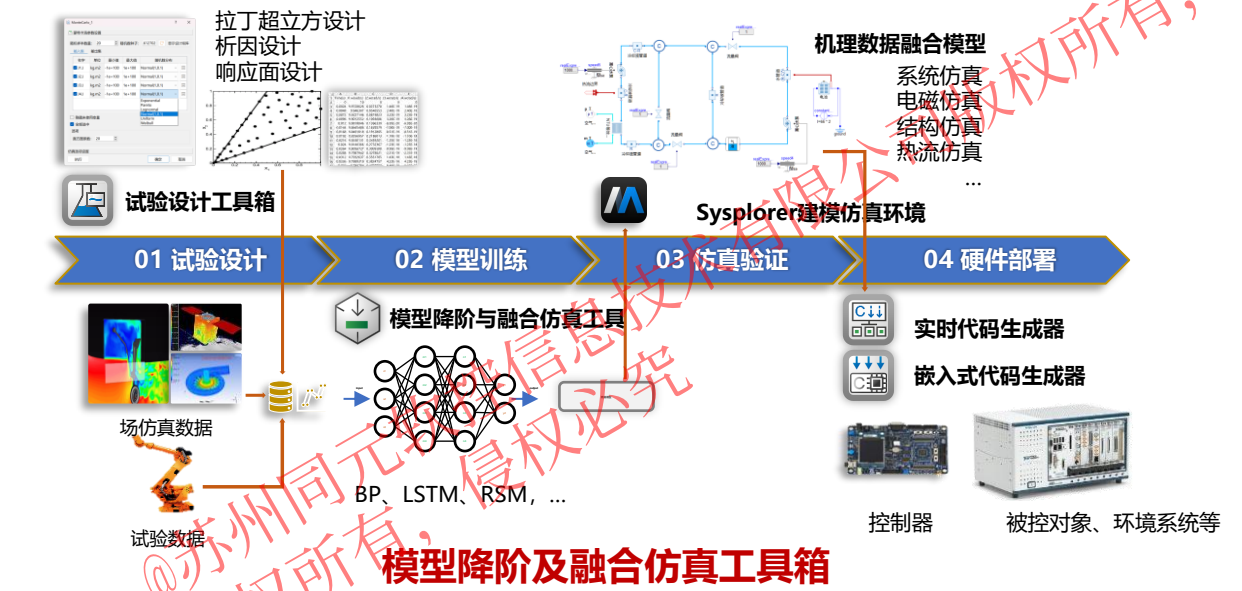
气体介质模型库

1.2 系统建模仿真环境 Sysplorer – 专业系列工具箱



设计优化系列工具箱

半物理系列工具箱



机械多体系列工具箱

1.3 软件下载安装

软件下载地址：<https://www.tongyuan.cc/download>



安装说明：

1. 根据需求下载软件，两款软件可独立运行
2. 安装过程中需关闭杀毒软件或弹窗点击信任
3. 两款软件均可选择安装在任意磁盘

2024b 历史版本



MWORXS.Syslab 2024b

科学计算环境

Windows 7/10/11、CentOS 7.9 更新日期：2024-08-30

[查看发行说明](#)

立即下载

Win 7/10/11



MWORXS.Sysplorer 2024b

系统建模仿真环境

Windows 7/10/11、CentOS 7.9 更新日期：2024-08-30

[查看发行说明](#)

立即下载

Win 7/10/11

同元软控官网→技术支持→软件下载

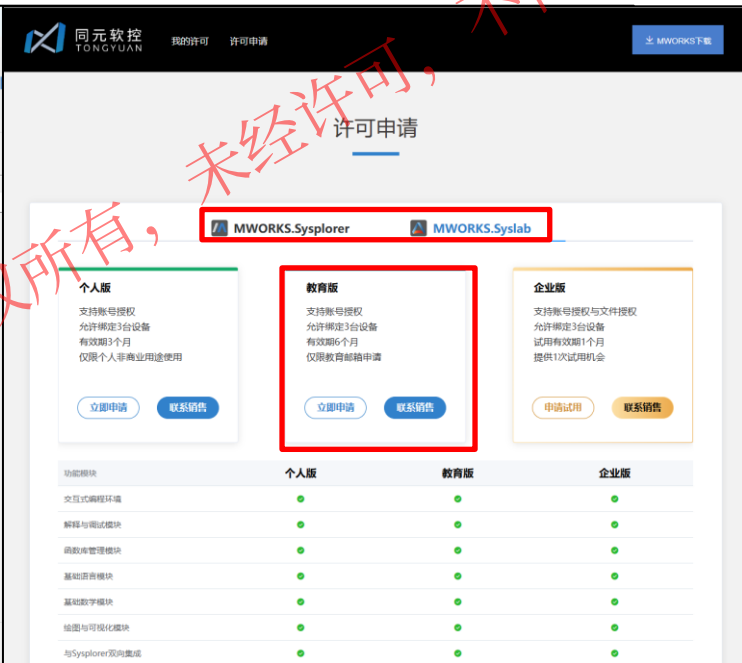


1.3 软件下载安装

激活方式:

个人: 通过学校邮箱注册账号激活

机房: 连接校园网通过服务器地址激活(部分高校已部署可用)



1. 通过edu邮箱注册账号

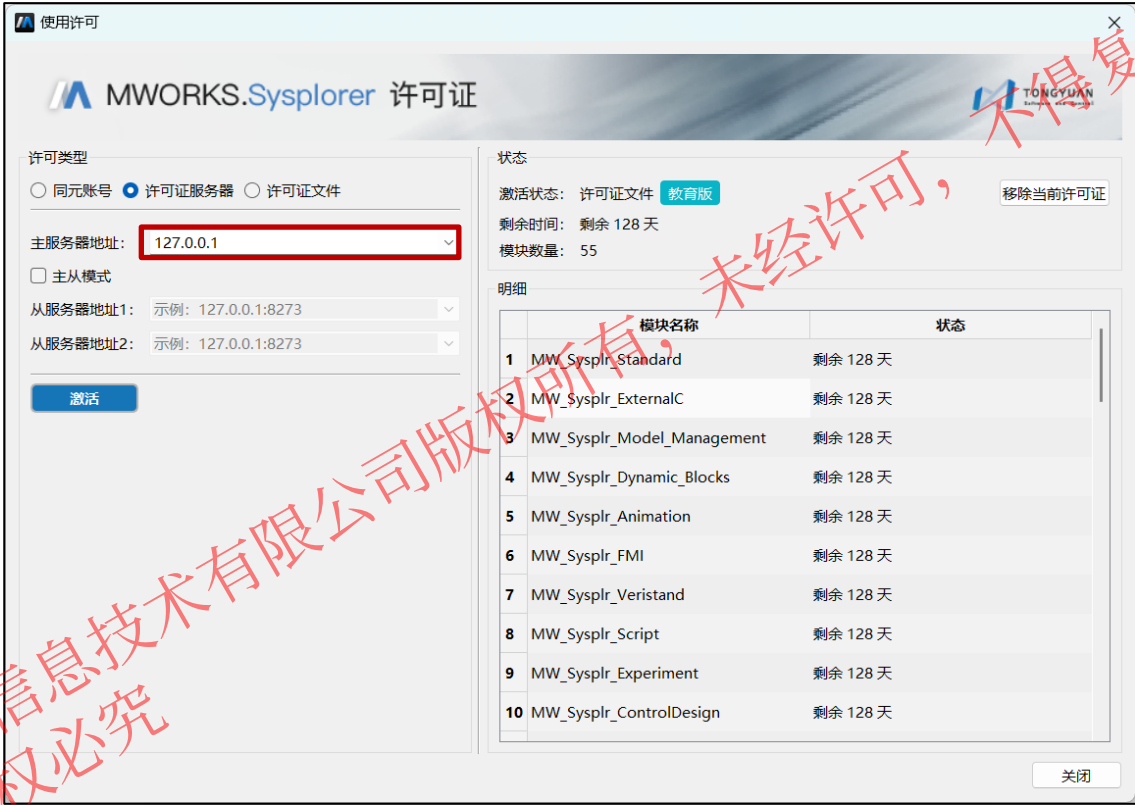
2. 申请对应软件的许可

3. 激活软件, 过期后可重复申请

1.3 软件下载安装

激活方式:

- 个人: 通过学校邮箱注册账号激活
- 机房: 连接校园网通过服务器地址激活(部分高校已部署可用)



1. 输入IP地址点击激活即可

说明:

1. 目前已有部分高校已部署校级服务器, 后续其他高校部署完成后会发主服务器地址至微信群和MoHub高校专区
2. 如老师机房着急上课, 可将授权服务部署至机房服务器



1.4 技术支持

MoHub技术交流/高校专区：提供大量常见问题FAQ，并安排专业工程师解答



高校专区：提供高校专属空间，促进校内互动与协作

MoHub网址：<https://mohub.net>

1.5 MWORKS相关资料

类别	材料
01-同元软控及MWORKS介绍	00-MWORKS产品介绍
	01-新一代科学计算与系统建模仿真平台MWORKS
	02-同元软控企业册
	03-同元软控标杆案例册
	04-同元软控航空案例册
	05-同元软控航天案例册
	06-同元软控能源案例册
	07-同元软控公司宣传片
	Logo
02-MWORKS产品功能列表	01-科学计算与系统建模仿真平台MWORKS-产品册
	02-MWORKS 用户手册
03-MWORKS与MATLAB功能对照	01-MWORKS与其他科学计算软件对比
	02-MWORKS与其他科学计算软件对比-录屏
04-同元基础培训材料	01-科学计算-培训材料
	02-系统建模仿真-培训材料
	03-培训材料列表-同元软控MWORKS
05-MWORKS应用推广培训	01-【虚拟教研室】师资培训系列讲座-2022
	02-【工信部人才交流中心】科学计算与系统建模仿真MWORKS培训-2023
	03-【北理工师资培训】科学计算与系统建模仿真MWORKS培训-2023
	04-【北航师资培训】科学计算与系统建模仿真MWORKS培训-2023
06-MWORKS.API	01-MWORKS.Sysplorer API详细说明
	02-MWORKS.Syslab API详细说明
07-MWORKS平台安装包	01-MWORKS.Sysplorer 安装包
	02-MWORKS.Syslab 安装包

获取方式:

百度网盘链接:

<https://pan.baidu.com/s/14sLpKBDDgNwUd89Lbtgpnw?pwd=TYRK>

提取码: TYRK

☐ 07-MWORKS平台安装包

☐ 06.MWORKS.API

☐ 05-MWORKS应用推广培训及课程替代研讨

☐ 04-同元基础培训材料

☐ 03-MWORKS与MATLAB功能对照

☐ 02-MWORKS产品功能列表

☐ 01-MWORKS平台介绍及典型案例

所有材料均为PDF版本

1.5 MWORKS相关资料

MoHub高校专区：提供教改内部资料和活动资讯



- 进入方式：**
1. MoHub账号绑定**学校邮箱**直接进入
 2. 若无学校邮箱可通过**邀请码**进入

- ☐ 07-MWORKS平台安装包
- ☐ 06.MWORKS.API
- ☐ 05-MWORKS应用推广培训及课程替代研讨
- ☐ 04-同元基础培训材料
- ☐ 03-MWORKS与MATLAB功能对照
- ☐ 02-MWORKS产品功能列表
- ☐ 01-MWORKS平台介绍及典型案例

所有材料均为PPT版本，可按需下载用于教学

MoHub网址：<https://mohub.net>



1.5 MWORKS相关资料

视频课程：提供各类软件使用视频课程，可供老师和学生自学



目录

1. MWOKRS.Sysplorer软件介绍

2. 软件界面简介

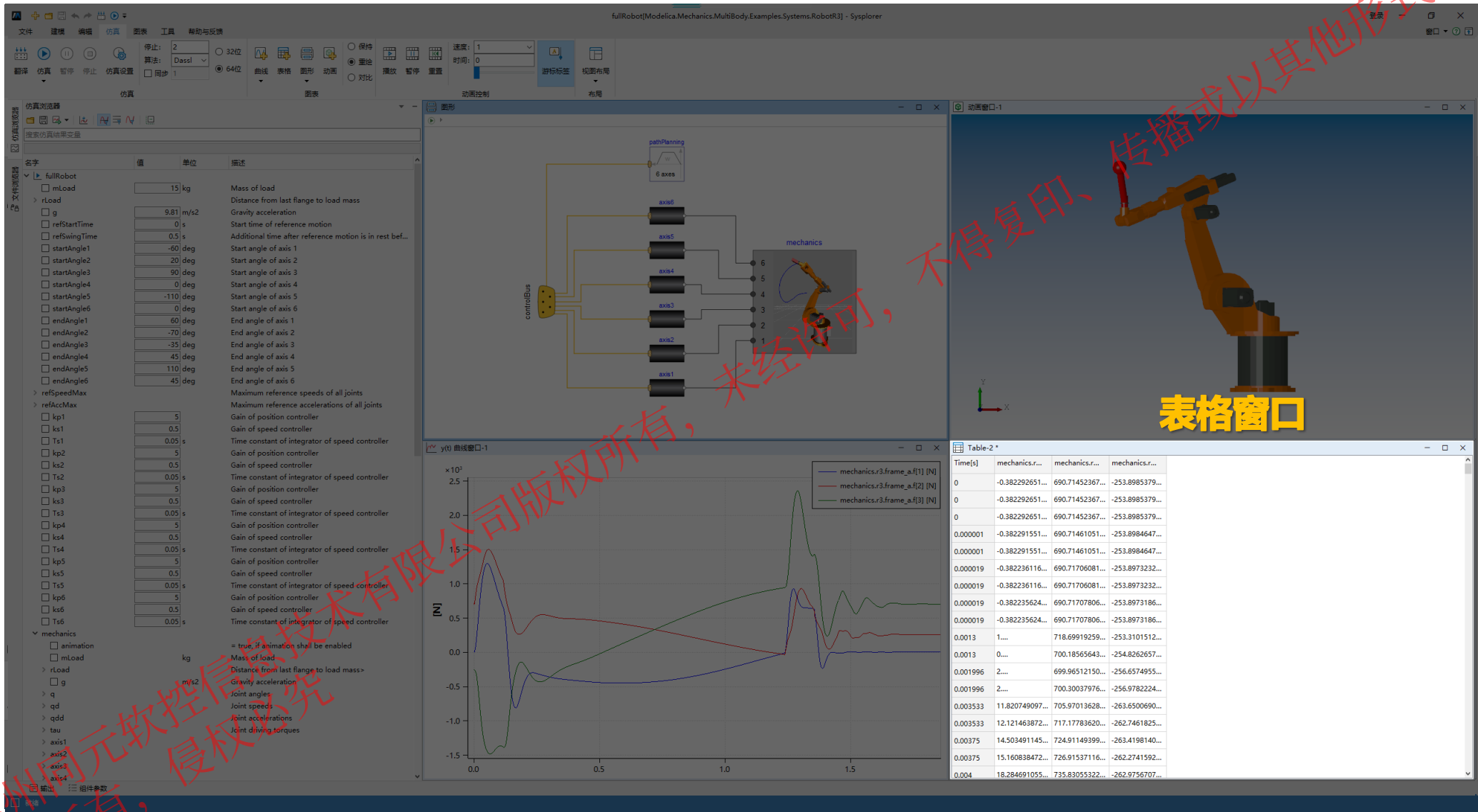
3. 图形化建模

4. Modelica文本建模

5. MWORKS.Sysplorer与Simulink之间的差异

6. MWORKS.Sysplorer与Syslab双向融合

2.MWOKRS.Sysplorer软件界面简介



2. MWOKRS.Sysplorer软件界面

工具栏，帮助->帮助，打开帮助文档



- 平台介绍
- 最新特征
- 案例演示
- 功能说明
- 同元库简介
- Modelica简介
- FMI规范



目录

1. MWOKRS.Sysplorer软件介绍

2. 软件界面简介

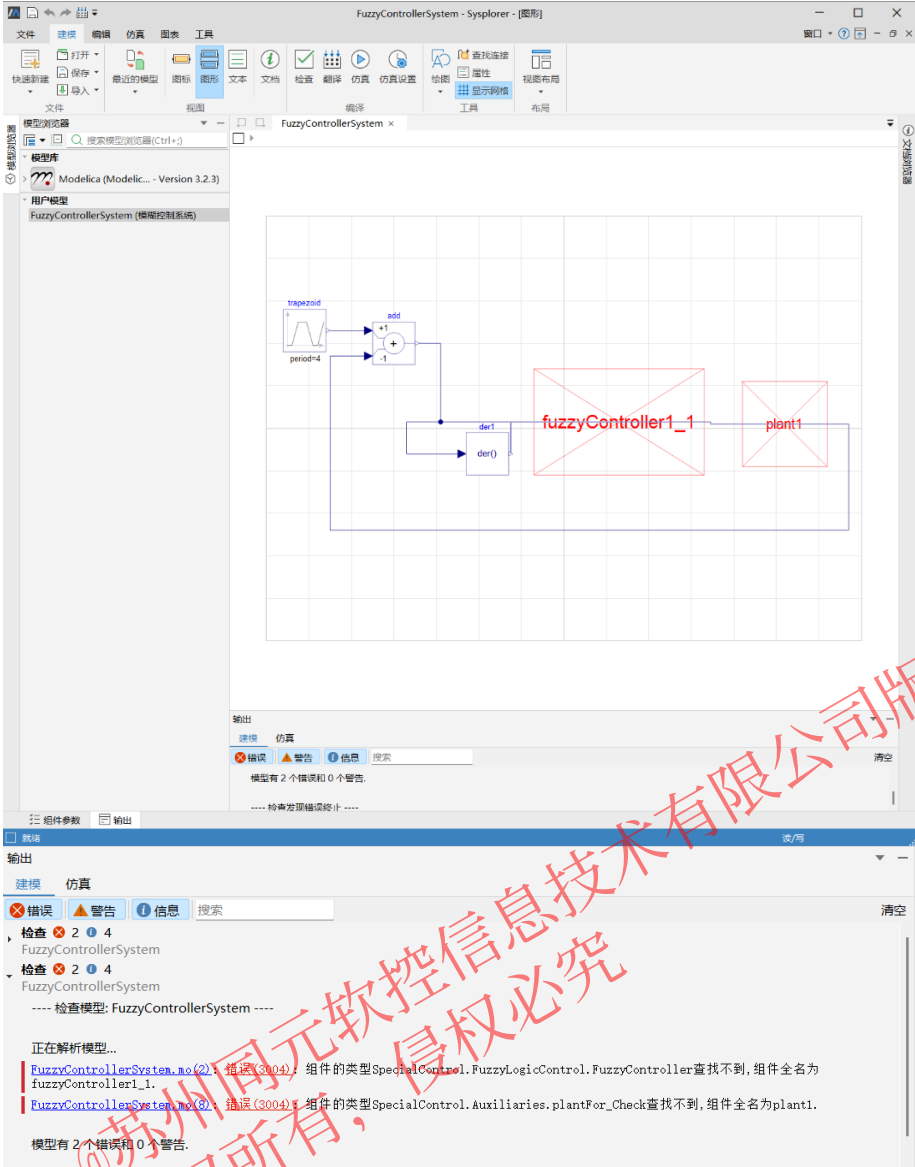
3. 图形化建模

4. Modelica文本建模

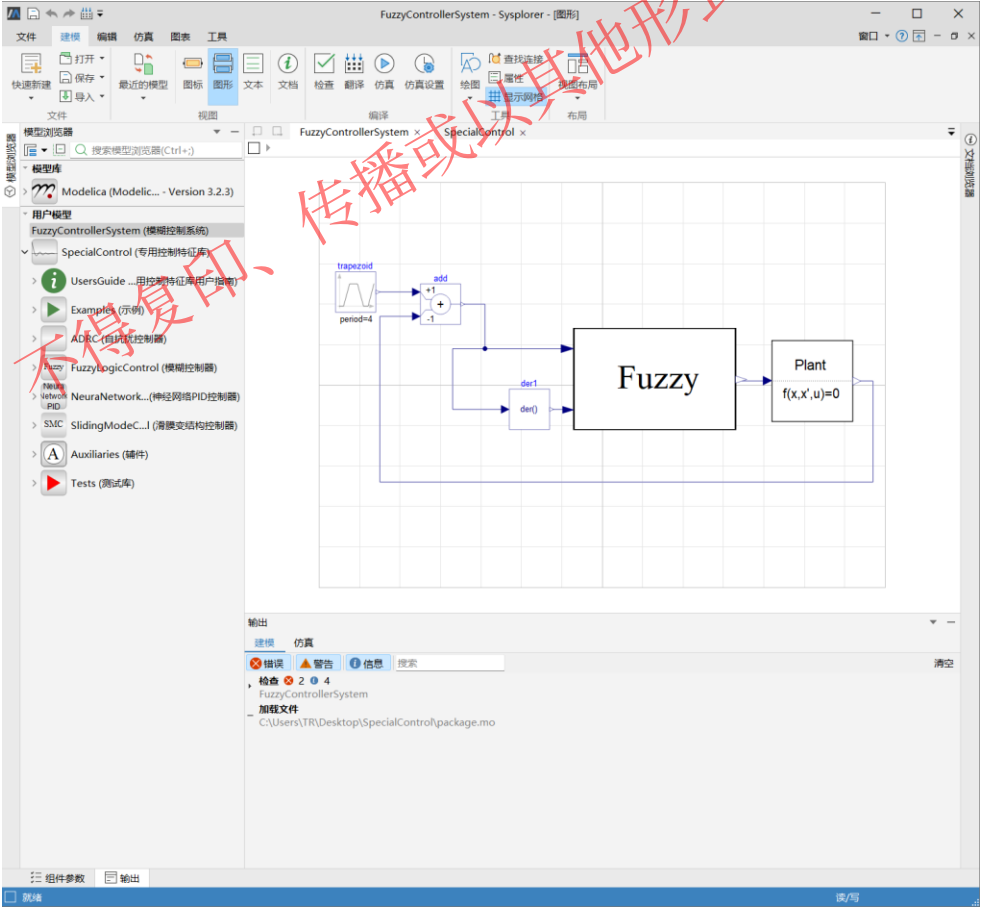
5. MWORKS.Sysplorer与Simulink之间的差异

6. MWORKS.Sysplorer与Syslab双向融合

3.1 图形化建模一般流程-模型库加载



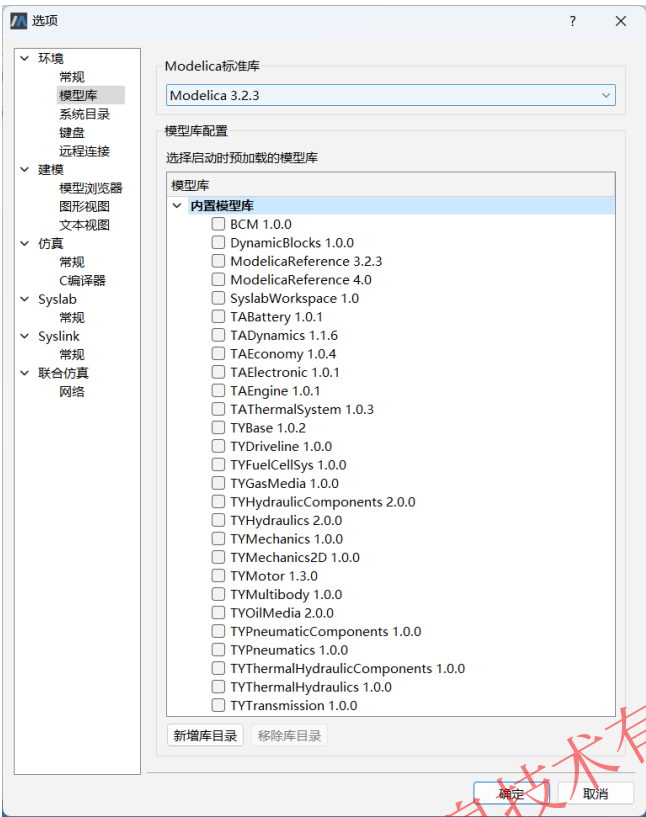
这是因为：
没有加载配套模型库



有哪些方式可以加载模型库？

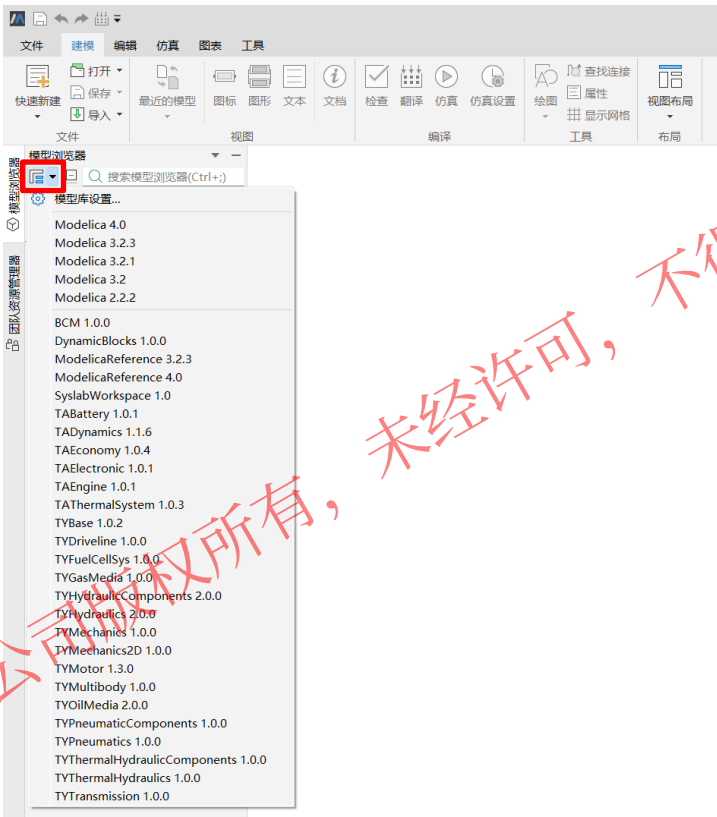
3.1 图形化建模一般流程-模型库加载

三种方式:



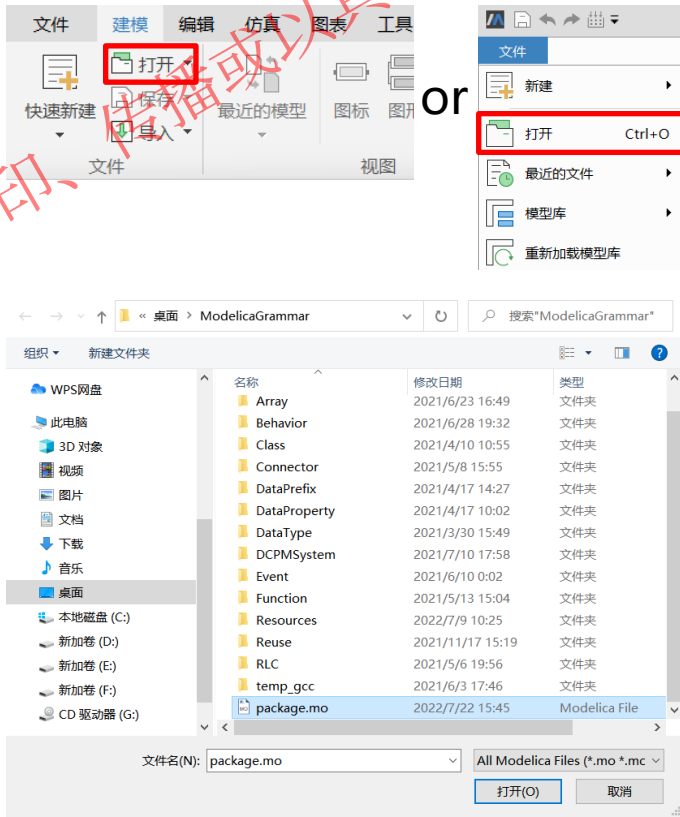
预加载模型库

每次打开软件自动打开预加载模型库
模型库不可修改



打开模型库

临时打开内置模型库
模型库不可修改



打开模型库

临时打开自定义模型库
模型库可修改

3.2 图形化建模一般流程-图形化建模(单摆)

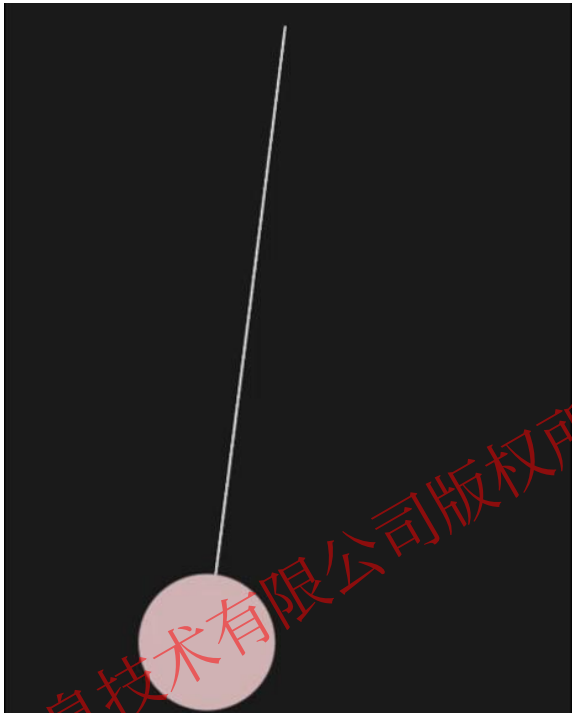


@苏州同元软控信息技术有限公司
版权所有，侵权必究



3.2 图形化建模一般流程-图形化建模(单摆)

物理对象分析：根据单摆物理结构和原理，自顶向下进行系统分解，分析系统的组成



自顶向下

系统分解

机构	固定副
	转动副
	杆件
边界条件	重力
	摩擦力

所有组件在Modelica标准库中均已有。

摆杆受重力的影响，绕固定点左右摆动，由于受摩擦力影响，摆杆逐渐在竖直方向停止。

3.2 图形化建模一般流程-图形化建模(单摆)



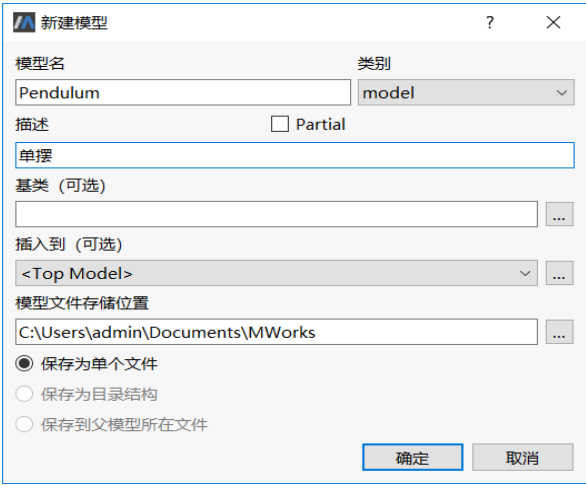
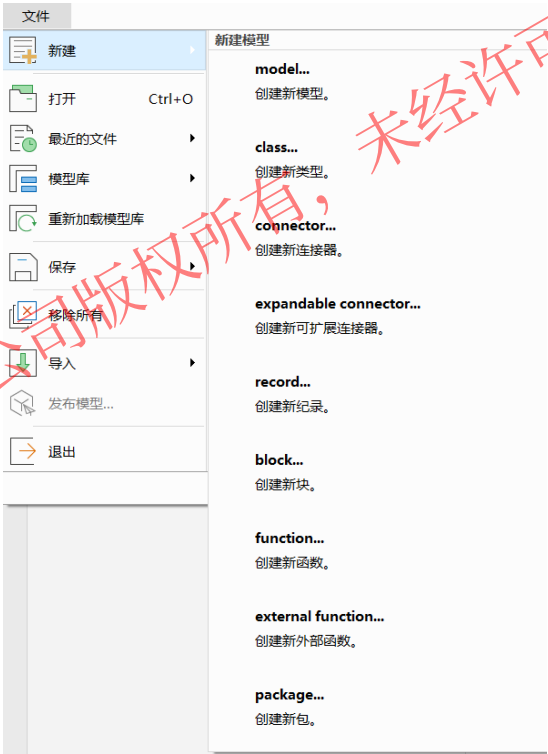
@苏州同元软控信息技术有限公司
版权所有，侵权必究



3.2 图形化建模一般流程-图形化建模(单摆)



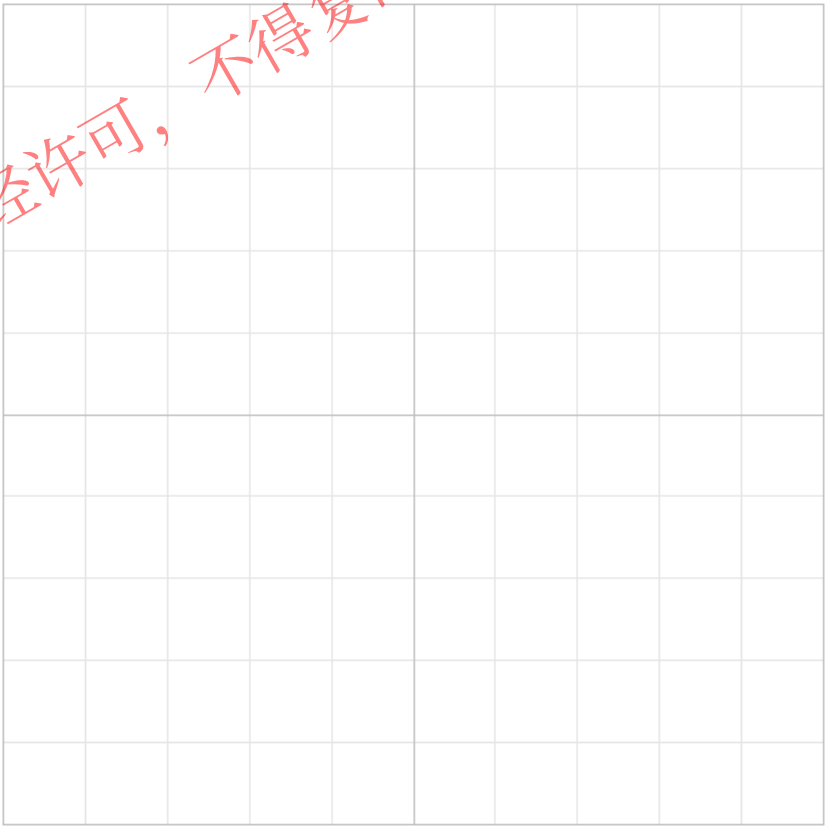
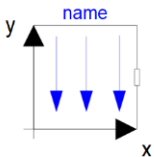
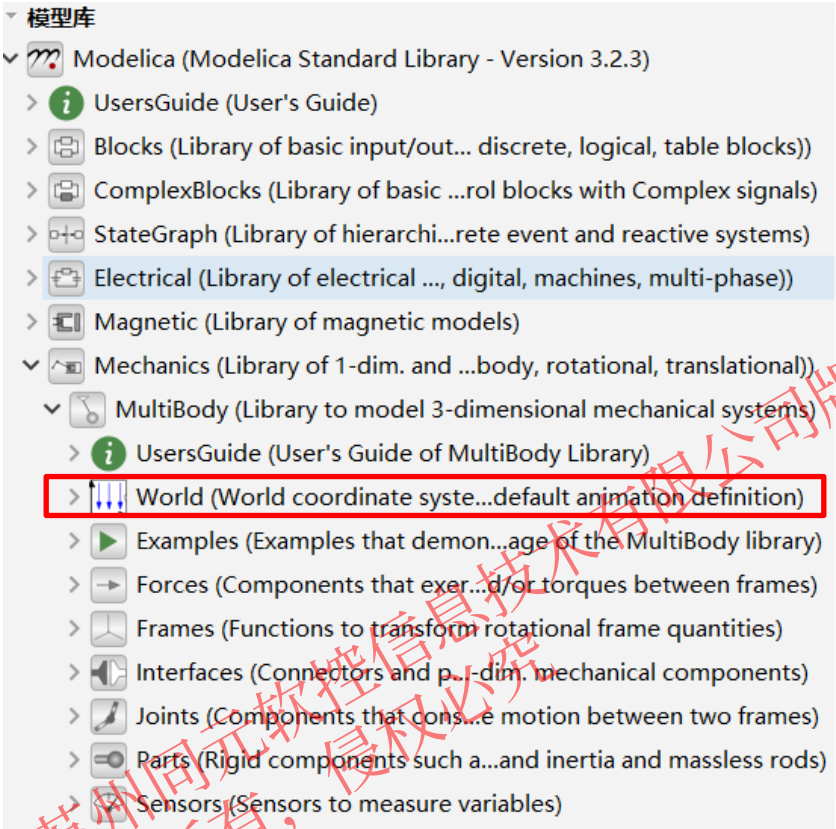
- 操作步骤:**
- 点击 “文件>新建>model...” ， 弹出 “新建模型” 对话框
 - 填写模型名为 “Pendulum” ， 描述为 “单摆”
 - 点击确定， 即可完成模型创建



3.2 图形化建模一般流程-图形化建模(单摆)



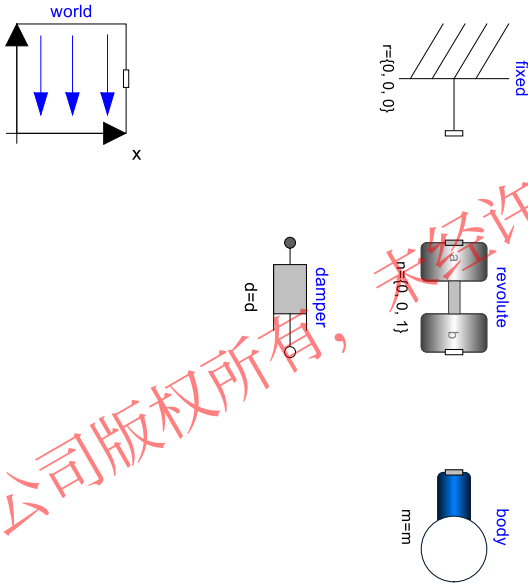
- 操作步骤：
- 选中Modelica标准库模型Modelica.Mechanics.MultiBody.World
 - 左键长按拖拽至右侧图形视图中



3.2 图形化建模一般流程-图形化建模(单摆)



操作步骤： 同"World"模型，拖入单摆模型所需要的其他组件，并根据物理拓扑关系适当排布。



组件路径如下：

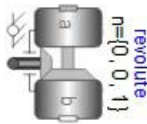
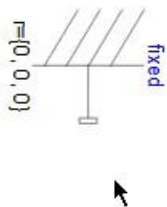
Modelica.Mechanics.MultiBody.Parts.Fixed
Modelica.Mechanics.MultiBody.Parts.Body

Modelica.Mechanics.MultiBody.Joints.Revolute
Modelica.Mechanics.Rotational.Components.Damper

3.2 图形化建模一般流程-图形化建模(单摆)



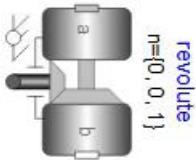
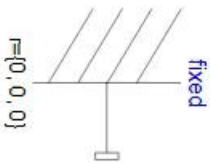
操作步骤： 选择接口，拖拽连接组件。



拖拽连接组件.mp4

操作步骤：

智能连接



智能连接组件.mp4

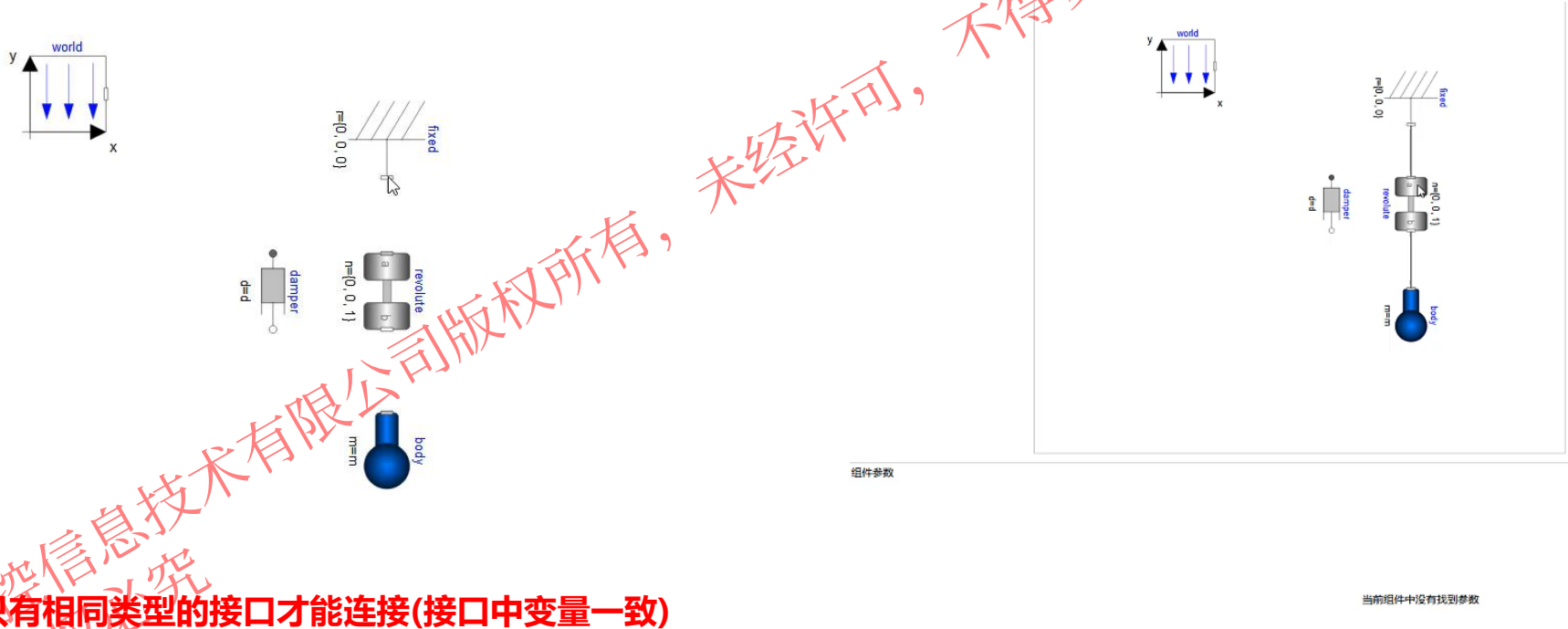
注：

- 需要在“选项”中“图形视图”里打开“移动端口时自动连接”
- “点击端口时自动连接”：依次选择两个接口，即可完成两个接口连接

3.2 图形化建模一般流程-图形化建模(单摆)



操作步骤： 按照上述方法依次连接组件的各个端口



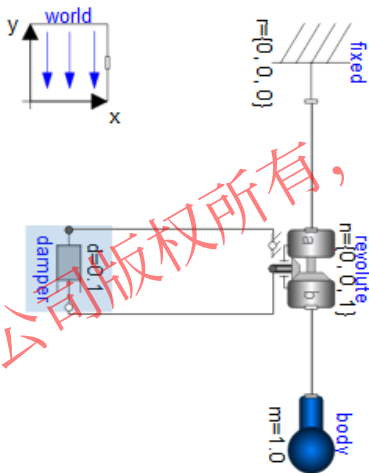
注意： 只有相同类型的接口才能连接(接口中变量一致)

错误✖：不同类型接口连接

3.2 图形化建模一般流程-图形化建模(单摆)



操作步骤：根据单摆的系统属性，设置模型参数。
如：转动副具有摩擦系数为 $0.1\text{N}\cdot\text{m}/(\text{rad}/\text{s})$ 的摩擦力,选中组件dampner，在组件参数面板中选中参数“d” 的值，修改为0.1，**需注意单位**。



组件参数			
Advanced			
参数			
phi_rel.start	0	deg	Relative rotation angle (= flange_b.phi - flange_a.phi)
w_rel.start	0	rad/s	Relative angular velocity (= der(phi_rel))
a_rel.start	0	rad/s2	Relative angular acceleration (= der(w_rel))
useHeatPort	☐		=true, if heatPort is enabled
d	0.1	N.m.s/r...	Damping constant

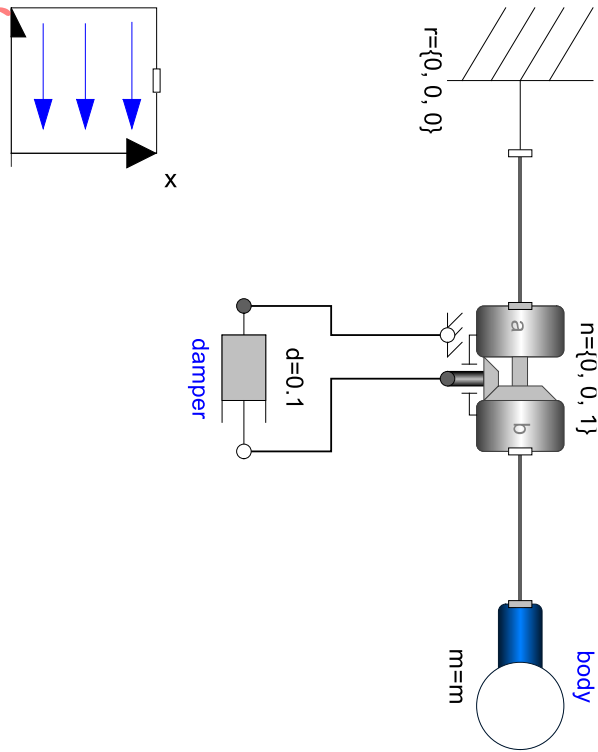
3.2 图形化建模一般流程-图形化建模(单摆)



操作步骤：参考组件 “damper” 的参数 “d” 的修改方法，修改其余组件参数

组件	参数	参数值	量纲
revolute	phi.start(初始角度)	50	deg
	useAxisFlange(是否使用外置接口)	True	/
body	r_cm(质心位置)	{0.5, 0, 0}	m
	m(质量)	1.0	kg
damper	d(阻尼)	0.1	N.m.s/rad

最终呈现：

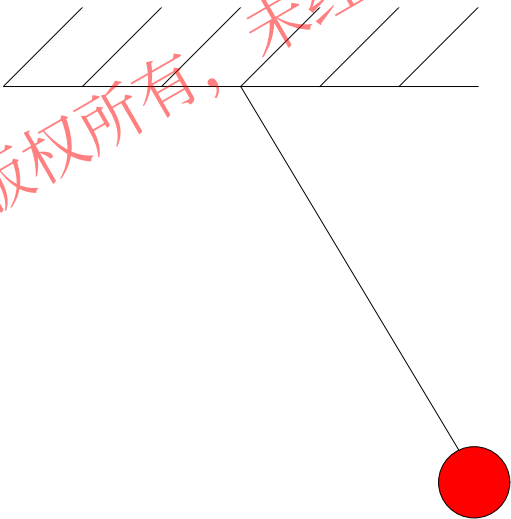


3.2 图形化建模一般流程-图形化建模(单摆)

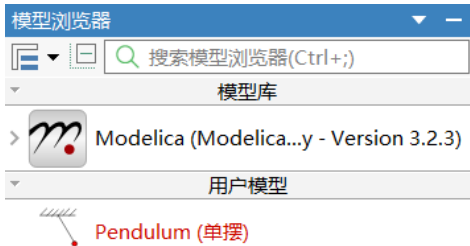


模型构造完成之后，通过图标绘制可以让模型更加**直观**

- 操作步骤：**
- 选择图标界面
 - 使用“建模”或“编辑”的基本图元进行绘制
 - 在图形属性中，为图形添加填充颜色及样式



给模型穿上“漂亮的衣服”



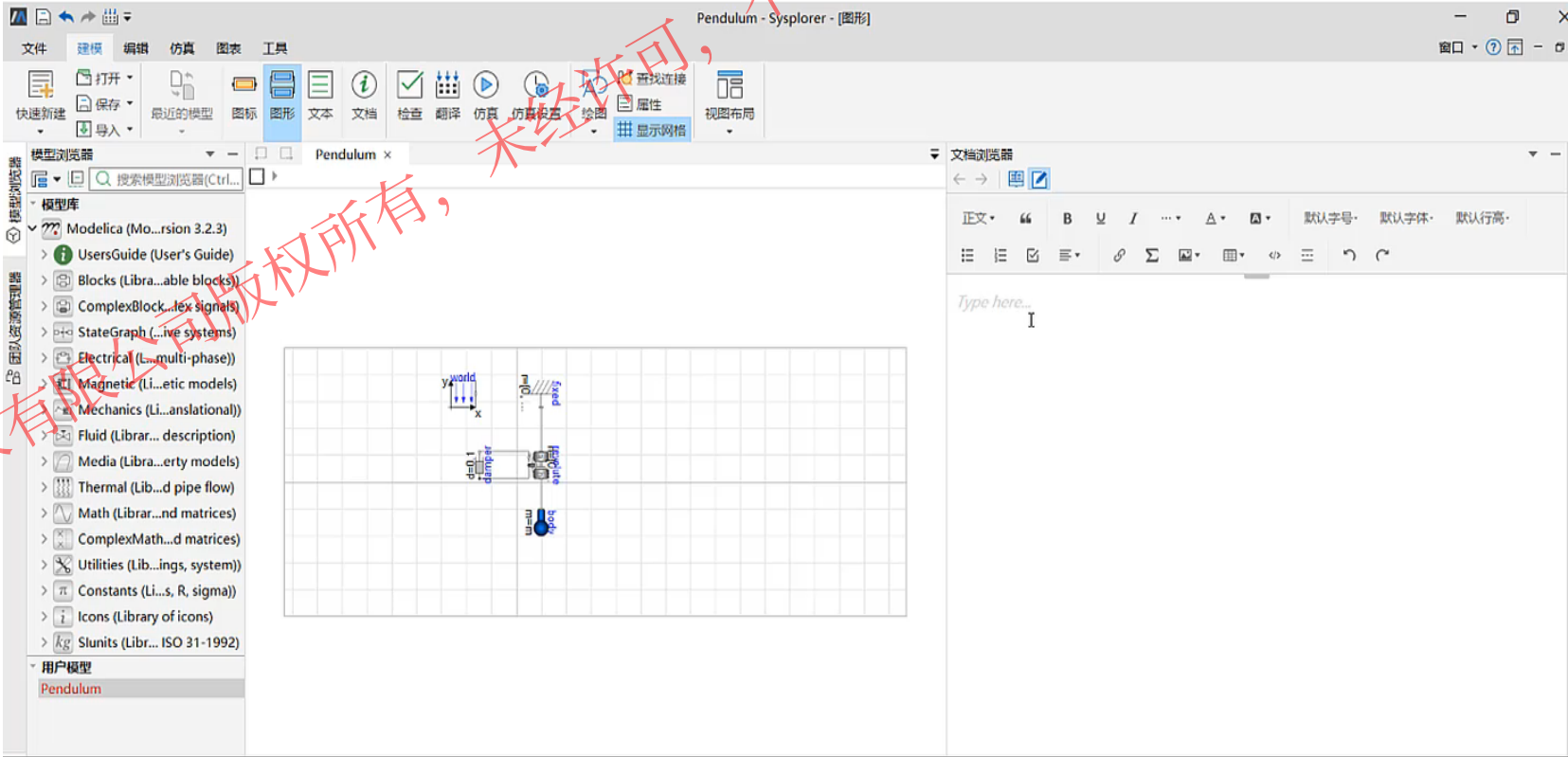
3.2 图形化建模一般流程-图形化建模(单摆)



模型构造完成之后，通过编辑文档可以让模型更加易用

操作步骤：

- 在模型对应的文档浏览器中进入编辑模式。
- 在文档浏览器中插入文字。
- 在文档浏览器中插入图片。
- 在文档浏览器中插入链接。



3.2 图形化建模一般流程-图形化建模(单摆)



@苏州同元软控信息技术有限公司
版权所有，侵权必究

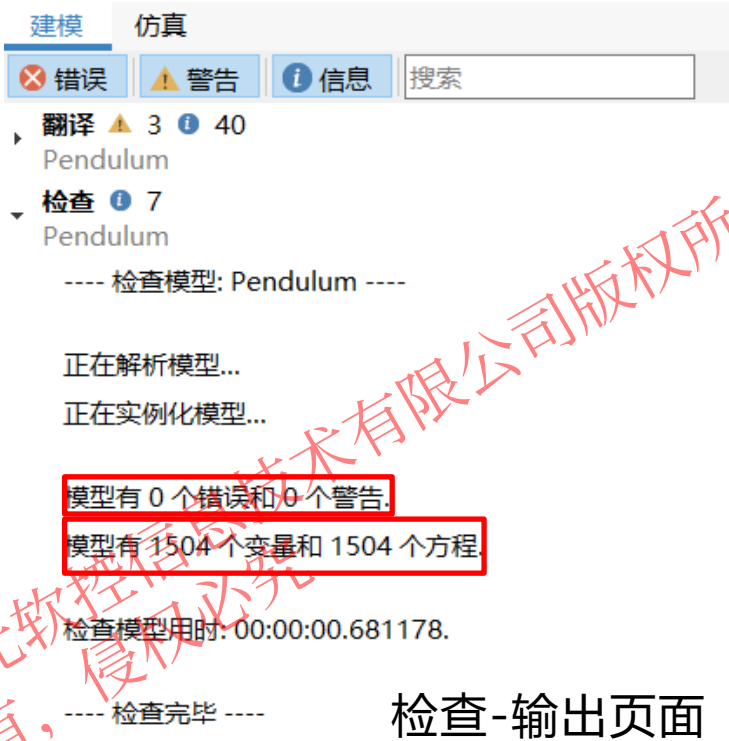


3.2 图形化建模一般流程-图形化建模(单摆)



通过模型检查，分析模型是否存在错误

操作步骤：点击建模菜单的“检查”



检查模型是否可以仿真条件:

- 判断模型中是否存在语法或其他错误，错误信息必须修改，警告仅起提示作用，可不修改。
- 检查模型是否完备(变量数和方程数一定要相等)，只有模型完备才有可能进行仿真。

3.2 图形化建模一般流程-图形化建模(单摆)



通过模型编译，分析模型是否并转化为可执行文件

操作步骤：点击建模（仿真）菜单的“翻译”

- 翻译信息：
- 模型内变量数=方程数
 - 常量
 - 参量
 - 变量
 - 微分变量
 - 方程
 -

建模 仿真

错误 警告 信息 搜索

翻译 1 35

Pendulum

翻译 35

Pendulum

---- 检查模型: Pendulum ----

正在解析模型...

正在实例化模型...

模型有 0 个错误和 0 个警告.

模型有 1504 个变量和 1504 个方程.

检查模型用时: 00:00:00.332159.

---- 检查完毕 ----

---- 翻译模型: Pendulum ----

正在解析模型...

正在实例化模型...

原始模型

常量:1

参量:172

变量:1504

微分变量:13

方程:1504

翻译-输出页面

文件 建模 编辑 仿真 图表 工具

文件 建模 编辑 仿真 图表 工具

翻译 仿真 暂停 停止 仿真设置

停止: 5

算法: Dassl

同步 1

32位

64位

输出

建模 仿真

错误 警告 信息 搜索

翻译 35

Pendulum

翻译 1 6

Pendulum

---- 检查模型: Pendulum ----

正在解析模型...

正在实例化模型...

错误(4259): 断言失败:assert(cardinality(frame_a) > 0, "Connector frame_a of revolute joint is not connected").

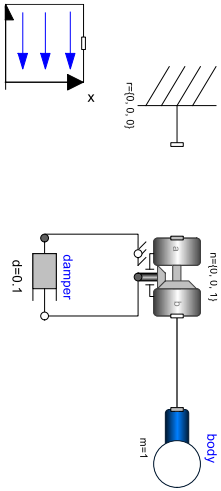
实例层次为"revolute".

模型有 1 个错误和 0 个警告.

---- 检查发现错误终止 ----

模型"Pendulum"编译失败。

存在错误翻译-输出页面

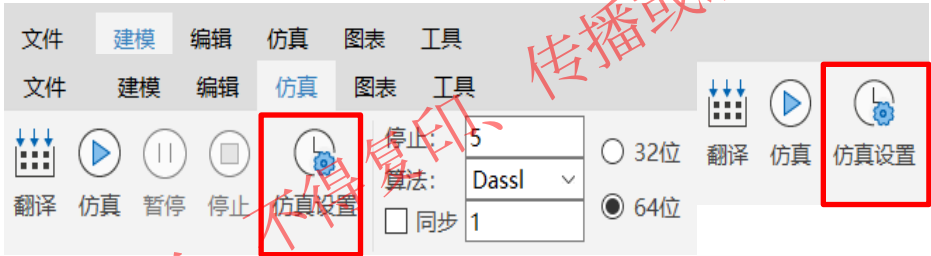


3.2 图形化建模一般流程-图形化建模(单摆)



仿真设置-根据分析需求选择仿真时长、积分算法等

操作步骤：点击建模（仿真）菜单的“仿真设置”



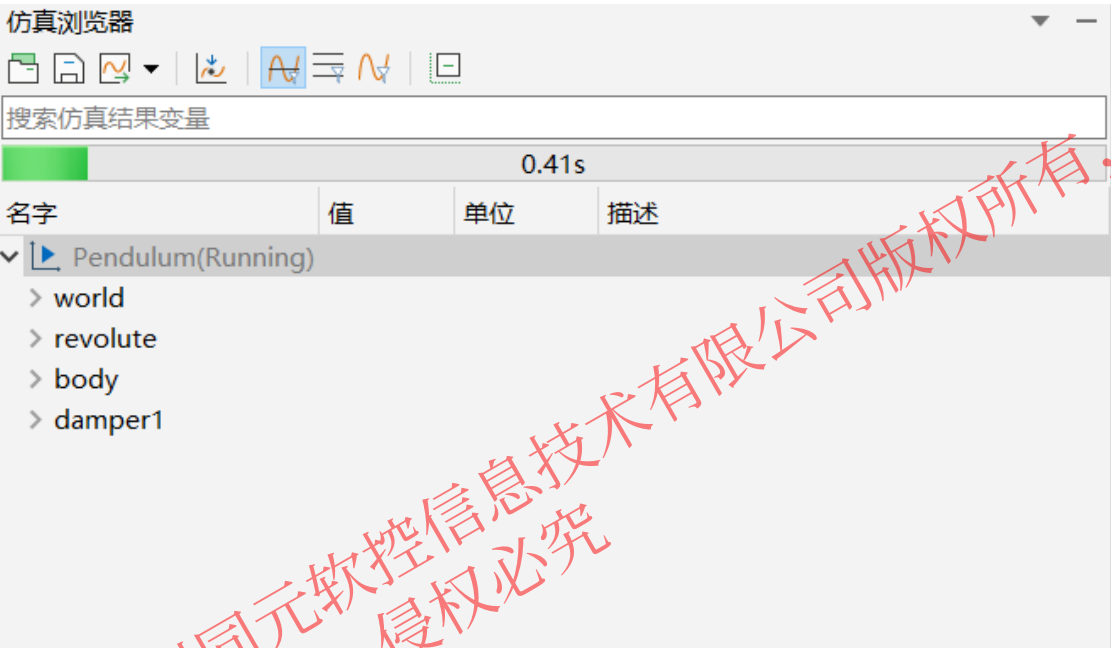
- **仿真区间**：仿真开始/停止的时间，此处设置为**0~5s**。
- **步长**：仿真输出点之间的间隔长度，此处设置步长为**0.01**。
- **步数**：仿真生成的输出间隔的数目。
- **算法**：MWORKS提供**21**种不同的仿真算法进行选择，并且提供自定义算法，此处使用**Dassl**算法。
- **精度**：指定每个仿真步长的局部精度。
- **积分步长**：选择变步长算法时为初始积分步长，选择定步长算法时为固定积分步长。
- **保存到模型**：对于可修改的模型，可以将仿真设置中的常规设置保存到模型中。

3.2 图形化建模一般流程-图形化建模(单摆)

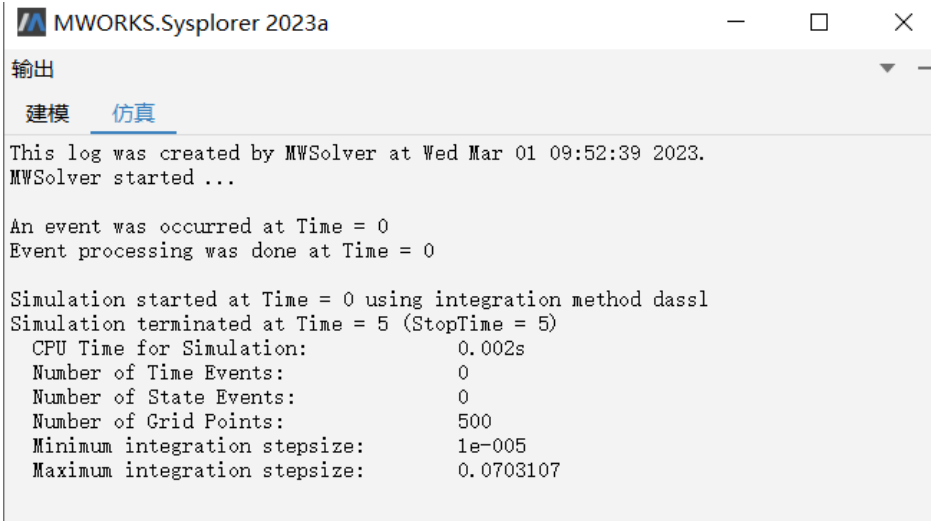


根据仿真设置系统调用模型翻译生成的求解器，计算模型中所有变量随时间变化的数据

操作步骤： 点击建模（仿真）菜单的“仿真”



仿真浏览器可查看模型仿真进度



仿真结束后可查看模型输出信息

3.2 图形化建模一般流程-图形化建模(单摆)

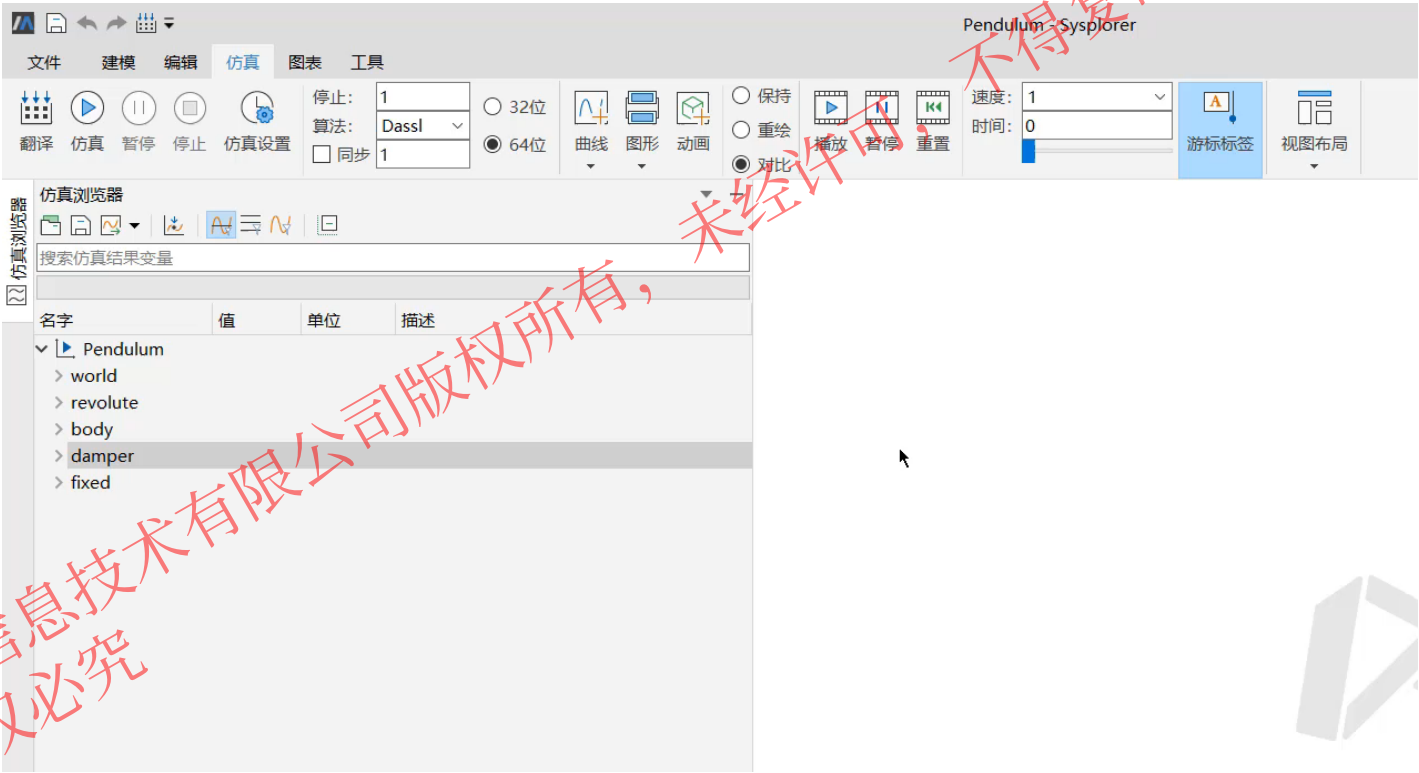


3.2 图形化建模一般流程-图形化建模(单摆)



通过模型的图形界面，快速定位目标变量，便于变量查找

操作步骤： 鼠标左键点击图形界面的组件，仿真浏览器自动定位该组件变量。

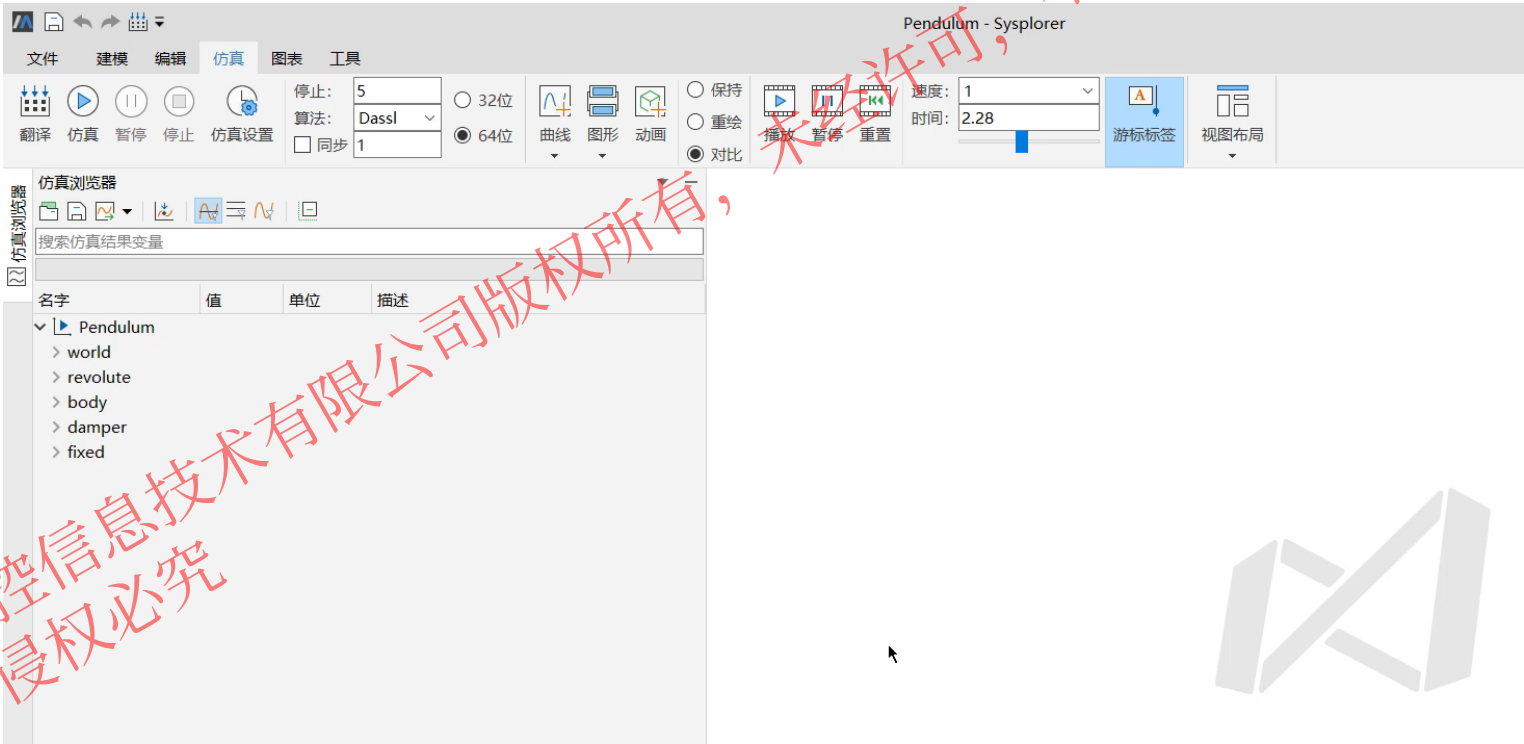


3.2 图形化建模一般流程-图形化建模(单摆)



y(t) 曲线窗口-以时间(time) 作为横坐标，查看模型中变量与仿真时间的关系

- 操作步骤:**
- 方式1：在仿真界面点击相关变量前面的空格或定位的相关变量拖拽至空白区域
 - 方式2：在图表界面定位的相关变量拖拽y(t)曲线窗口中或点击相关变量空格



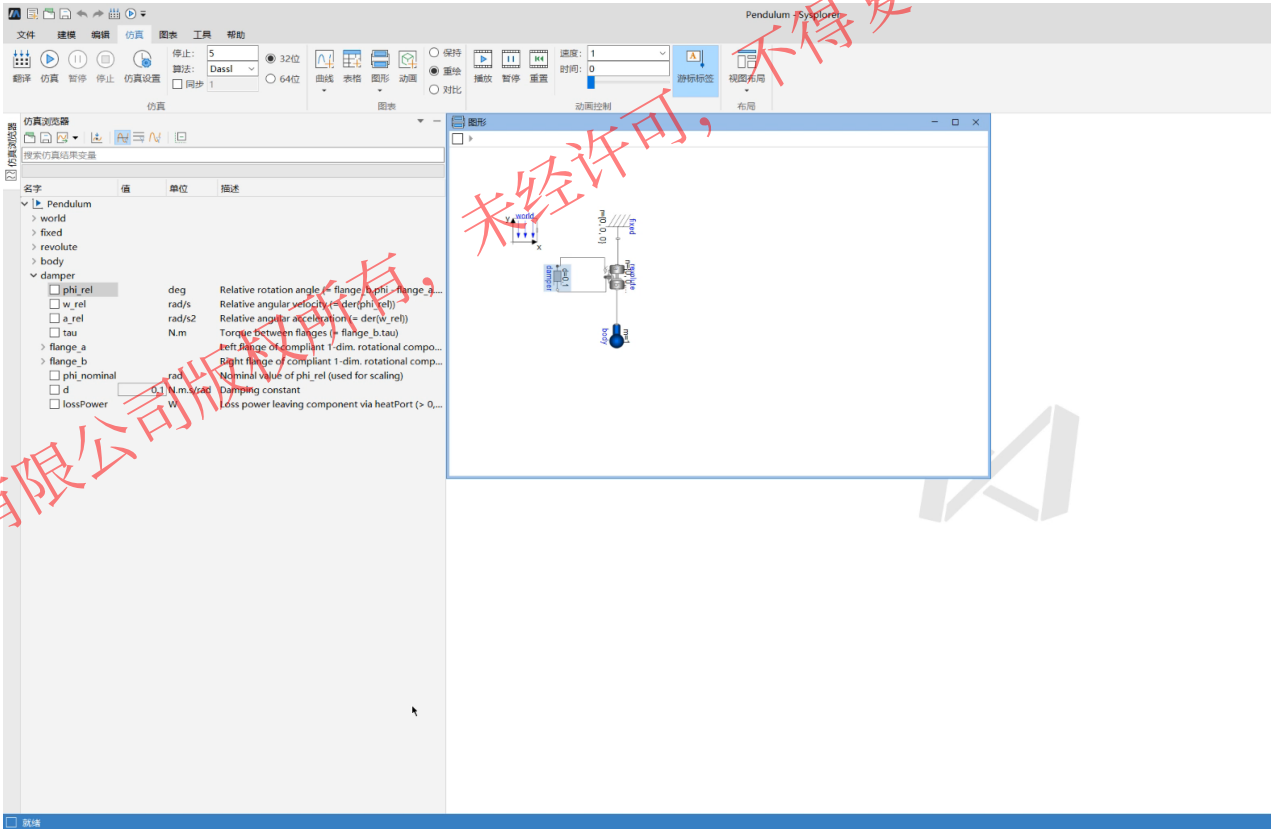
3.2 图形化建模一般流程-图形化建模(单摆)



结果表格查看-以时间(time) 作为横坐标，查看模型中变量与仿真时间的关系

操作步骤：新建表格窗口，勾选或直接拖入变量

注：
表格中的数据可直接复制至Excel
或其他表格中



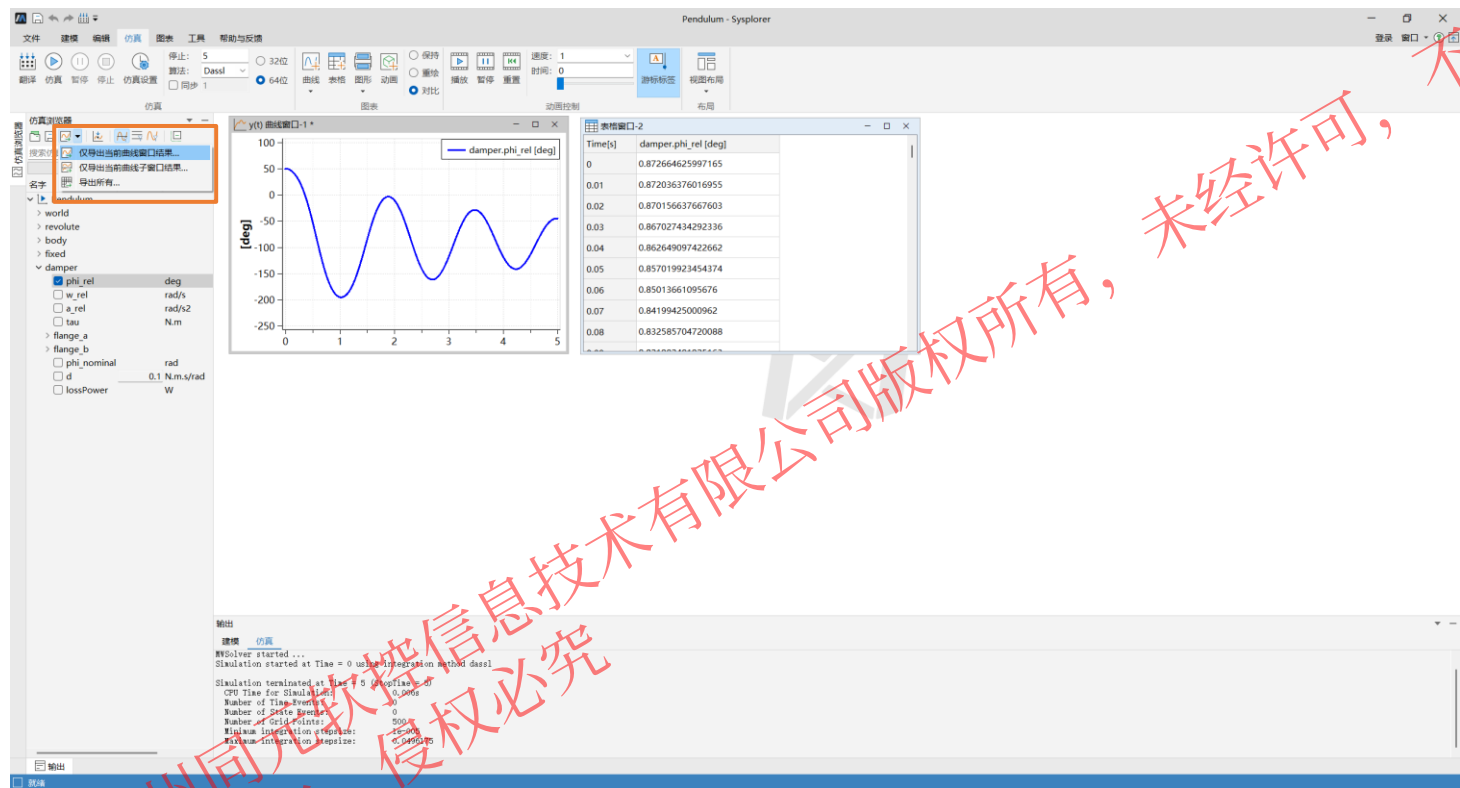
结果表格窗口.mp4

3.2 图形化建模一般流程-图形化建模(单摆)

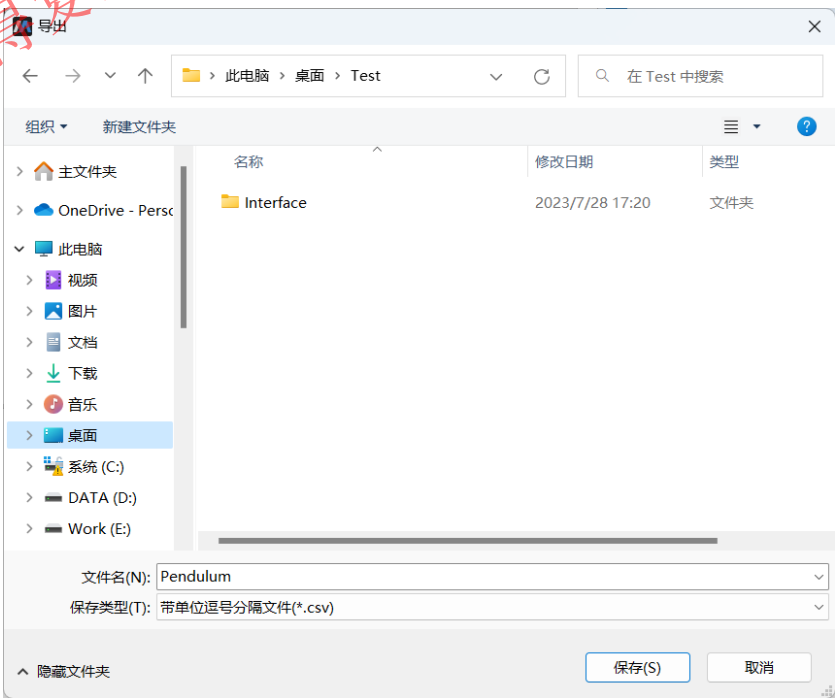


结果数据导出：可以一键导出全部仿真数据和窗口数据

操作步骤：选择导出数据方式



结果导出



结果导出选项

3.2 图形化建模一般流程-图形化建模(单摆)



y(x) 曲线窗口-以第一次拖入的变量作为横坐标，查看模型中变量与变量之间的关系

- 操作步骤:**
- 方式1：按住shift，在仿真界面定位的相关变量拖拽至空白区域
 - 方式2：在图表界面定位的相关变量拖拽y(x)曲线窗口中或点击相关变量勾选框



3.2 图形化建模一般流程-图形化建模(单摆)



曲线绘制模式（仅当重复仿真时生效）

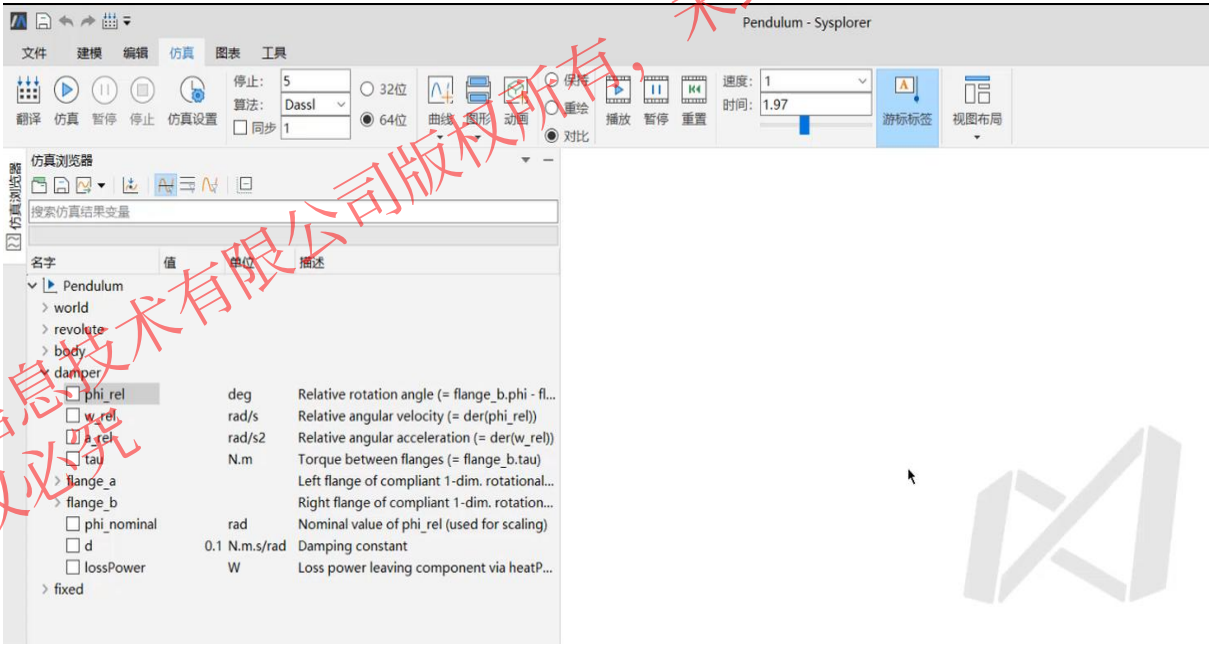
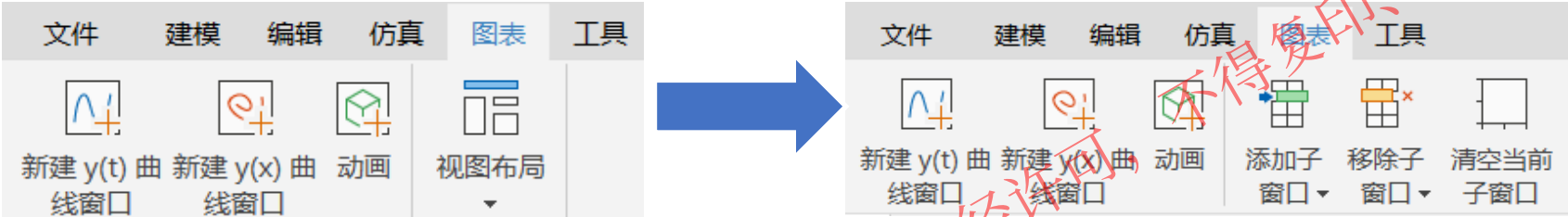


- 保持：保持当前变量曲线不变
- 重绘：基于新实例重新绘制变量曲线
- 对比：保留当前变量曲线，并基于新实例再次绘制变量曲线（不支持y(x)窗口对比）

3.2 图形化建模一般流程-图形化建模(单摆)



图表窗口下，视图区域空白时，无子窗口操作面板，新建曲线后可生成子窗口

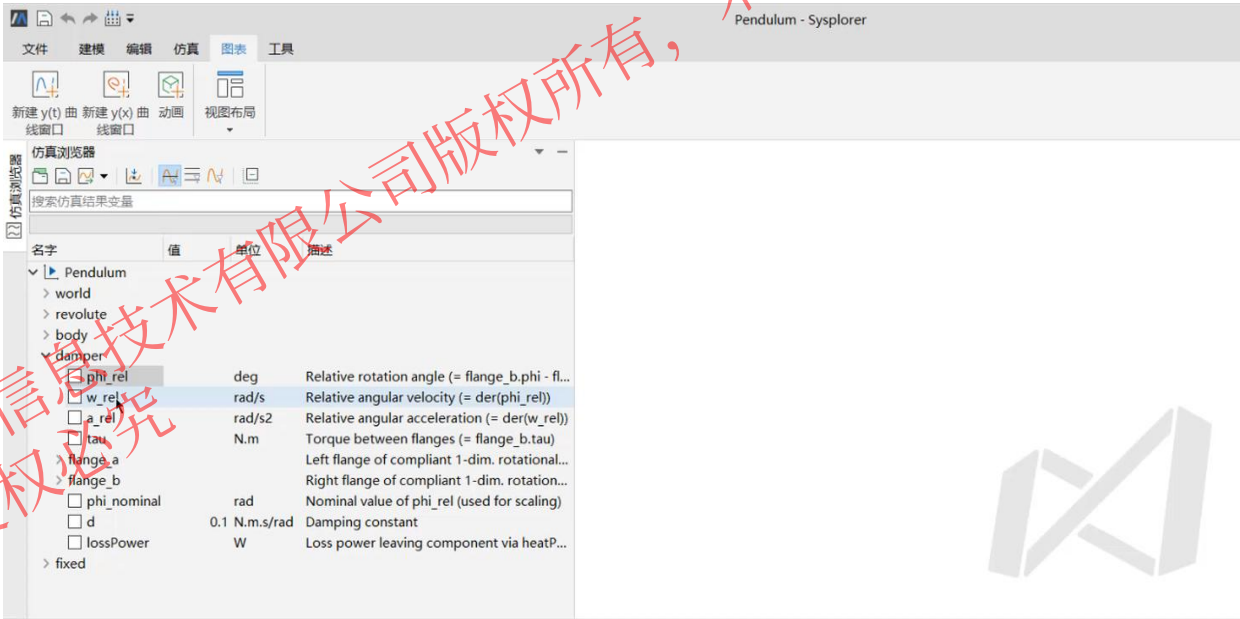
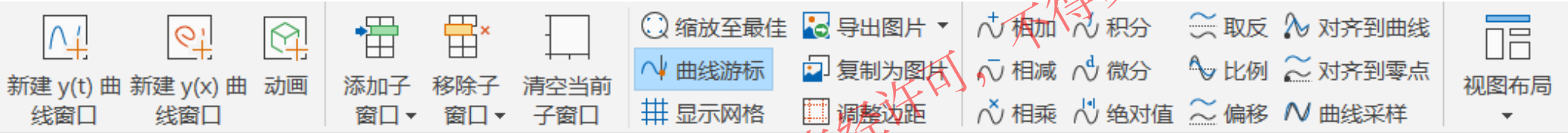


3.2 图形化建模一般流程-图形化建模(单摆)



图表窗口下，新建曲线后可通过曲线游标进行曲线查看操作

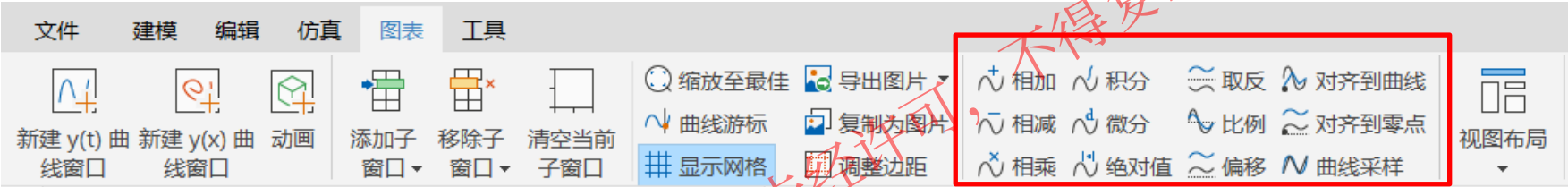
操作步骤： 鼠标左键点击图表窗口下的“曲线游标”进行曲线查看。框选局部区域放大，辅助查看值



3.2 图形化建模一般流程-图形化建模(单摆)



图表窗口下，新建曲线后可进行曲线运算操作



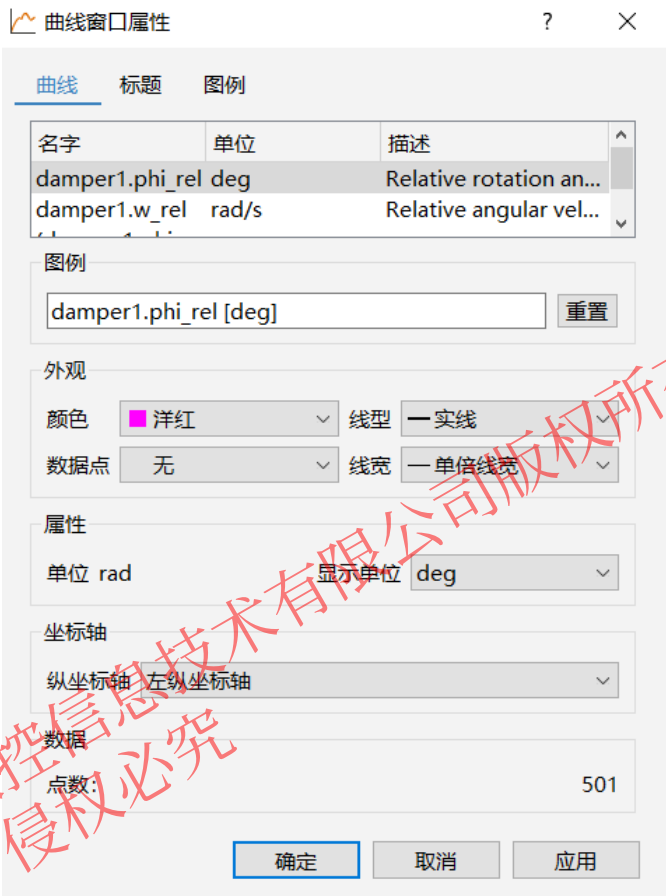
操作步骤：

- 按住ctrl键，选中同一个曲线窗口中的多条曲线，即可进行曲线运算中的相加、相减、相乘；
- 只能选择一条曲线进行积分、微分、对齐到曲线、对齐到零点、曲线采样操作；
- 选中多条曲线分别进行运算绝对值、取反、取比例、偏移操作。

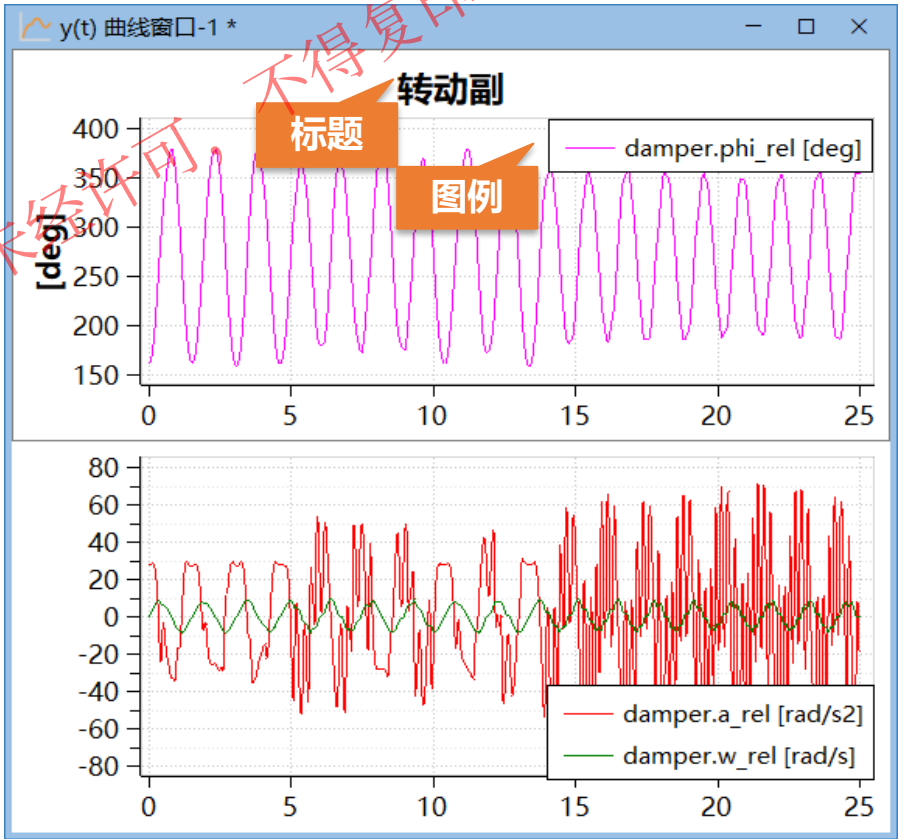
3.2 图形化建模一般流程-图形化建模(单摆)



操作步骤：右键点击曲线窗口，配置曲线窗口属性：图例名称、标题、颜色等进行设置。



曲线窗口属性



曲线设置

3.2 图形化建模一般流程-图形化建模(单摆)

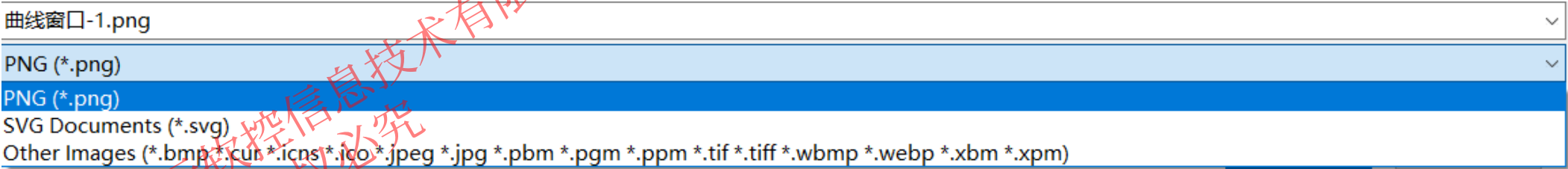


操作步骤：鼠标左键点击图表窗口下的“导出图片”进行图片的导出。



- 可以选择：
- 1.导出曲线窗口图片
 - 2.导出子窗口图片

多种图片格式选择：



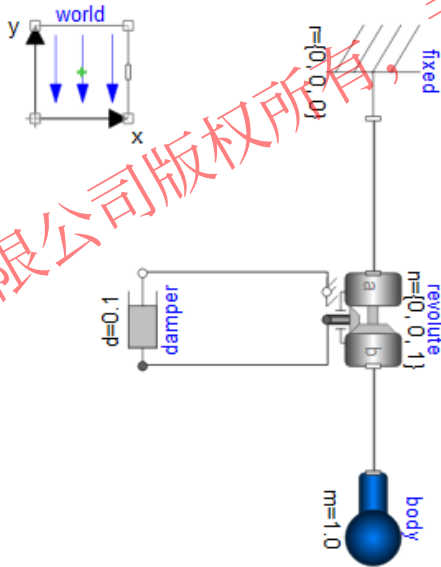
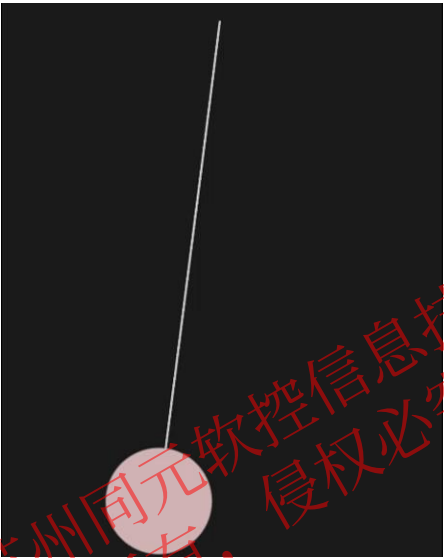
3.2 图形化建模一般流程-图形化建模(单摆)



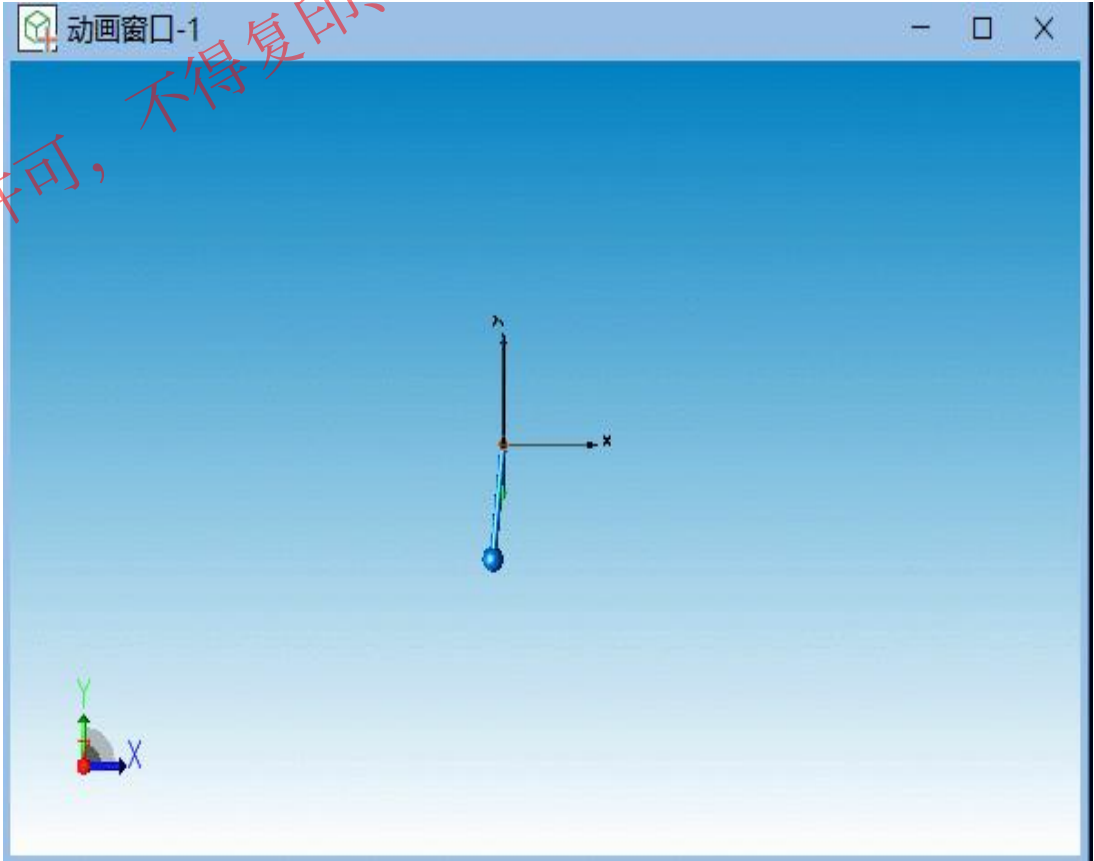
MWORKS.Sysplorer提供仿真后三维动画创建、播放的功能（只有机械多体才可以查看）



以单摆为例



单摆模型图

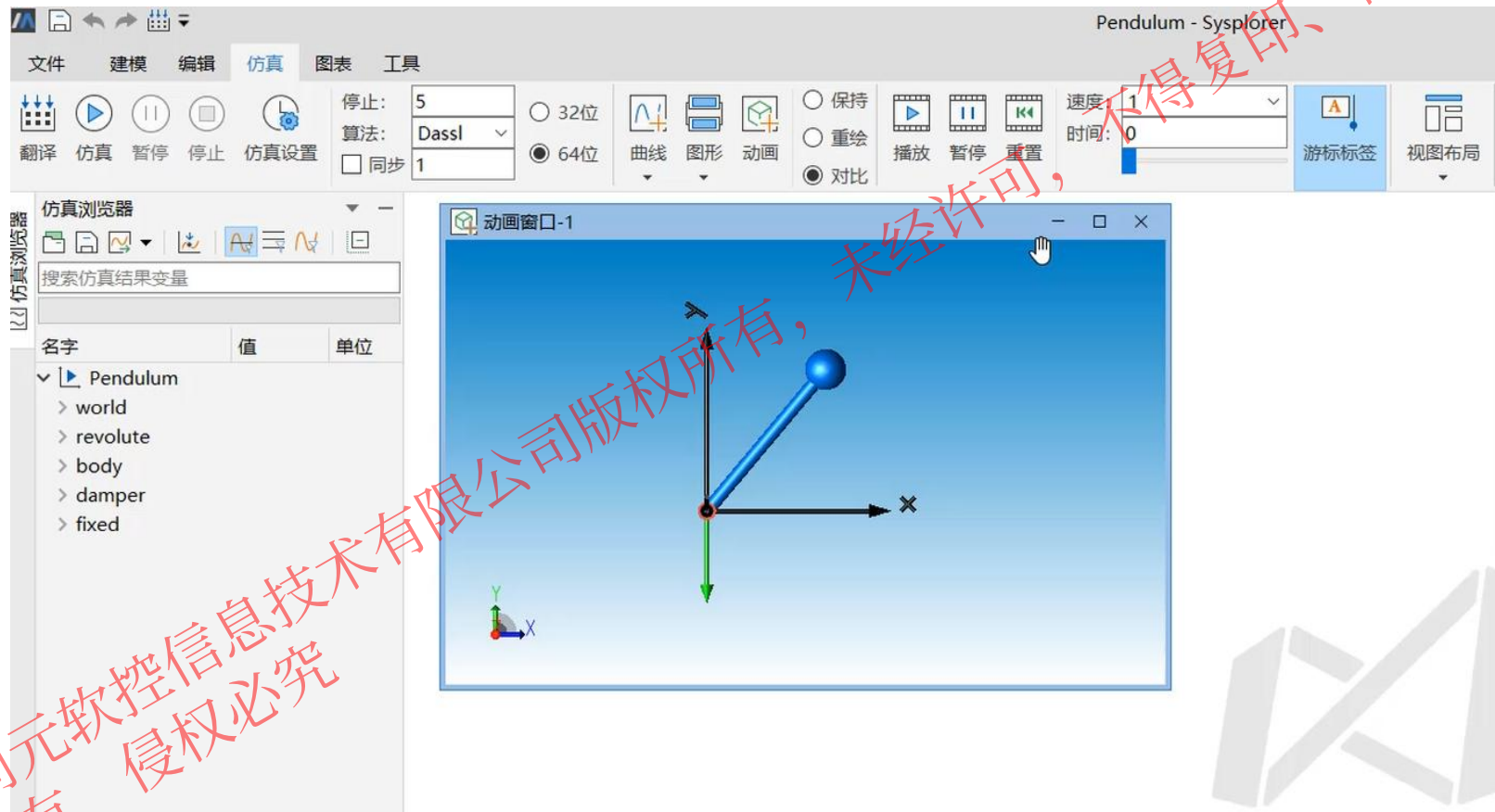


单摆动画

3.2 图形化建模一般流程-图形化建模(单摆)



动画控制支持动画播放、暂停、停止及播放速度调整等功能。



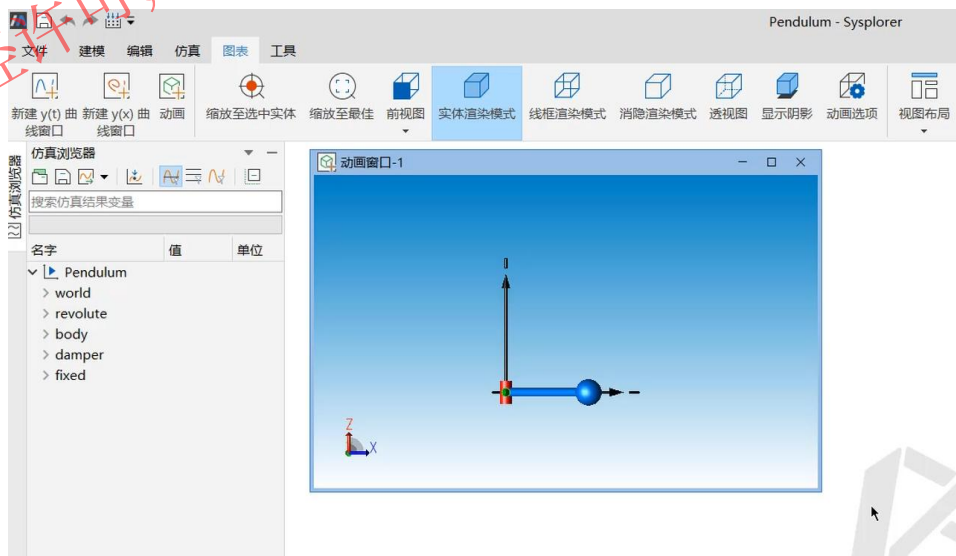
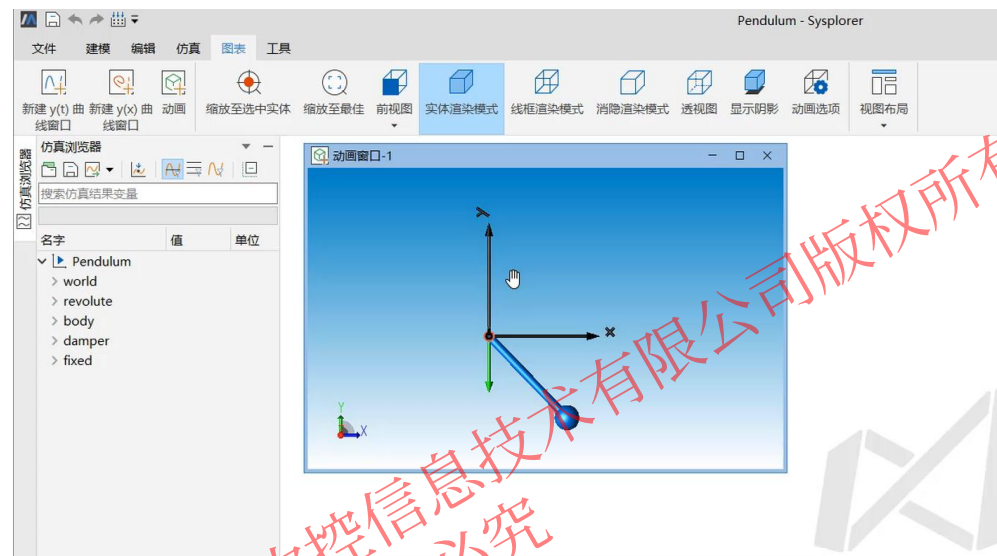
单摆动画控制

3.2 图形化建模一般流程-图形化建模(单摆)



图表工具区中的动画展示栏提供了视图切换及显示模式改变的工具：

- 视图切换：前视图、后视图、左视图、右视图、俯视图、仰视图、轴测图。
- 改变显示模式：实体渲染模式、线框渲染模式、消隐渲染模式、透视图显示阴影

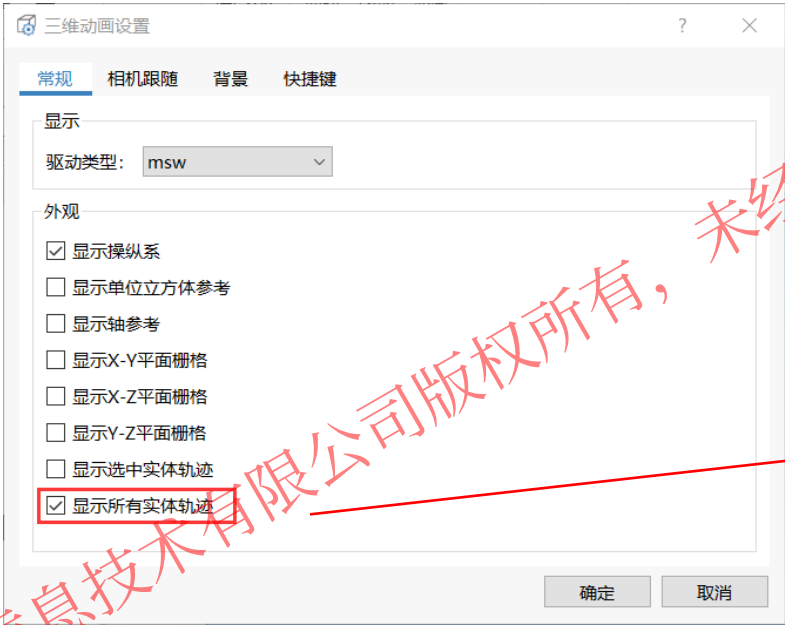


3.2 图形化建模一般流程-图形化建模(单摆)

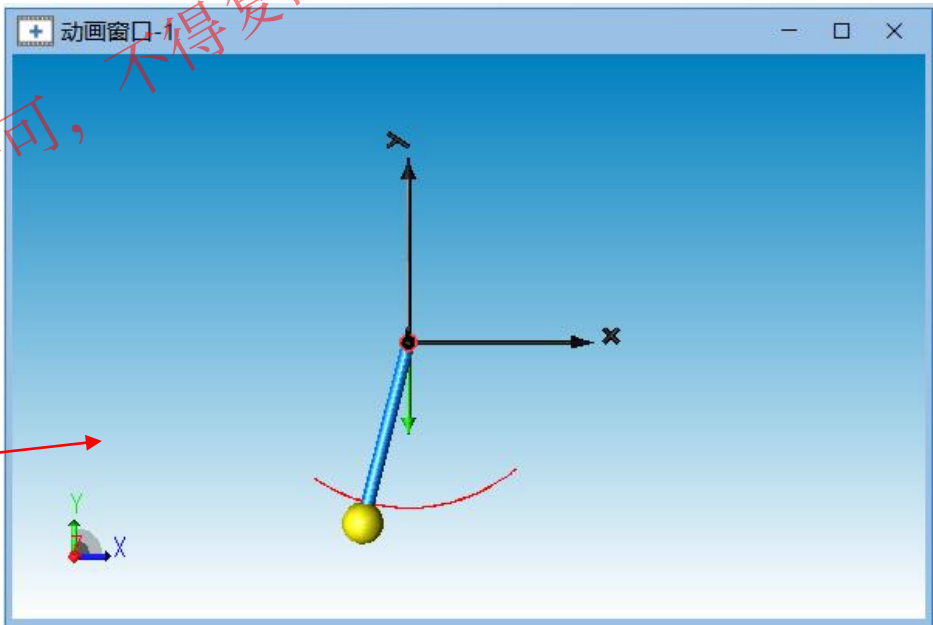


动画窗口配置:

- 外观
- 相机跟随
- 背景
- 窗口快捷键



三维动画设置窗口



单摆显示所有实体轨迹

目录

1. MWOKRS.Sysplorer软件介绍

2. 软件界面简介

3. 图形化建模

4. Modelica文本建模

5. MWORKS.Sysplorer与Simulink之间的差异

6. MWORKS.Sysplorer与Syslab双向融合

4.1 Modelica代码一般结构

文本视图是一个文本编辑器，可以显示和编辑模型的Modelica文本。

行号栏

```
1 model DoublePendulum
2   "Simple double pendulum with two revolute joints and two
3
4   inner Modelica.Mechanics.MultiBody.World world annotation
5     transformation(extent = {{-100, -10}, {-80, 10}}));
6   Modelica.Mechanics.MultiBody.Joints.Revolute revolute1(usi
9   Modelica.Mechanics.Rotational.Components.Damper damper(
11     annotation (Placement(transformation(extent = {{-60, 30
12   Modelica.Mechanics.MultiBody.Parts.BodyBox boxBody1(r = {
14     annotation (Placement(transformation(extent = {{-20, -1
15
16   Modelica.Mechanics.MultiBody.Joints.Revolute revolute2(ph:
18   Modelica.Mechanics.MultiBody.Parts.BodyBox boxBody2(r = {
20
21   annotation (Placement(transformation(origin = {70.0, 0.
23   Modelica.Mechanics.MultiBody.Parts.PointMass pointMass(
26 equation
27   connect(damper.flange_b, revolute1.axis) annotation (Line
```

编辑区

- 错误定位
- 编码助手
- 语法高亮
- 打开类型定义
- 展开与折叠
- 查找和替换
- ...



4.1. Modelica代码一般结构

```
model Resistance "电阻, 带有温度接口"
//参数定义
parameter Modelica.SIunits.Resistance R = 10 "电阻值";
parameter Modelica.SIunits.Temperature T_ref = 300.15 "参考温度";
parameter Modelica.SIunits.LinearTemperatureCoefficient alpha = 0 "电阻温度系数";
//变量声明
Modelica.SIunits.Resistance R_act "实际阻值";
Modelica.SIunits.Voltage v "电压";
Modelica.SIunits.Current i "电流";
//接口实例化
PositivePin p
  annotation (...);
NegativePin n
  annotation (...);
Modelica.Blocks.Interfaces.RealInput T
  annotation (...);

equation
//机理方程
v = R_act * i;
R_act = R * (1 + alpha * (T - T_ref));
//接口方程
v = p.v - n.v;
0 = p.i + n.i;
i = p.i;
end Resistance
```

定义模型名称: model+模型名称+ “备注”

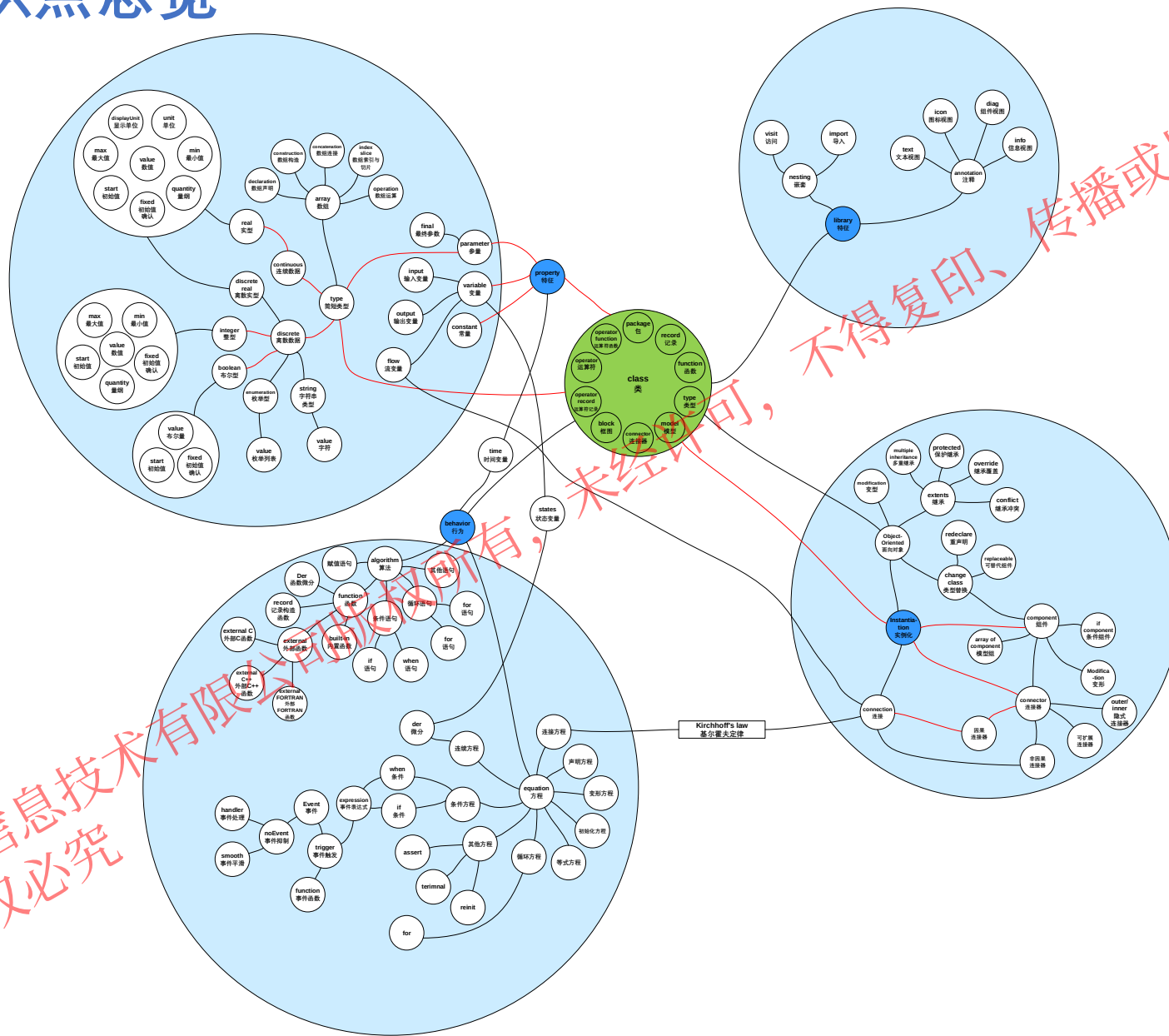
- 声明模型参数
- 声明模型变量
- 声明继承或被调用的模型 (如接口)

- 通过方程或算法, 描述模型行为

结束模型定义: end+模型名称;



4.2 Modelica知识点总览



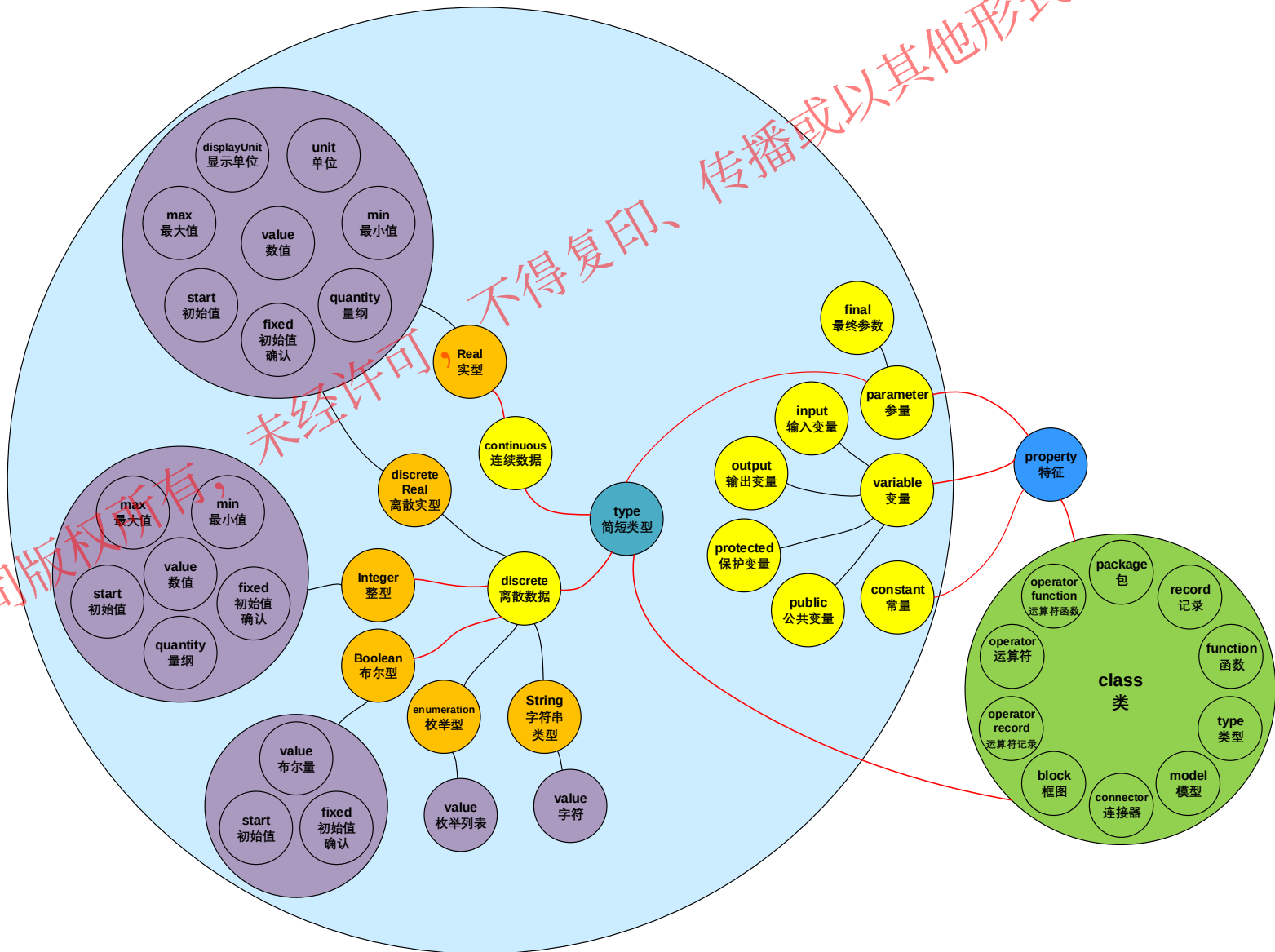
4.2 Modelica知识点总览-类及内置类型

类及内置类型—构成模型的最基本属性

Modelica语言定义了：

- 10种特化类
- 5种数据类型
- 多种前缀关键词
- 多种数据属性关键词

```
class Fall "自由落体"  
  final parameter Real g = 9.81;  
  Real h(start = 10);  
  Real v;  
equation  
  der(h) = v;  
  der(v) = -g;  
end Fall;
```



4.2 Modelica知识点总览-数组

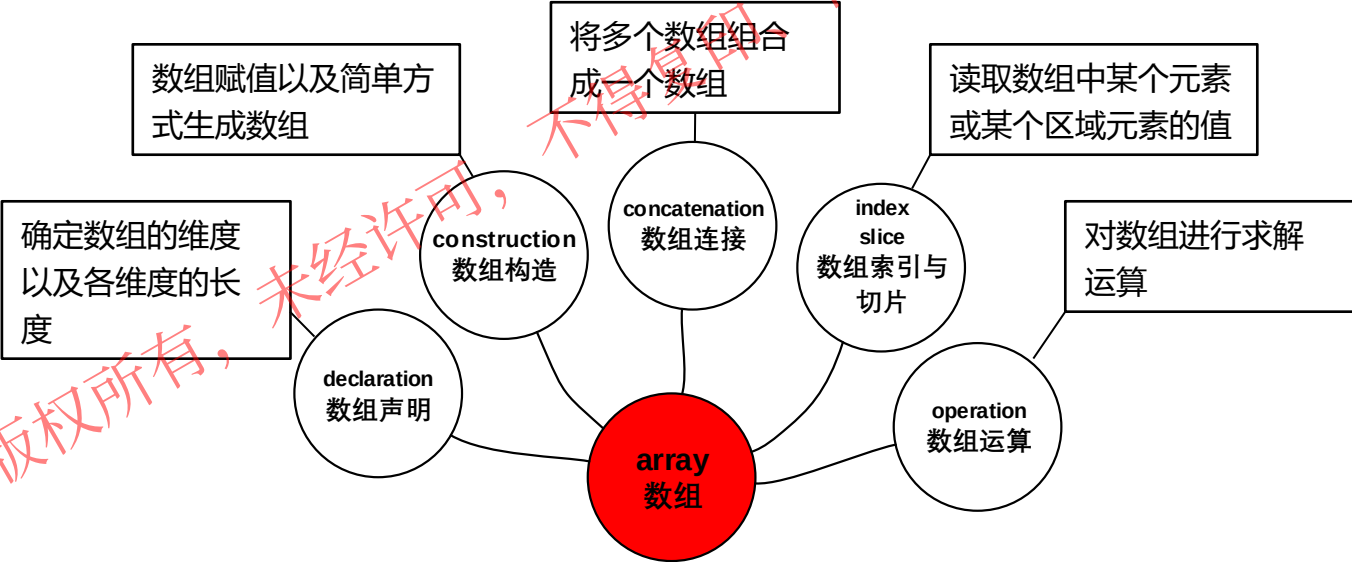
数组—多个同类型变量的集合

Modelica可定义多维数组:

- 0维数组: 标量, 如: 3.14
- 1维数组: 向量, 如:
- 2维数组: 矩阵, 如: $\begin{Bmatrix} v_x & v_y & v_z \\ 1 & 2 & 3 \\ 2 & 1 & 2 \\ 3 & 2 & 1 \end{Bmatrix}$

.....

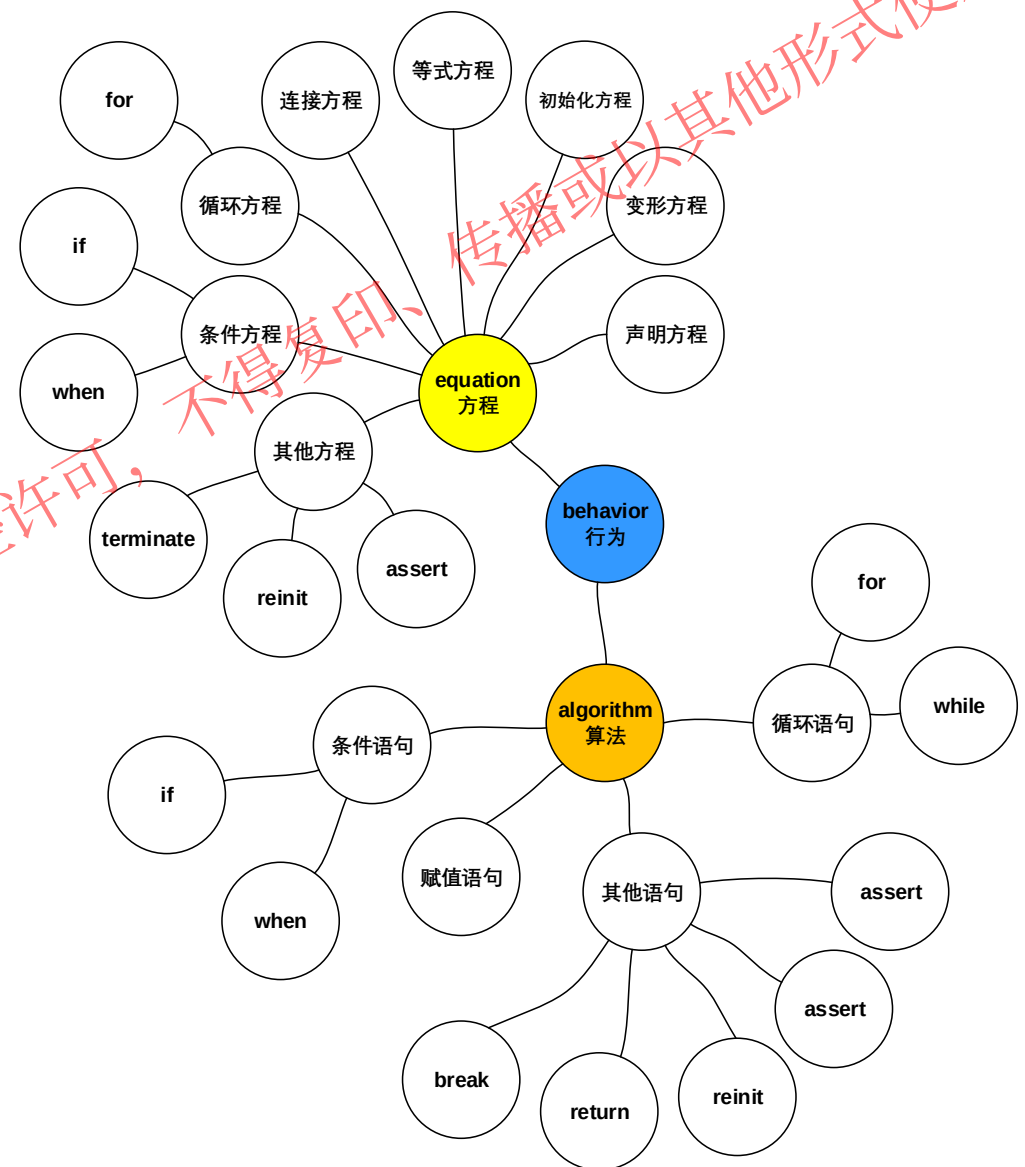
```
model FlatThrow "平抛运动"
  final parameter Real g = 9.81;
  parameter Real v0 = 1;
  Real x(start = 0);
  Real h(start = 10);
  Real position[2] = {x, h};
  Real v[2];
equation
  v[1] = v0;
  v[1] = der(x);
  v[2] = der(h);
  der(v[2]) = -g;
end FlatThrow;
```



4.2 Modelica知识点总览-模型行为描述

模型行为描述—通过方程或算法定义模型的行为

声明区域方程	声明方程		
	变形方程		
	初始化方程	initial equation	
方程区域方程	常规方程	等式方程	=
		连接方程	connect
		循环方程	for
		条件方程	if
			when
		其他方程	assert()
			terminate
			reinit()
算法区语句	算法语句	赋值语句	“:=”
		循环语句	for
			while
		条件语句	if
			when
		其他语句	break
			return
			assert
			terminate
			reinit

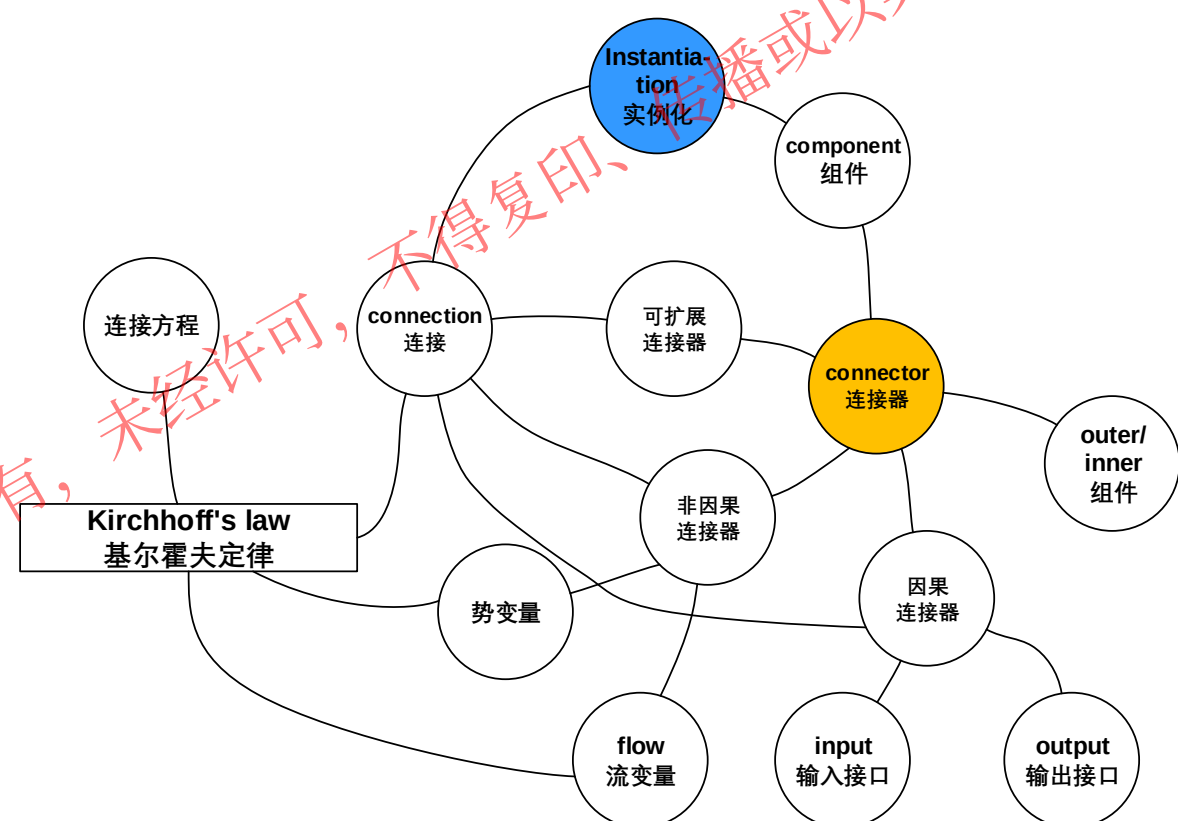
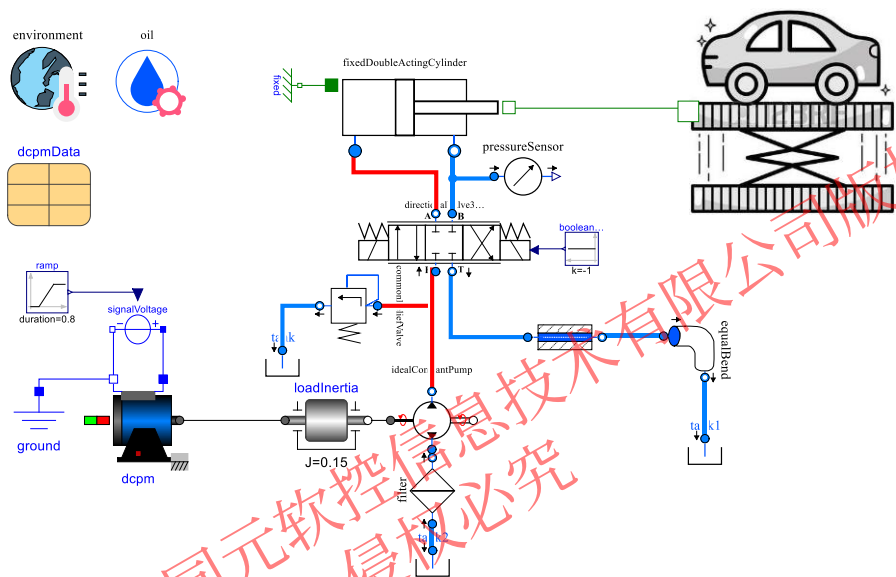


4.2 Modelica知识点总览-连接与连接器

连接与连接器—定义组件与外部的通讯

连接器：

- 因果连接器
- 非因果连接器
- 可扩展连接器
- 隐式连接器



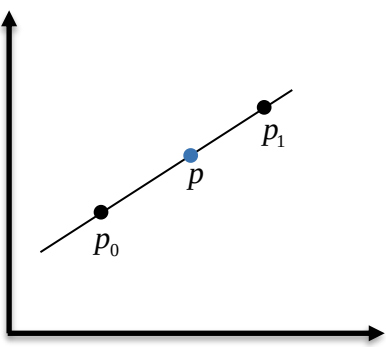
复杂系统上的多种连接器使用

4.2 Modelica知识点总览-函数

函数—数学模型的天然组成部分

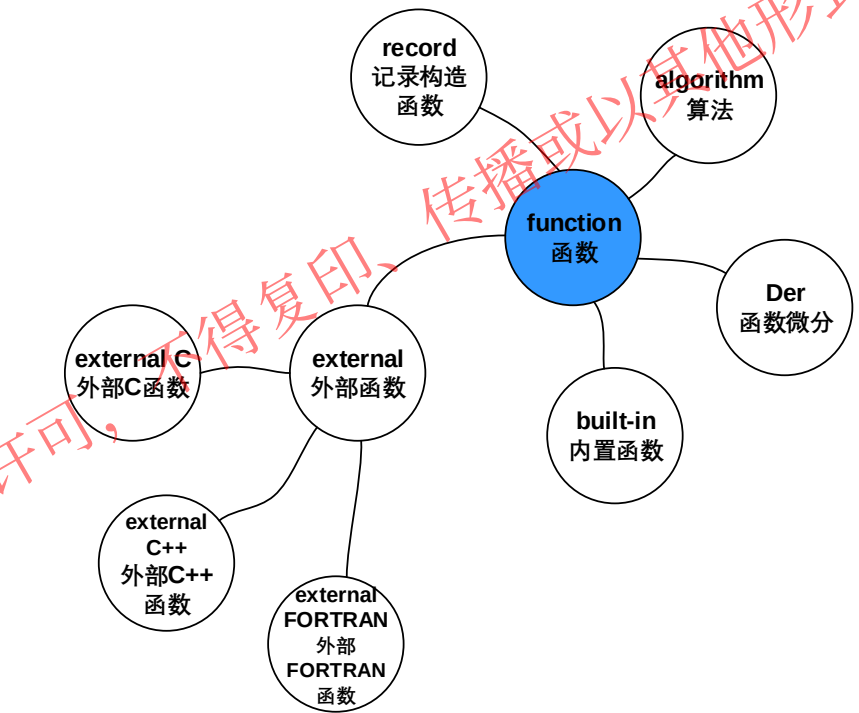
Modelica提供了：

- 多种内部函数
- 自定义函数
- C、C++、FORTRAN外部函数集成



如何计算一条直线上的点？

$$y(x, x0, y0, x1, y1) = x \frac{y1 - y0}{x1 - x0} - \frac{x1 + x0}{2} \frac{y1 - y0}{2} + \frac{y1 + y0}{2}$$



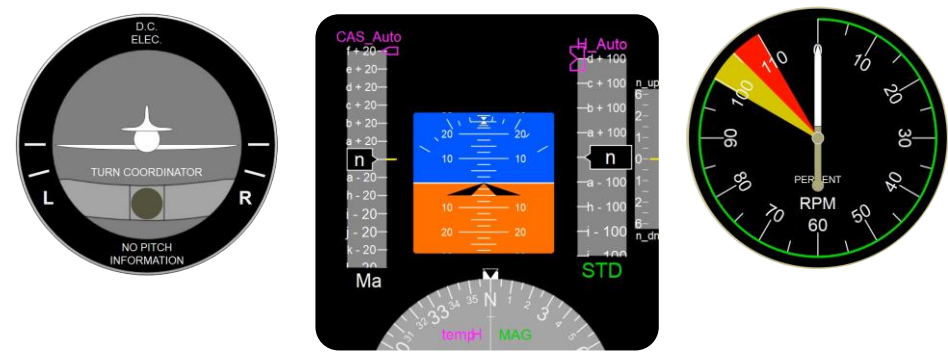
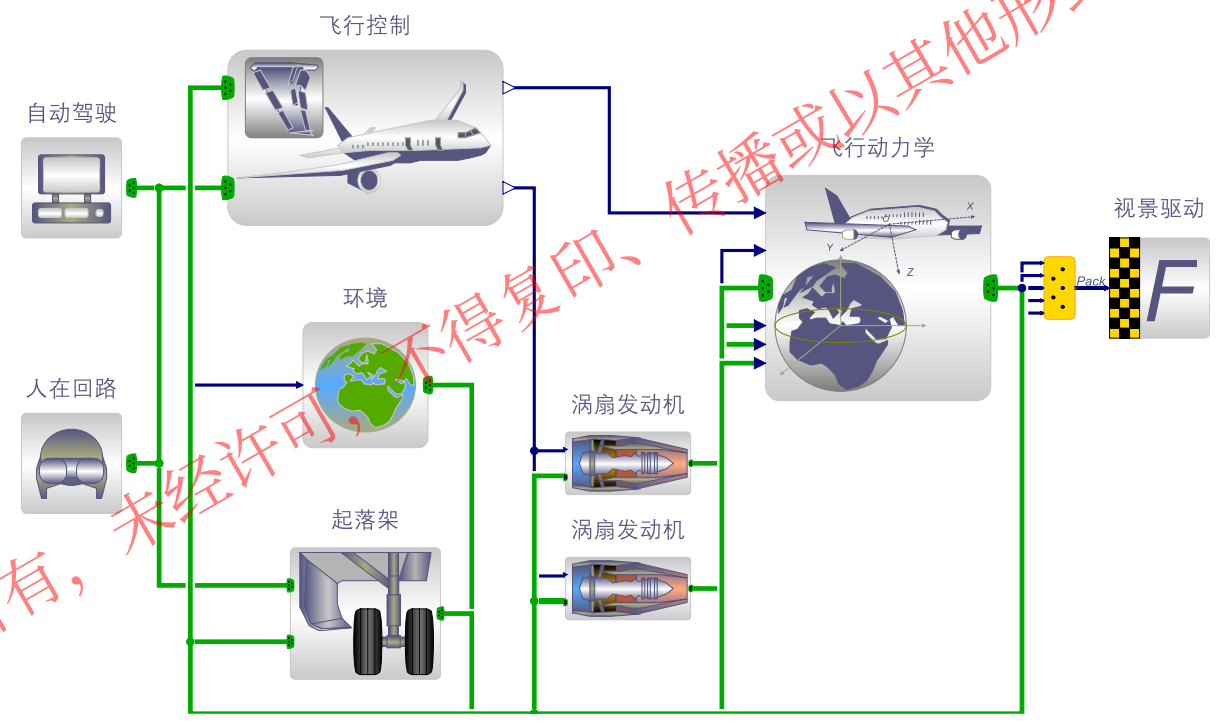
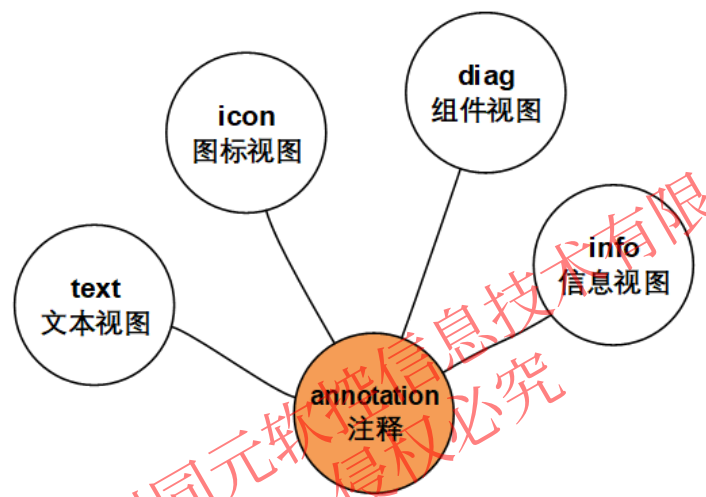
```
function LineWithProtected
input Real x;
input Real p0[2];
input Real p1[2];
output Real y;
protected
Real m = (p1[2] - p0[2]) / (p1[1] - p0[1]);
Real b = (p1[2] + p0[2] - m * (p1[1] + p0[1])) / 2.0;
algorithm
y := m * x + b;
end LineWithProtected;
```



4.2 Modelica知识点总览-注解

注解—定义与Modelica模型关联的附加信息

- 注解功能：
- 组件参数框设计
 - 图标设计
 - 模型帮助文档设计
 - 动态显示设计
 - 文本其他注释



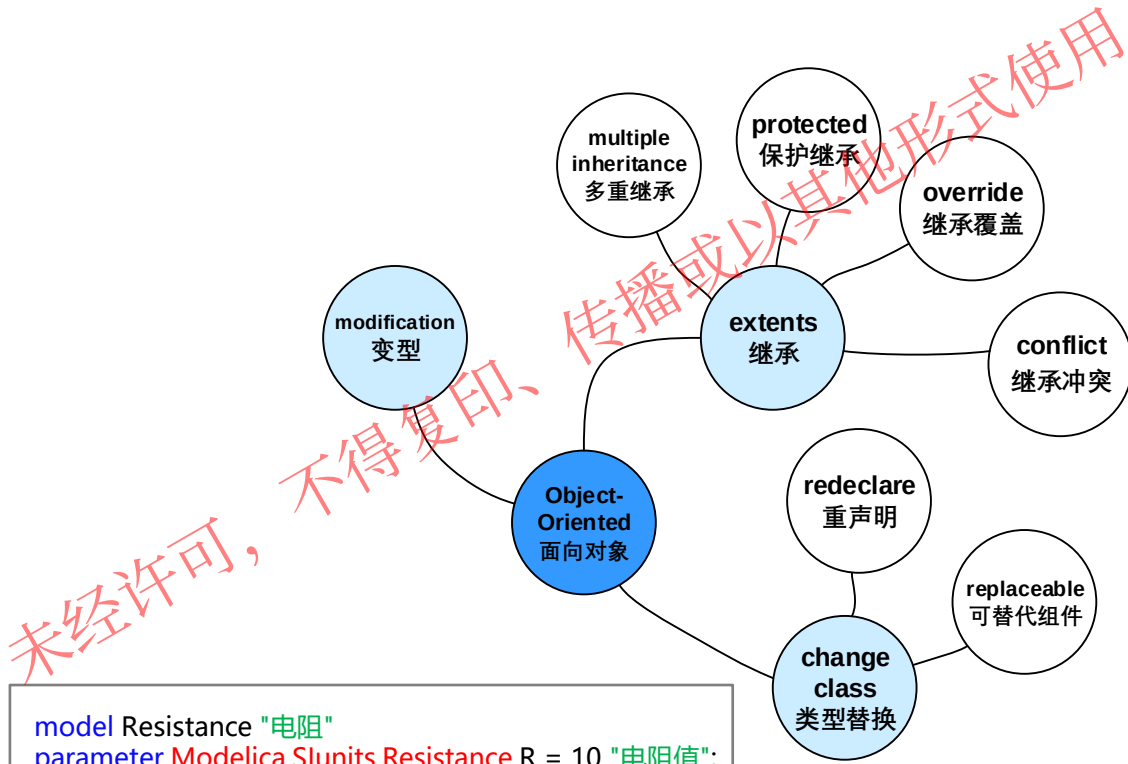
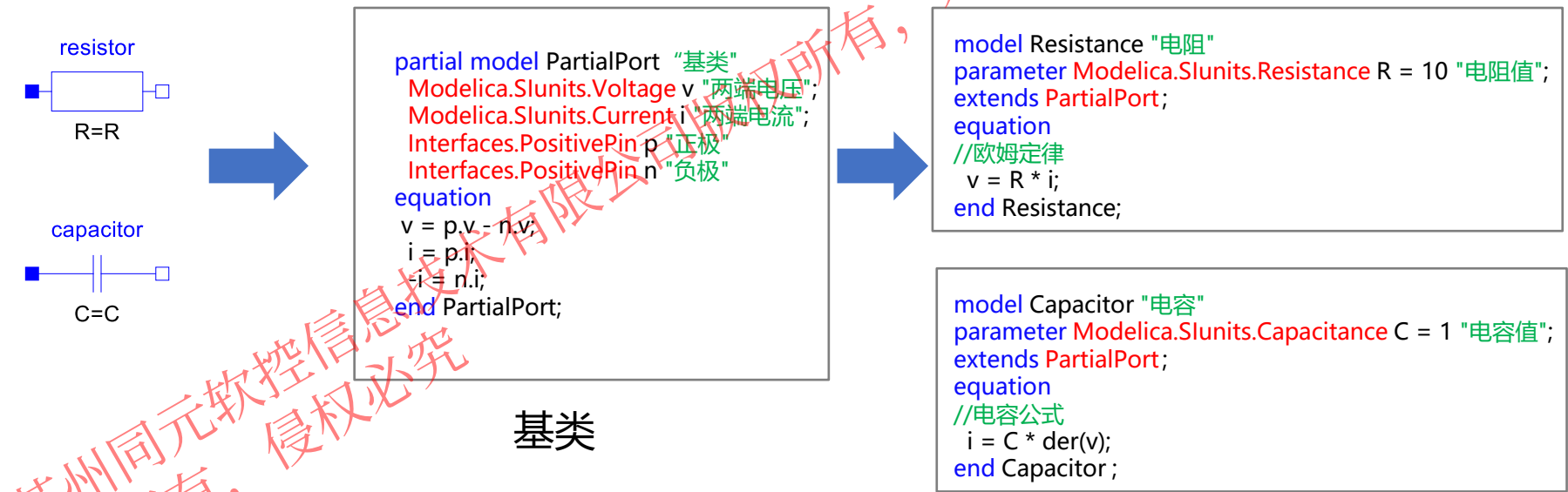
飞机仪表台.mp4

4.2 Modelica知识点总览-模型重用

模型重用—极大地提高建模效率

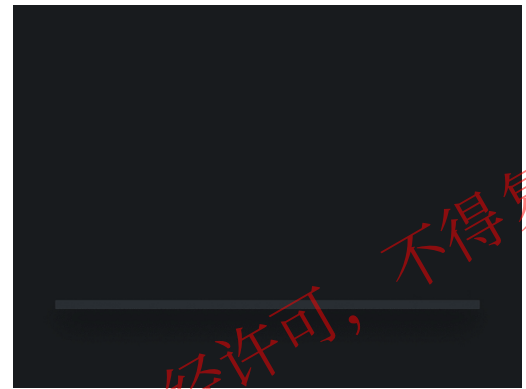
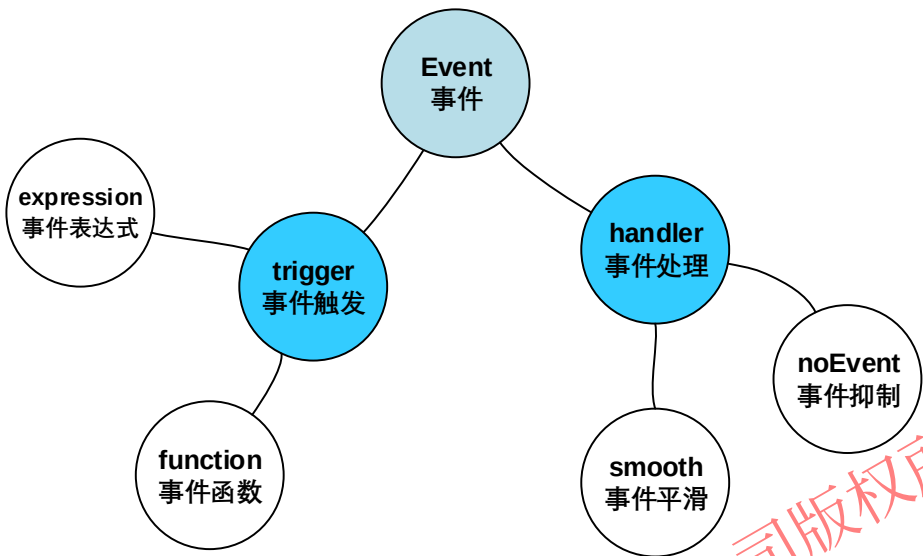
模型重用:

- 继承重用
- 实例化重用
- 重声明重用



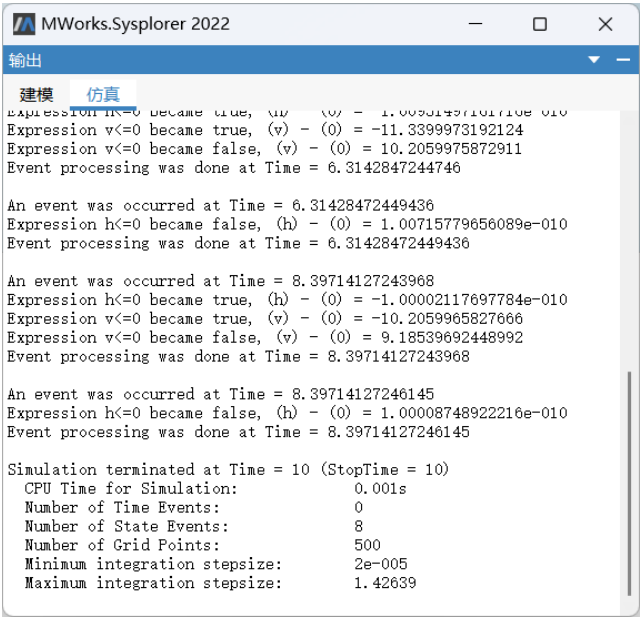
4.2 Modelica知识点总览-事件

事件—系统变量在特定时间点上瞬时的、不连续的发生变化



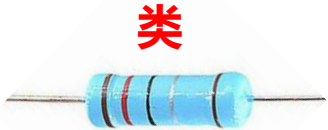
弹跳小球.gif

```
model Ball "弹跳小球"
  final parameter Real g = 9.8 "重力加速度";
  parameter Real coef = 0.9 "弹性系数";
  parameter Real h0 = 10 "初始高度";
  Real h "小球高度";
  Real v "小球速度";
  Boolean flying "是否运动";
  initial equation
    h = h0;
  equation
    flying = not (h <= 0 and v <= 0);
    der(v) = if flying then -g else 0;
    v = der(h);
    when h <= 0 then
      reinit(v, -coef * v);
    end when;
end Ball;
```



4.3 文本建模电阻示例-理论分析

如果想要构建一个电阻组件，我该怎么做？



属性分析

- 电阻是导体本身的一种属性，因此导体的电阻与导体是否接入电路、导体中有无电流、电流的大小等因素无关。
- 电阻是描述导体导电性能的物理量，用R表示。
- 电阻的单位是欧姆，简称欧，用希腊字母 “Ω”表示

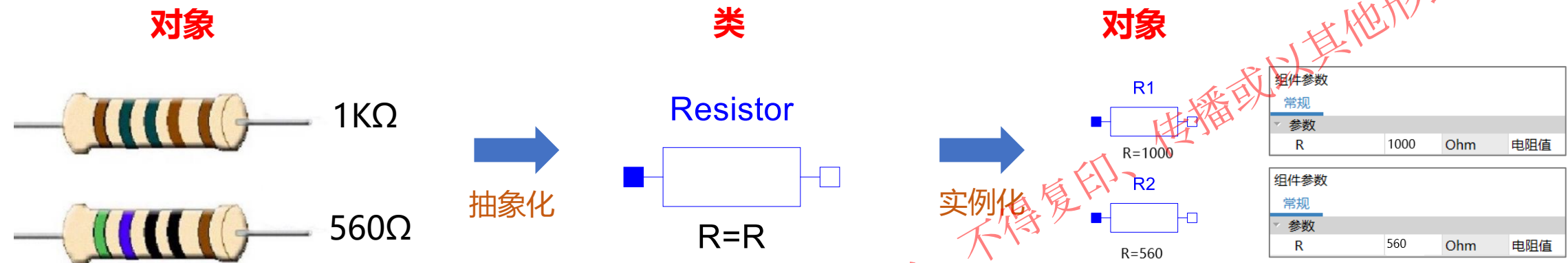
行为分析

- 电阻由导体两端的电压U与通过导体的电流I的比值来定义。
欧姆定律：
$$R = \frac{U}{I}$$
- 进入电阻电流为正值，离开电阻的电流为负值，电阻的电流的代数和等于零。
- 电阻的电压为电阻两端所加的电压



以上的电阻每一个都要单独建模么？

4.3 文本建模电阻示例-理论分析



- 接口：两个电学接口
- 属性：电阻值
- 行为：欧姆定律 $U = I \cdot R$

只需要构建一个电阻模型即可

```
model Object "对象"  
  Modelica.Electrical.Analog.Basic.Resistor R1(R = 1000) "电阻1"  
    annotation (...);  
  Modelica.Electrical.Analog.Basic.Resistor R2(R = 560) "电阻2"  
    annotation (...);  
  annotation (...);  
end Object;
```

拖拽类至模型中并定义参数，即为实例化对象

4.3 文本建模电阻示例-电阻模型开发



```
model 模型名称 "备注"
//===参数定义===
parameter 数据类型 参数名=参数值 "备注"

//===变量定义===
数据类型+变量 "备注";

//===接口实例化===
接口路径 接口1名称 "接口1"
接口路径 接口1名称 "接口1"

//===行为描述===
equation
    定义方程或者算法
.....

end 模型名称;
```

Modelica代码一般结构

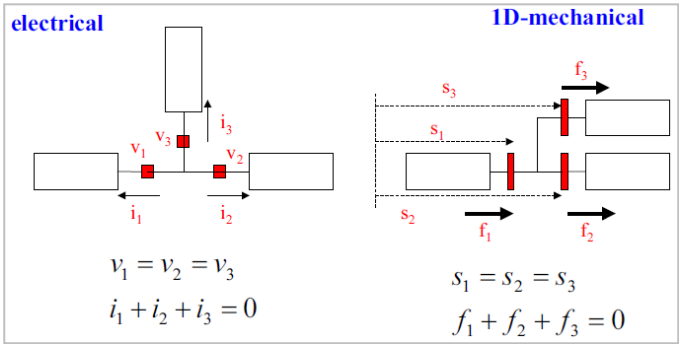


4.3 文本建模电阻示例-电阻模型开发

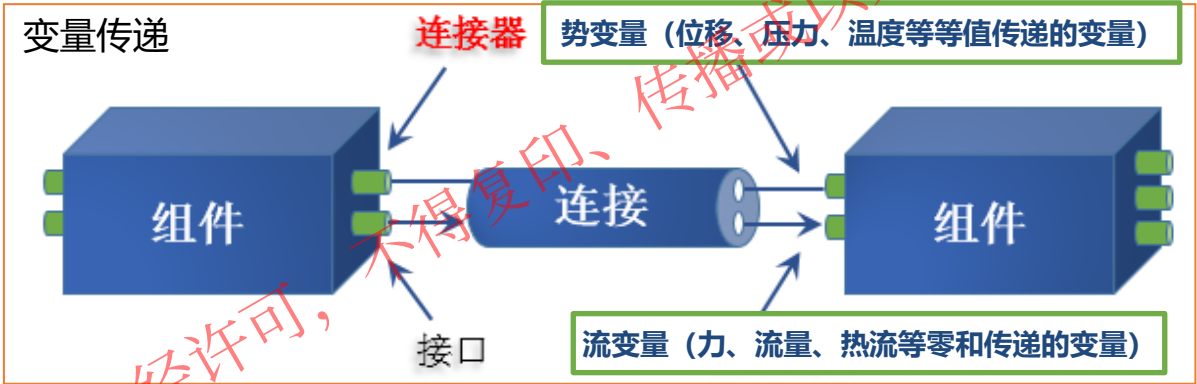


广义基尔霍夫定律

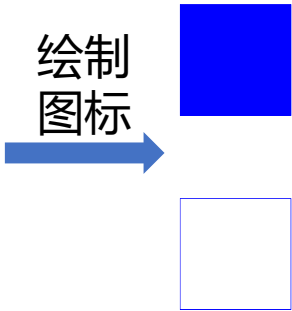
端口所有势变量相等；
端口所有流变量和零。



领域	势变量		流变量	
平移机械	s	位移	f	力
转动机械	φ	角度	τ	转矩
电子	v	电压	i	电流
液压	p	压力	Ṃ	流速
热力学	T	温度	Q̇	熵流
化学	μ	化学势	Ṅ	粒子流



```
connector PositivePin "正极"  
  Modelica.SIunits.Voltage v "电势"; //势变量  
  flow Modelica.SIunits.Current i "电流"; //流变量  
end PositivePin;  
  
connector NegativePin "负极"  
  Modelica.SIunits.Voltage v "电势"; //势变量  
  flow Modelica.SIunits.Current i "电流"; //流变量  
end NegativePin;
```

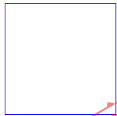
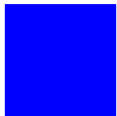


- 新建接口
 - 定义势变量v，流变量i
 - 绘制图标
- 详见课程《Modelica语法-连接与连接器(非因果连接器)》

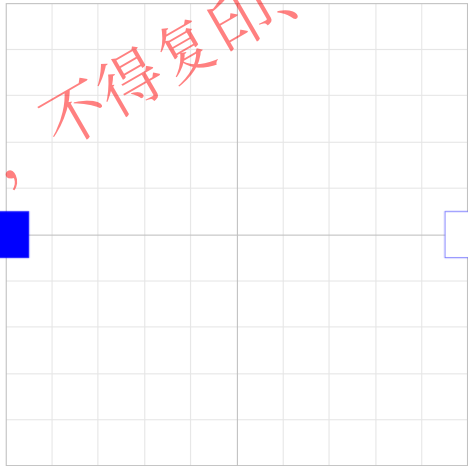
4.3 文本建模电阻示例-电阻模型开发



```
connector PositivePin "正极"  
  Modelica.SIunits.Voltage v "电势"; //势变量  
  flow Modelica.SIunits.Current i "电流"; //流变量  
end PositivePin;  
  
connector NegativePin "负极"  
  Modelica.SIunits.Voltage v "电势"; //势变量  
  flow Modelica.SIunits.Current i "电流"; //流变量  
end NegativePin;
```



接口实例化



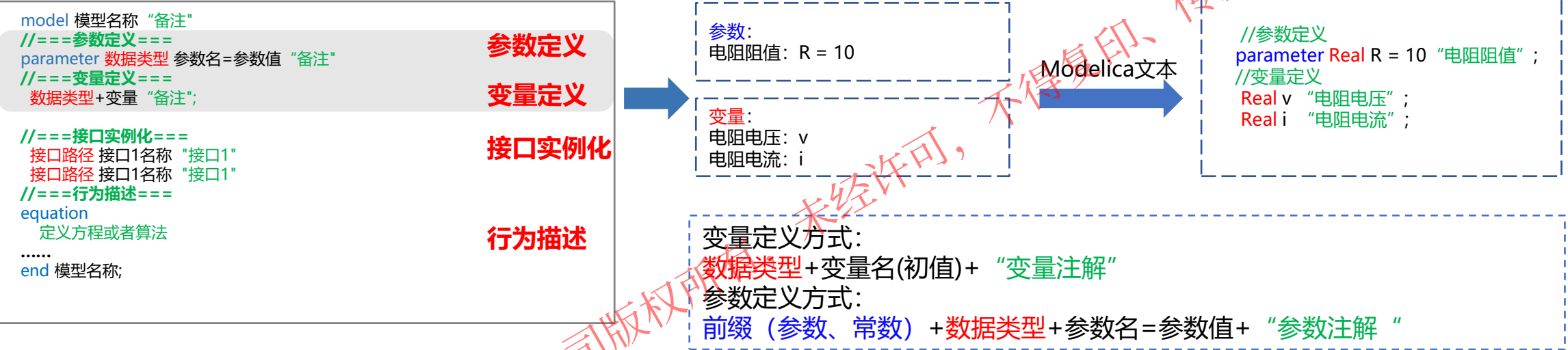
```
model Resistance "电阻 "  
  //接口实例化  
  //形式: 接口路径+接口名称  
  PositivePin positivePin;  
  NegativePin negativePin;  
end Resistance;
```

电阻"模型文本视图自动生成代码

将接口直接拖拽至"电阻"模型图形视图

详见课程《Modelica语法-连接与连接器(非因果连接器)》

4.3 文本建模电阻示例-电阻模型开发



数据类型

Modelica语言提供了5大数据类型: **Real、Integer、Boolean、String、enumeration**

详见课程《Modelica语法-类与内置类型-基本类型(基础数据类型及其属性)》

前缀 (参数、常数)

前缀 (参数、常数) 用于定义组件值在分析过程中的可变性: **parameter、constant**

详见课程《Modelica语法-类与内置类型-基本类型(数据的前缀用法)》



4.3 文本建模电阻示例-电阻模型开发



参数:
电阻阻值: $R = 10$

变量:
电阻电压: v
电阻电流: i

有物理含义
Modelica文本

```
//参数定义
parameter Modelica.SIunits.Resistance R = 10 "电阻阻值" ;
//变量定义
Modelica.SIunits.Voltage v "电阻电压" ;
Modelica.SIunits.Current i "电阻电流" ;
```

无物理含义
Modelica文本

```
//参数定义
parameter Real R = 10 "电阻阻值" ;
//变量定义
Real v "电阻电压" ;
Real i "电阻电流" ;
```

Modelica标准库中已根据ISO 31-1992国际标准单位制，声明了450个左右的物理量类型

因此，建议在建模过程中直接引用Modelica标准库中已进行预定义的物理量
Modelica还可以自行执行单位推导计算用来方便用户检查变量定义

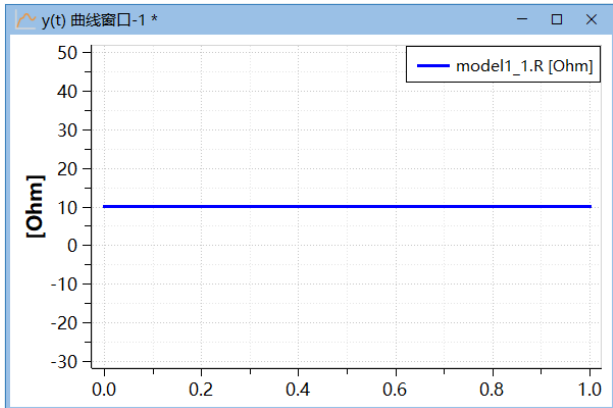
上述模型实现只反映了其数学特性，但是并没有明确模型中各个变量的物理属性，比如 R 、 v 、 i 等分别是什么？即：

- 各变量物理单位是什么？
- 更为重要的是：这些方程是否具有物理一致性？

注：物理一致性是方程等式两边是否具有相同的物理单位

那么，我们应该怎么做？

详见课程《Modelica语法-类与内置类型-基本类型(type)》



4.3 文本建模电阻示例-电阻模型开发



构建一个模型，需要定义对象的属性和行为。
前面已经学习了对象属性的定义，那么**对象的行为该如何定义？**

特点：以**陈述式**方程表达模型的行为，模型行为即模型的数学方程或物理方程。

```
model Resistance "电阻"  
  parameter Modelica.SIunits.Resistance R = 10;  
  Modelica.SIunits.Voltage v;  
  Modelica.SIunits.Current i;  
  PositivePin positivePin;  
  NegativePin negativePin;  
  
  equation  
    v = R * i;  
    i = positivePin.i;  
    positivePin.i + negativePin.i = 0;  
    v = positivePin.v - negativePin.v;  
  
end Resistance;
```

```
model 模型名称 "备注"  
  //===参数定义===  
  parameter 数据类型 参数名=参数值 "备注"  
  //===变量定义===  
  数据类型+变量 "备注";  
  //===接口实例化===  
  接口路径 接口1名称 "接口1"  
  接口路径 接口1名称 "接口1"  
  
  //===行为描述===  
  equation  
    定义方程或者算法  
  .....  
end 模型名称;
```

参数定义

变量定义

接口实例化

行为描述

- 欧姆定律： $R = \frac{U}{I}$
- 进入电阻电流为正值，离开电阻的电流为负值，电阻的电流的代数和等于零
- 电阻的电压为电阻两端所加的电压

详见课程《Modelica语法-模型行为描述(方程)》



4.3 文本建模电阻示例-电阻模型开发



```
model Resistance "电阻"
parameter Modelica.SIunits.Resistance R = 10;
Modelica.SIunits.Voltage v;
Modelica.SIunits.Current i;
PositivePin positivePin;
NegativePin negativePin;
equation
v = R * i;
i = positivePin.i;
positivePin.i + negativePin.i = 0;
v = positivePin.v - negativePin.v;
end Resistance "电阻";
```



- 检查语法
- 检查方程数和变量数



- 6个变量:
- v
 - i
 - positivePin.v
 - positivePin.i
 - negativePin.v
 - negativePin.i

- 6个方程:
- $v = R * i;$
 - $i = \text{positivePin.i};$
 - $\text{positivePin.i} + \text{negativePin.i} = 0;$
 - $v = \text{positivePin.v} - \text{negativePin.v};$
 - $\text{positivePin.i} = 0;$
 - $\text{negativePin.i} = 0;$

只有方程数=变量数，模型才完整，
否则需要检查模型中的变量和方程



4.3 文本建模电阻示例-电阻模型开发

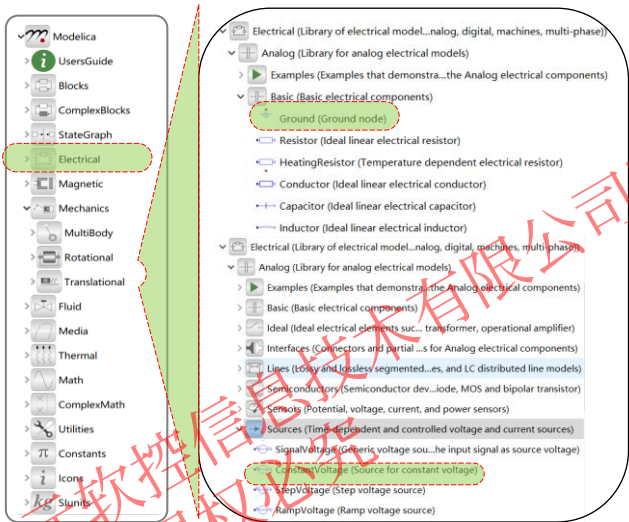
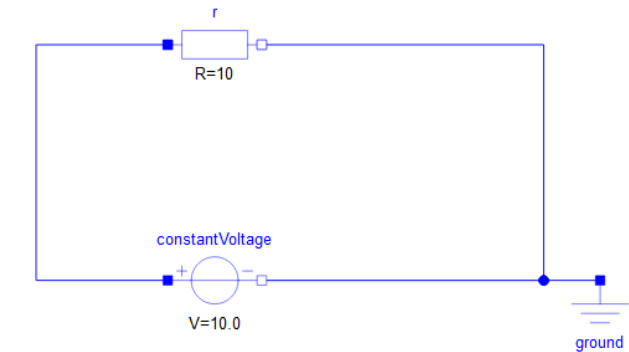


- 选择图标界面
- 使用“建模”或“编辑”的基本图元进行绘制
- 在图形属性中，为图形添加填充颜色及样式



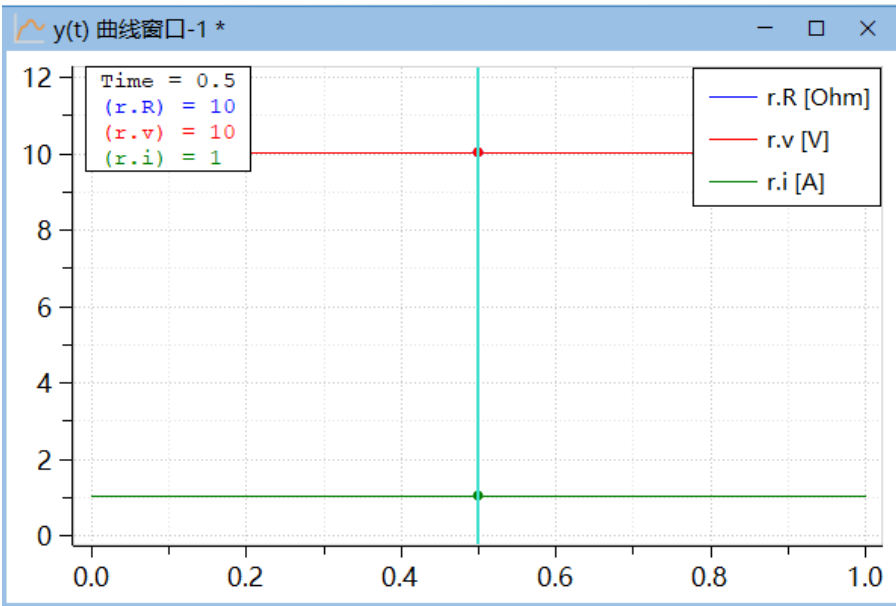
给模型穿上“漂亮的衣服”

4.3 文本建模电阻示例-电阻模型开发



名字	值	单位	描述
R			
r			
R	10	Ohm	电阻值
v	10	V	电阻两端电压
i	1	A	流过电阻的电流

结果查看



接地组件路径-Modelica. Electrical.Analog.Basic.Ground
恒压电源路径-Modelica. Electrical.Analog.Sources. ConstantVoltage



目录

1. MWOKRS.Sysplorer软件介绍

2. 软件界面简介

3. 图形化建模

4. Modelica文本建模

5. MWORKS.Sysplorer与Simulink之间的差异

6. MWORKS.Sysplorer与Syslab双向融合

5.1 因果建模与非因果建模

问题1：有哪些方式可以进行ODE方程组求解？

Lorenz 方程的求解，

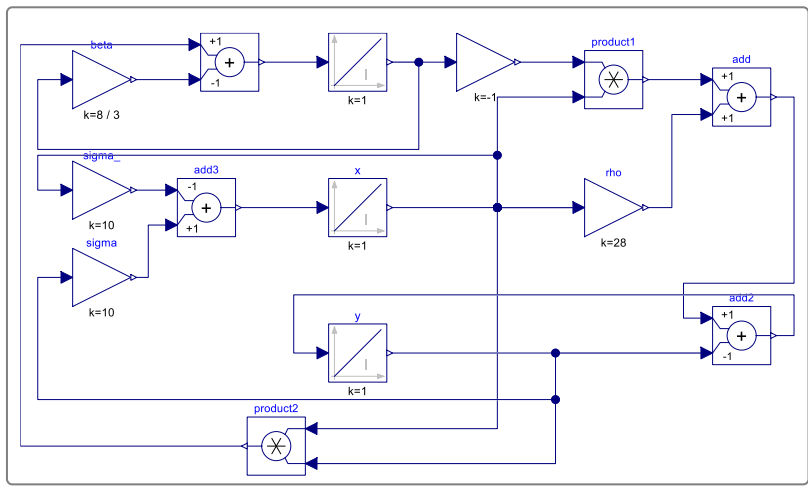
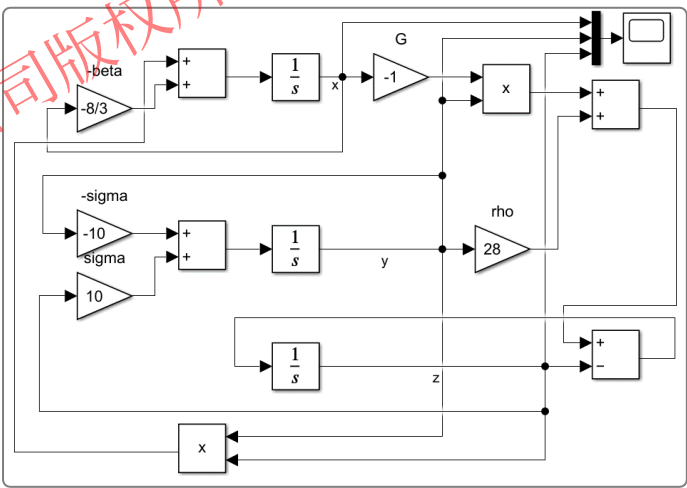
其中 $\sigma = 10, \rho = 28, \beta = 8/3$,

且初值为： $x_0 = 1, y_0 = z_0 = 0$

$$\begin{cases} \frac{dx}{dt} = \sigma(y - x) \\ \frac{dy}{dt} = x(\rho - z) - y \\ \frac{dz}{dt} = xy - \beta z \end{cases}$$

```
clear;
f=@(t,x)[-8/3*x(1)+x(2)*x(3);
-10*x(2)+10*x(3);
-x(1)*x(2)+28*x(2)-x(3)];%描述微分方程系统
tf=100; %仿真结束时间
x0=[0;0;1e-9]; %系统初值
options=odeset;
options.RelTol=1e-9; %仿真设置
[t,x]=ode45(f,[0,tf],x0,options); %调用算法求解
plot(t,x); %结果作图
legend('x1','x2','x3');
plot3(x(:,1),x(:,2),x(:,3)); %相空间作图
```

```
#一阶显微分方程组—洛伦兹方程
function Lorenz(du, u, p, t)
du[1] = 10.0 * (u[2] - u[1])
du[2] = u[1] * (28.0 - u[3]) - u[2]
du[3] = u[1] * u[2] - (8 / 3) * u[3]
end
u0 = [10, 0, 0]
tspan = (0, 100)
prob = ODEProblem(Lorenz, u0, tspan)
sol = solve(prob, DP5(), saveat = 0.1)
plot3(sol[1, :], sol[2, :], sol[3, :])
```



5.1 因果建模与非因果建模

方程的形式描述模型行为

Lorenz 方程的求解，其中 $\sigma = 10, \rho = 28, \beta = 8/3$ ，且初值为： $x_0 = 1, y_0 = z_0 = 0$

$$\begin{cases} \frac{dx}{dt} = \sigma(y - x) \\ \frac{dy}{dt} = x(\rho - z) - y \\ \frac{dz}{dt} = xy - \beta z \end{cases}$$

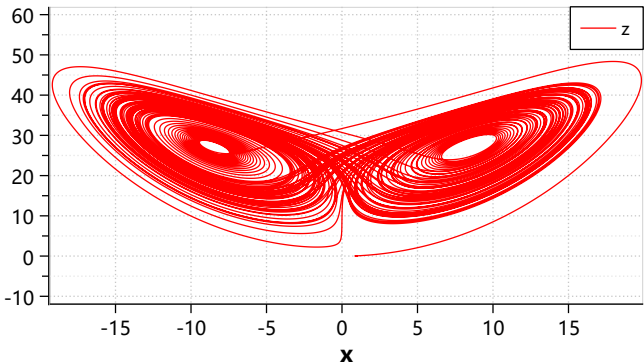
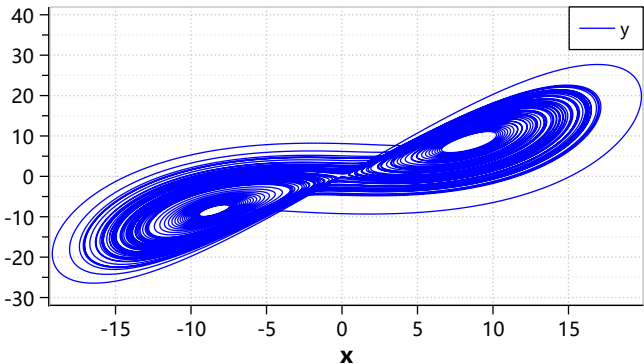
```
model Lorenz
  parameter Real sigma = 10;
  parameter Real rho = 28;
  parameter Real beta = 8 / 3;
  Real x(start = 1.0);
  Real y(start = 0.0);
  Real z(start = 0.0);
equation
  der(x) = sigma * (y - x);
  der(y) = x * (rho - z) - y;
  der(z) = x * y - beta * z;
end Lorenz;
```

定义模型名称: model+模型名称+ “备注”

- 声明模型参数
- 声明模型变量
- 声明继承或被调用的模型 (如接口)

- 通过方程或算法，描述模型行为

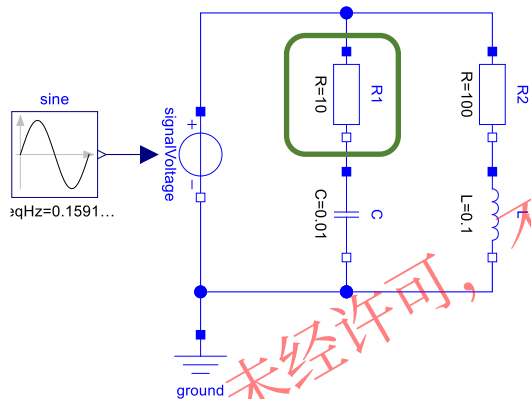
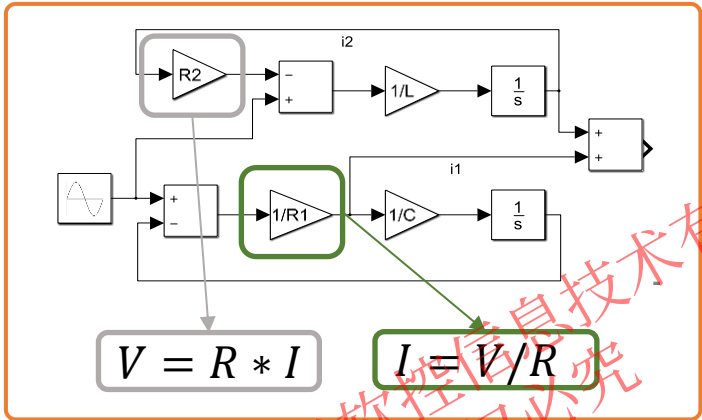
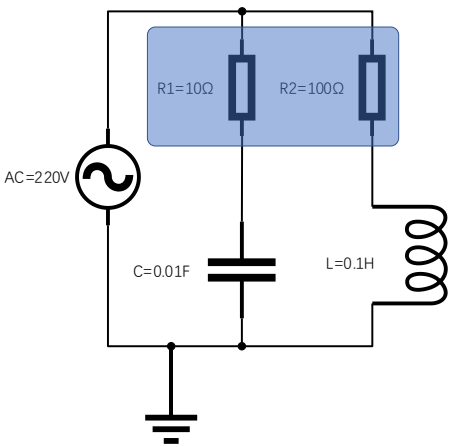
结束模型定义: end+模型名称;



以方程的形式描述模型行为，不需要考虑计算步骤和赋值关系，可极大提高建模效率

5.1 因果建模与非因果建模

问题2：因果建模和非因果建模方式如何构建模型？



欧姆定律：
 $V = R * I$ 或 $I = V/R$

非因果建模：
Modelica语言可以根据
机理方程直接构建等式，
无需推导因果关系。

```
model Resistance1 "电阻"
  annotation ();
  parameter Modelica.SIunits.Resistance R = 10 "电阻值";
  Modelica.SIunits.Voltage v "电阻两端电压";
  Modelica.SIunits.Current i "电阻两端电流";
  Interfaces.PositivePin positivePin "正极" annotation (...);
  Interfaces.NegativePin negativePin "负极" annotation (...);
equation
  v = positivePin.v - negativePin.v; //电阻两端电压
  i = positivePin.i; //流过电阻电流
  -i = negativePin.i;
  v = R * i; //欧姆定律
end Resistance;
```

```
model Resistance2 "电阻"
  annotation ();
  parameter Modelica.SIunits.Resistance R = 10 "电阻值";
  Modelica.SIunits.Voltage v "电阻两端电压";
  Modelica.SIunits.Current i "电阻两端电流";
  Interfaces.PositivePin positivePin "正极" annotation (...);
  Interfaces.NegativePin negativePin "负极" annotation (...);
equation
  i = v / R; //欧姆定律
  v = positivePin.v - negativePin.v; //电阻两端电压
  i = positivePin.i; //流过电阻电流
  -i = negativePin.i;
end Resistance2;
```

完全等价

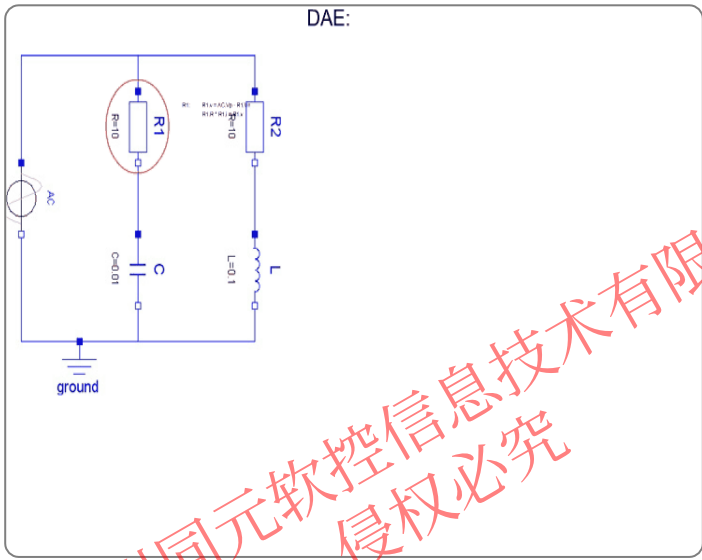


5.1 因果建模与非因果建模

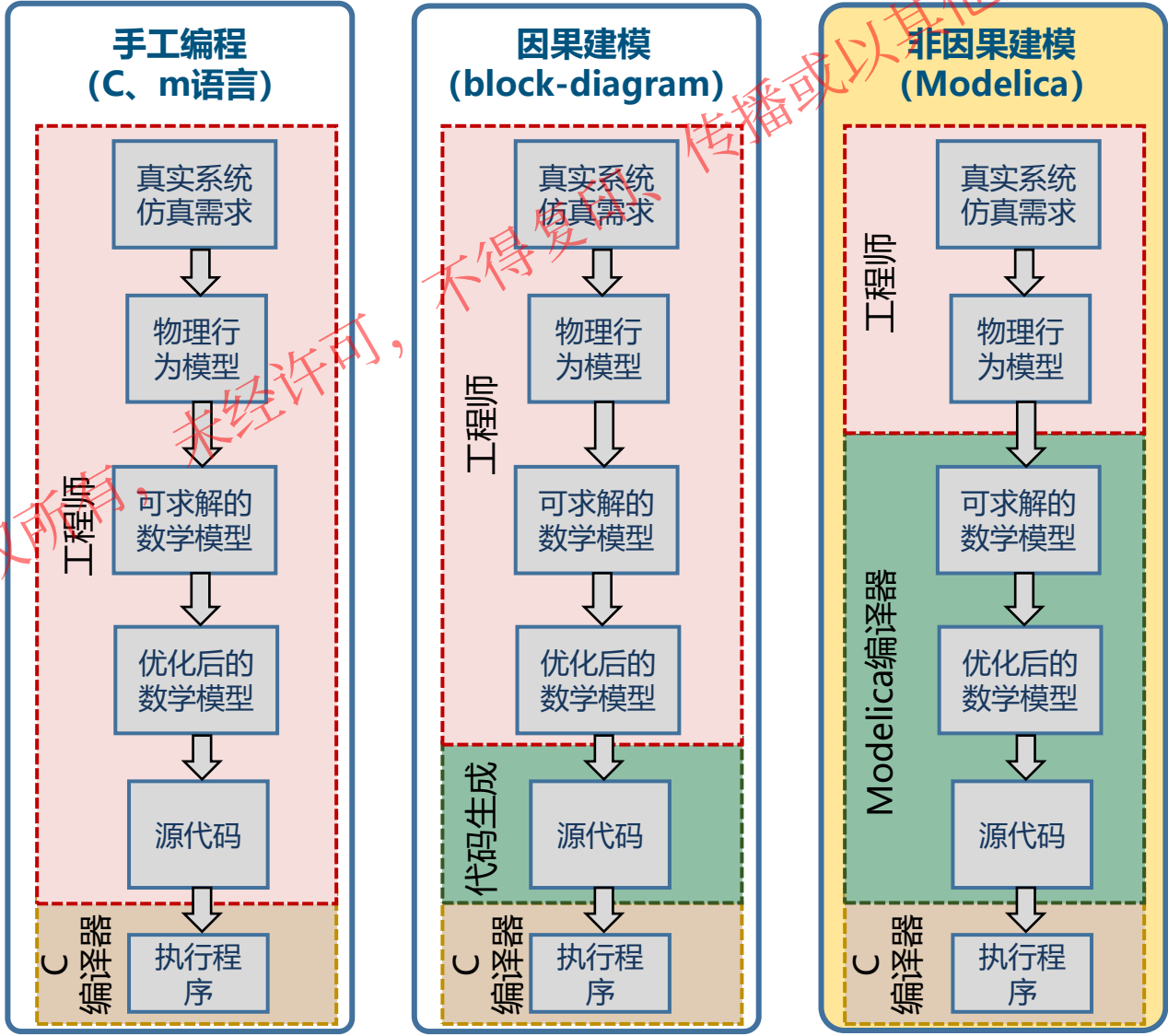
因果建模VS非因果建模

基于数学公式建模，不必关心求解与编程细节

Modelica建模仿真工具能够决定如何自动求解方程变量，无需手工处理。工程师集中精力于建立对象的数学模型，不必关心模型的求解和编程实现的过程，大大提高建模的效率以及模型的可重用性。

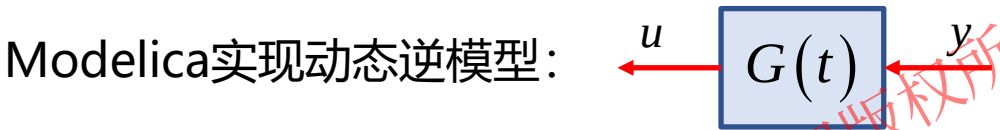
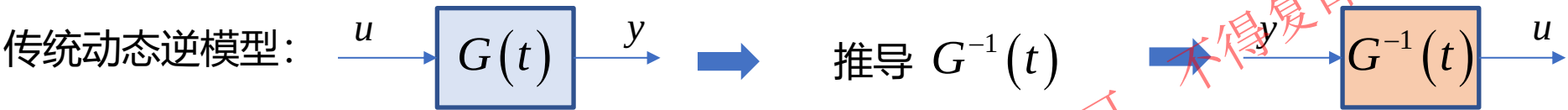


非因果建模演示



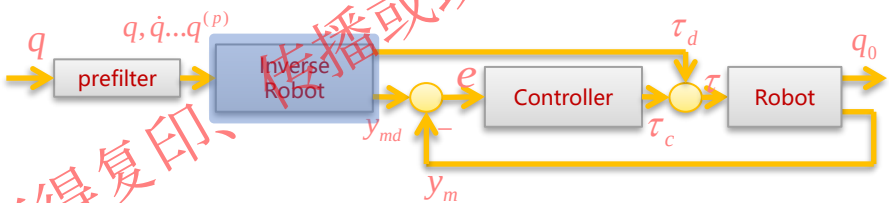
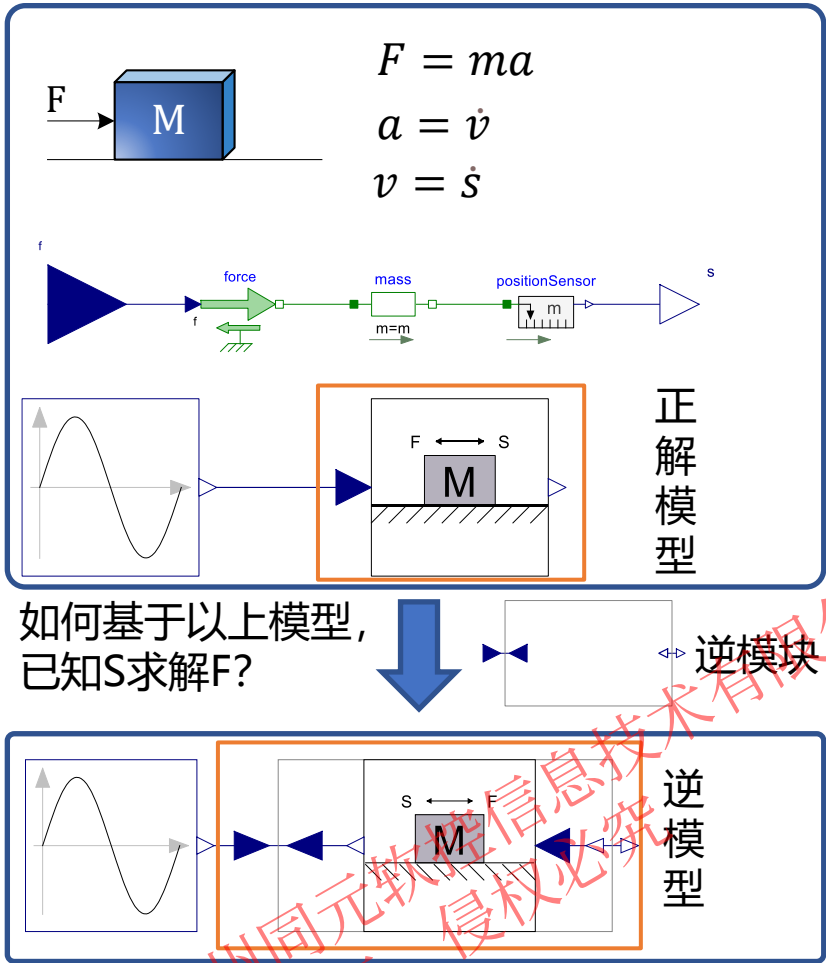
5.1 因果建模与非因果建模

非因果是否意味着：已知模型输出，可以反求其输入？

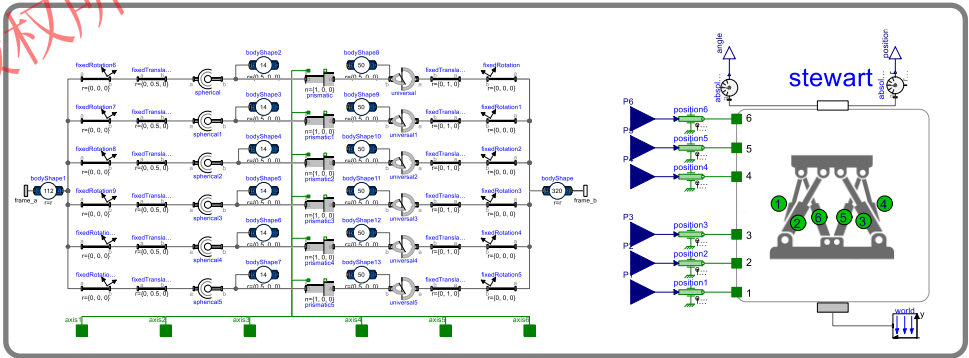


5.1 因果建模与非因果建模

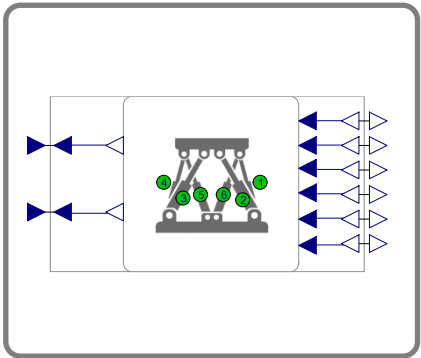
基于非因果建模的动态逆



动态逆模型表现为**高阶DAE方程**，手工推导十分繁杂，利用Modelica非因果建模的方式则非常容易求解。



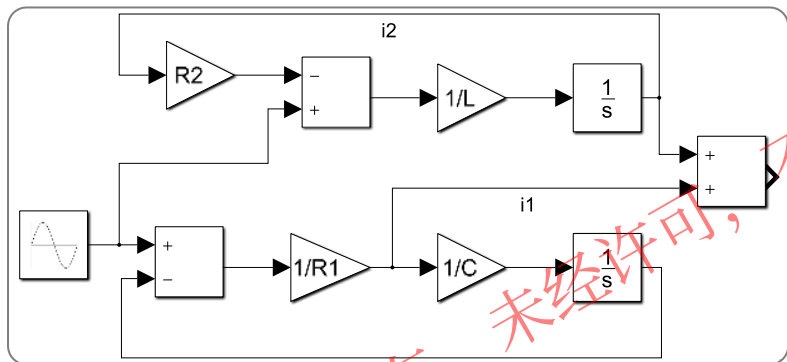
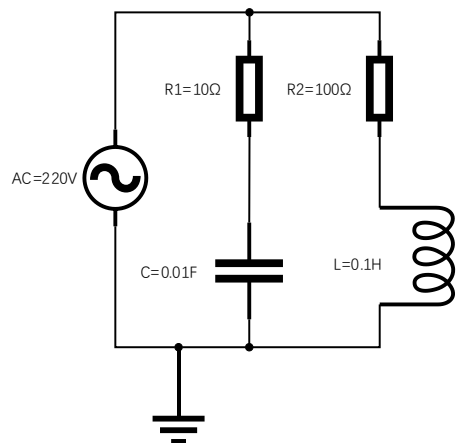
1. 根据物理拓扑关系形成正解模型



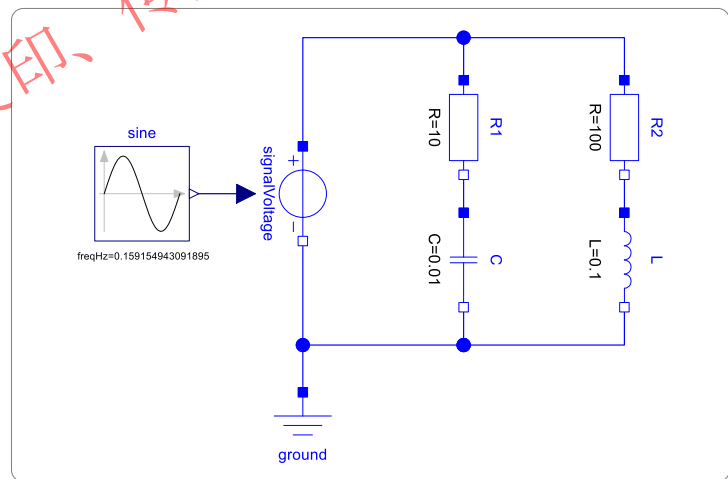
2. 使用逆模块封装

5.2 面向函数与面向对象

问题3：如何根据物理现象构建系统模型？



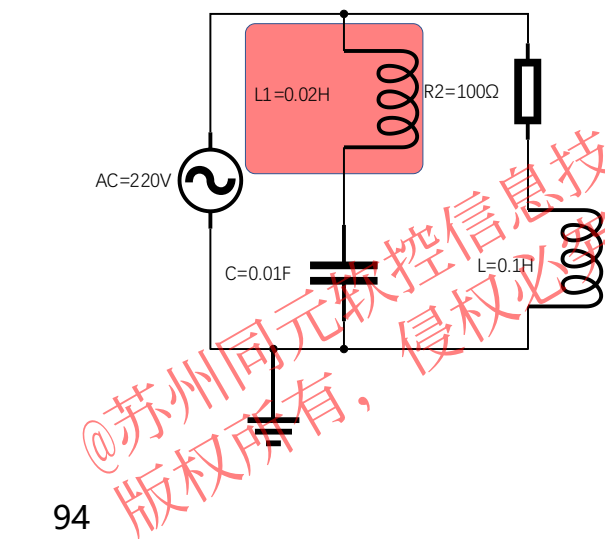
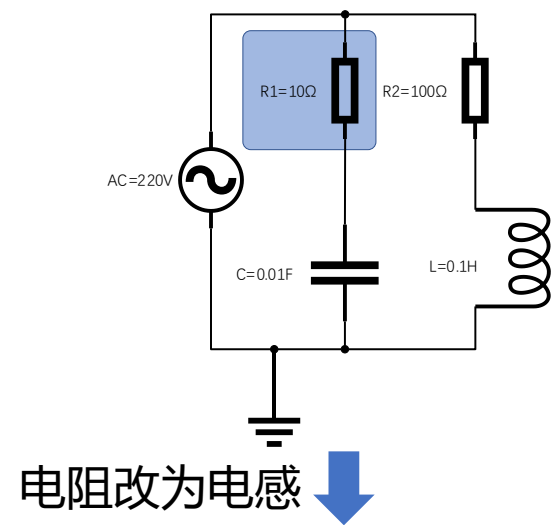
- 确定系统的输入与输出信号
- 建立方程组
- 将输入带入输出，推导输入输出函数
- 根据输入输出函数，拖放并连接组件



- 根据物理拓扑关系，拖放并连接组件

5.2 面向函数与面向对象

问题4：系统架构不变的情况下，如何调整某些组件的型号？



1. 确定系统的输入与输出信号

2. 建立方程组

3. 将输入带入输出，推导输入输出函数

4. 根据输入输出函数，拖放并连接组件

$$u(t) = R_1 i_1(t) + \frac{1}{C} \int i_1(t) dt$$
$$u(t) = R_2 i_2(t) + L \frac{di_2(t)}{dt}$$
$$i(t) = i_1(t) + i_2(t)$$
$$i_1(t) = \frac{1}{R_1} \left(u(t) - \frac{1}{C} \int i_1(t) dt \right)$$
$$i_2(t) = \int \frac{u(t) - R_2 i_2(t)}{L} dt$$

1. 根据物理拓扑关系，拖放并连接组件

5. 确定系统的输入与输出信号

6. 重新建立方程组

7. 将输入带入输出，推导输出函数

8. 根据输入输出函数，拖放并连接组件

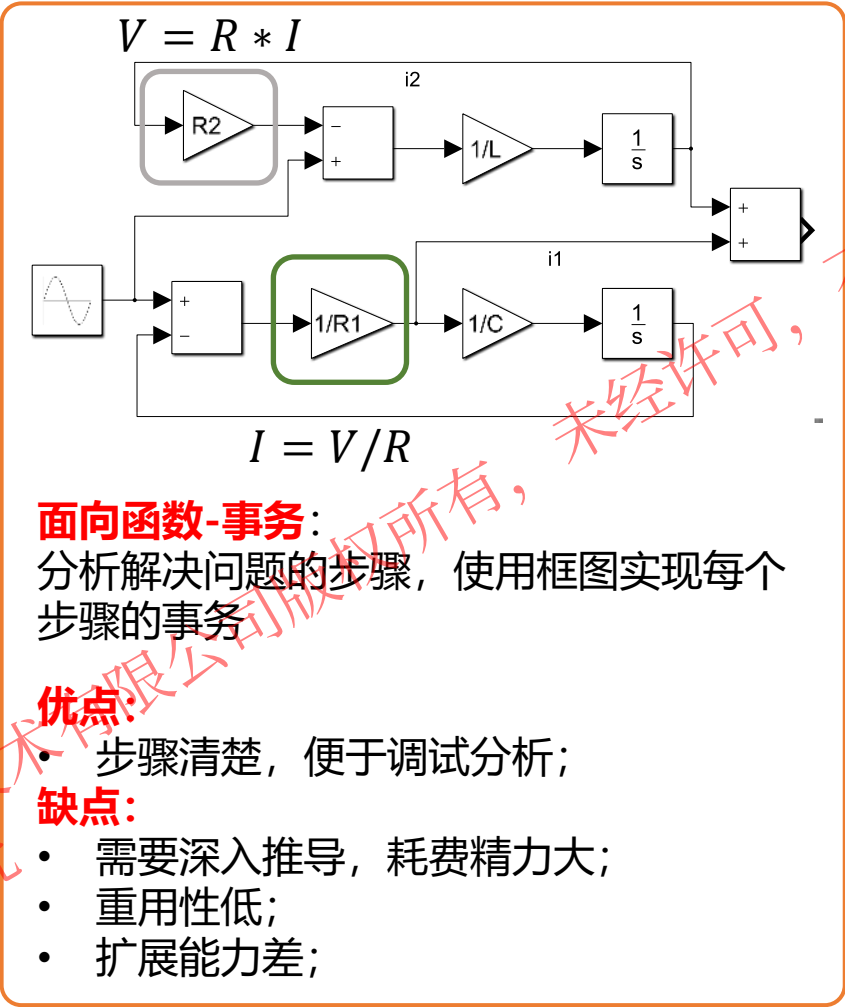
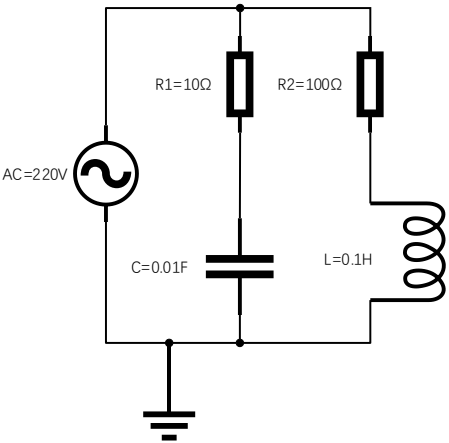
$$u(t) = L_1 \frac{di_1(t)}{dt} + \frac{1}{C} \int i_1(t) dt$$
$$u(t) = R_2 i_2(t) + L \frac{di_2(t)}{dt}$$
$$i(t) = i_1(t) + i_2(t)$$
$$i_1(t) = \frac{1}{L_1} \int u(t) dt - \frac{1}{CL_1} \iint i_1(t) dt$$
$$i_2(t) = \int \frac{u(t) - R_2 i_2(t)}{L} dt$$

2. 替换组件类

传播或以其他形式使用

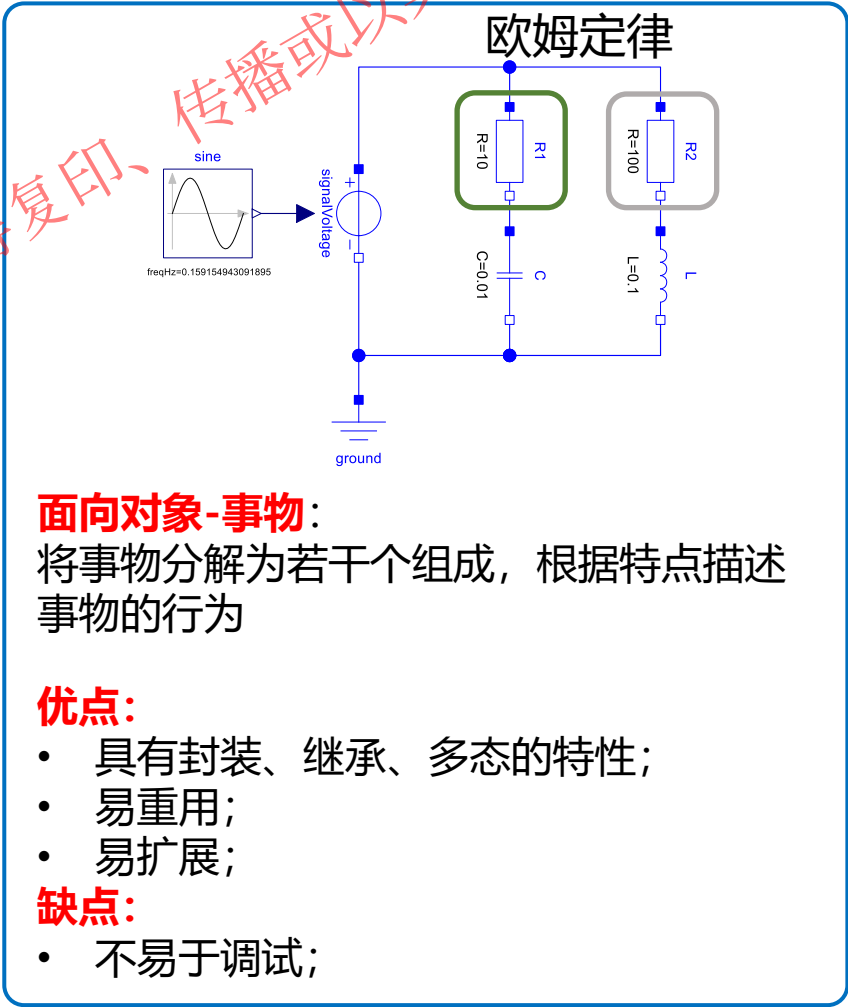
5.2 面向函数与面向对象

面向函数VS面向对象



面向函数-事务:
分析解决问题的步骤，使用框图实现每个步骤的事务

- 优点:**
- 步骤清楚，便于调试分析;
- 缺点:**
- 需要深入推导，耗费精力大;
 - 重用性低;
 - 扩展能力差;



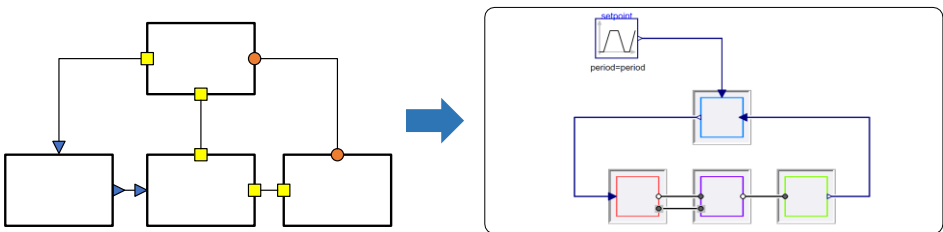
面向对象-事物:
将事物分解为若干个组成，根据特点描述事物的行为

- 优点:**
- 具有封装、继承、多态的特性;
 - 易重用;
 - 易扩展;
- 缺点:**
- 不易于调试;

5.2 面向函数与面向对象

基于面向对象的模型演进

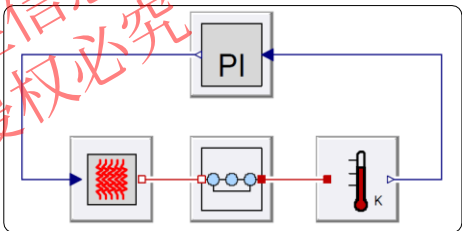
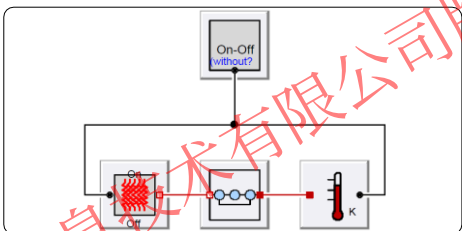
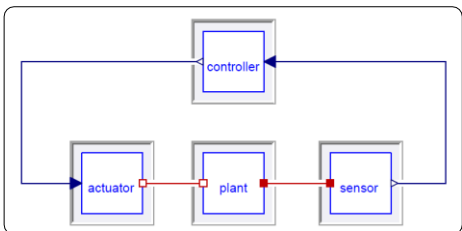
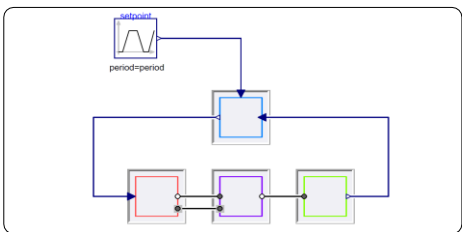
考虑系统在不同阶段对同一系统粒度需求的差异性，结合Modelica语言面向对象的特征，在架构接口保持不变的情况下，可以通过重声明机制实现对模型的演进，以适用于不同阶段的仿真需求。



系统架构



基于Modelica重声明机制的模型演进



架构接口基类模型

```
Partial Model BaseArchitecture
replaceable Interfaces.PlantModel plant;
replaceable Interfaces.ControlSystem controller;
replaceable Interfaces.Sensor sensor;
replaceable Interfaces.Actuator actuator;
```

实例化并重声明的仿真模型

```
Model BaseSystem
extends BaseArchitecture(
  redeclare replaceable Implementations.BasicPlant plant,
  redeclare replaceable Implementations.BasicController controller,
  redeclare replaceable Implementations.IdealSensor sensor,
  redeclare replaceable Implementations.IdealActuator actuator);
```

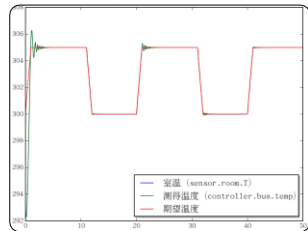
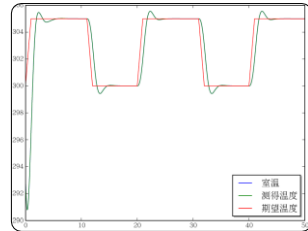
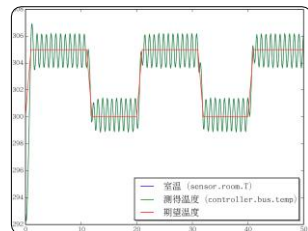
重声明替换实现模型演进

```
Model BaseSystemI
extends BaseArchitecture(
  redeclare replaceable Implementations.Plant plant(C=2, G=1, h=2),
  redeclare replaceable Implementations.OnOffController controller(setpoint=300),
  redeclare replaceable Implementations.TempSensor sensor,
  redeclare replaceable Implementations.Actuator actuator);
```

根据场景、需求，模型粒度的进一步细化

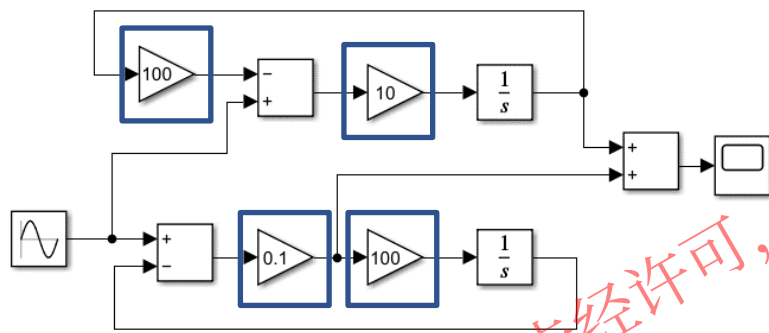
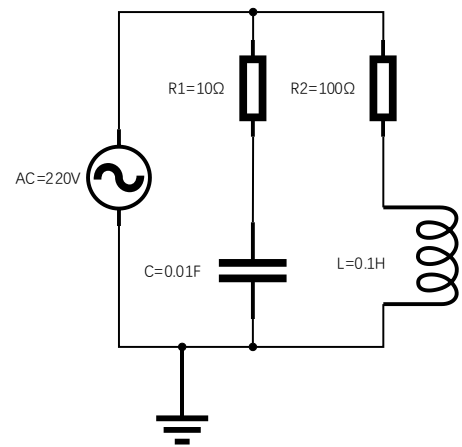
```
Model BaseSystemII
extends BaseArchitecture(
  redeclare replaceable Implementations.Plant plant(C=2, G=1, h=2),
  redeclare replaceable Implementations.PIDController controller(setpoint=300, T=1, k=200),
  redeclare replaceable Implementations.TempSensor sensor,
  redeclare replaceable Implementations.Actuator actuator);
```

动态仿真



5.3 信号建模与物理建模

问题5：查看组件的参数，结果有何异同？



- 100: R2
- 0.1: 1/R1
- 100: 1/C
- 10: 1/L

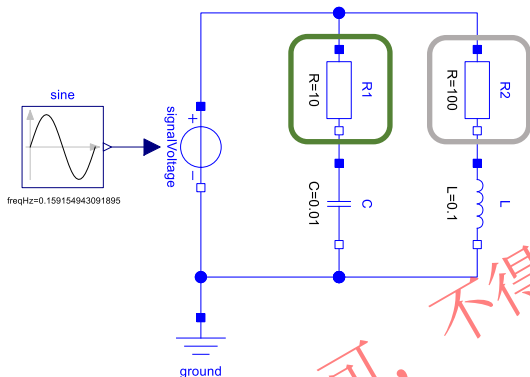
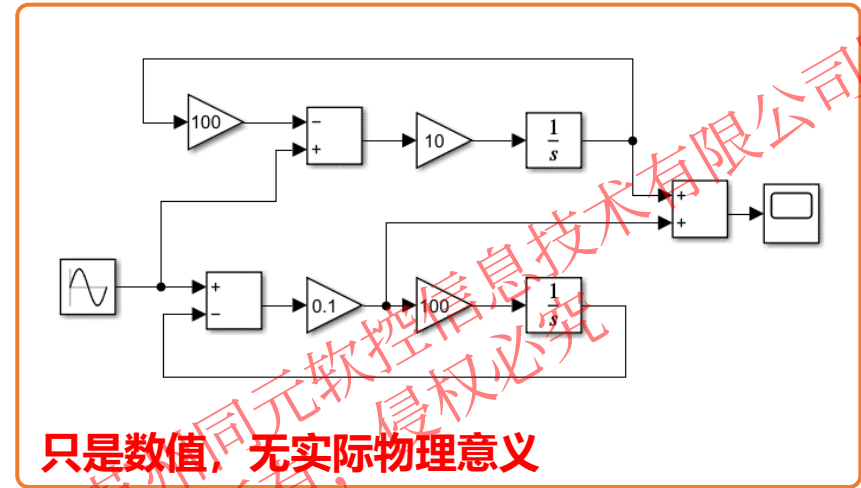
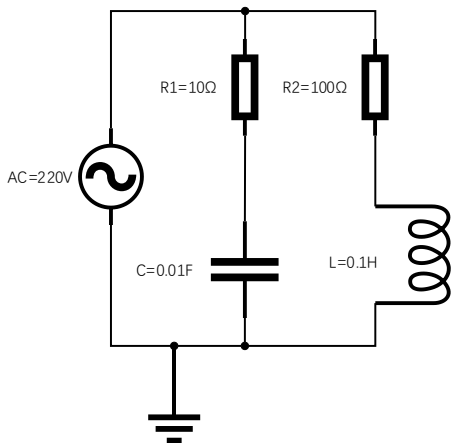
均为无量纲数值，无实际物理意义

组件参数		
常规		
参数		
C	1	F 电容值
		F
		uF

每一个数值都有物理量纲，均具有物理含义，可自动进行单位换算。

5.3 信号建模与物理建模

信号建模VS物理建模



Modelica标准库中已根据ISO 31-1992
国际标准单位制，声明了450个左右的物
理量类型，可声明变量的物理量纲

```
model Resistance1 "电阻"  
  annotation ();  
  parameter Modelica.SIunits.Resistance R = 10 "电阻值";  
  Modelica.SIunits.Voltage v "电阻两端电压";  
  Modelica.SIunits.Current i "电阻两端电流";  
  Interfaces.PositivePin positivePin "正极" annotation (...);  
  Interfaces.NegativePin negativePin "负极" annotation (...);  
equation  
  v = positivePin.v - negativePin.v; //电阻两端电压  
  i = positivePin.i; //流过电阻电流  
  -i = negativePin.i;  
  v = R * i; //欧姆定律  
end Resistance;
```

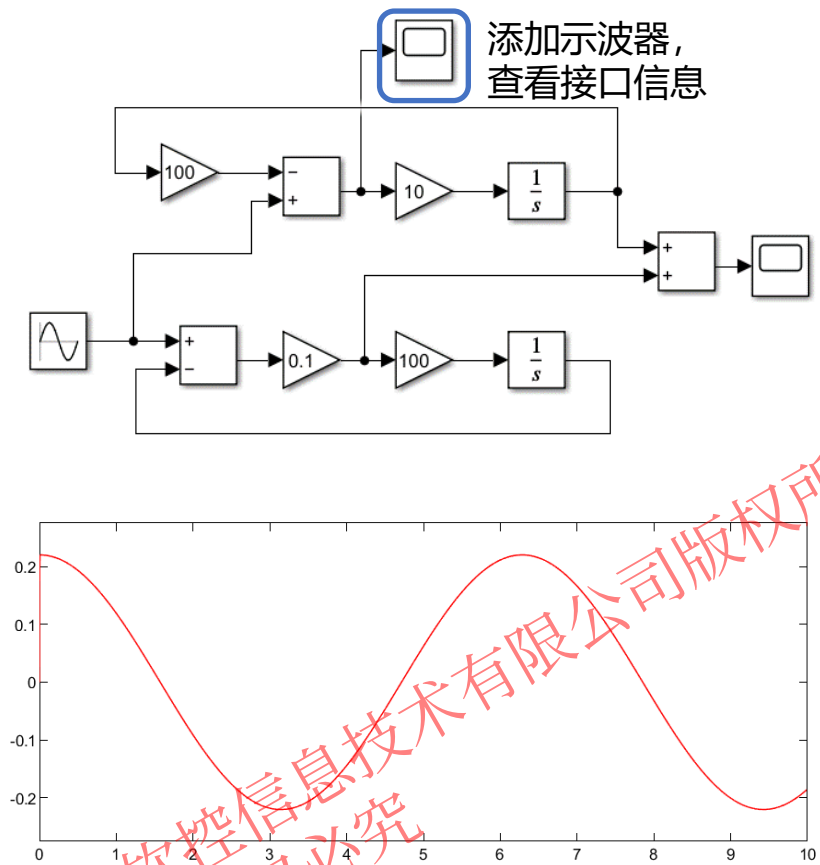
```
model Resistance1 "电阻"  
  annotation ();  
  parameter Modelica.SIunits.Resistance R = 10 "电阻值";  
  Modelica.SIunits.Voltage v "电阻两端电压";  
  Modelica.SIunits.Current i "电阻两端电流";  
  Interfaces.PositivePin positivePin "正极" annotation (...);  
  Interfaces.NegativePin negativePin "负极" annotation (...);  
equation  
  v = positivePin.v - negativePin.v; //电阻两端电压  
  i = positivePin.i; //流过电阻电流  
  -i = negativePin.i;  
  v = R / i; //欧姆定律  
end Resistance;
```

---- 检查模型: ModelicaGrammar.RLC.BasicModel.Resistance ----
正在解析模型...
正在实例化模型...
警告(5002): 方程 $v = \text{positivePin.v} - \text{negativePin.v}$ 的单位不匹配:
v 的单位为 m/s,
positivePin.v - negativePin.v 的单位为 V.
警告(5002): 方程 $v = R / i$ 的单位不匹配:
v 的单位为 m/s,
R / i 的单位为 Ohm/A.

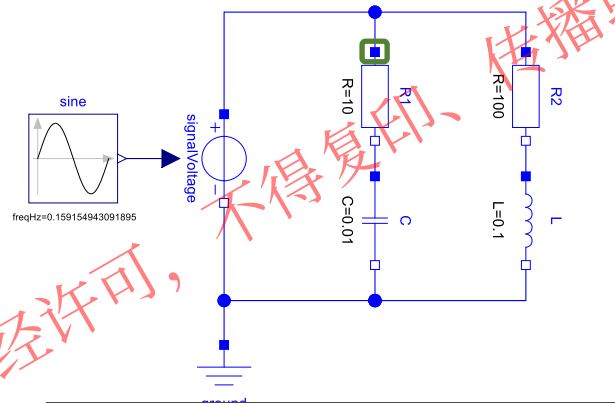
Modelica 还规定了复杂数学表达式单位计算的规则。以便检测模型中单位属性的错误或不一致。这对模型开发者来说是一个巨大的优势!

5.4 接口机制

问题6：如何查看R1仿真结果信息？



接口中仅传递是数值信号，因此称为**信号流连接器**。



仿真浏览器

搜索仿真结果变量

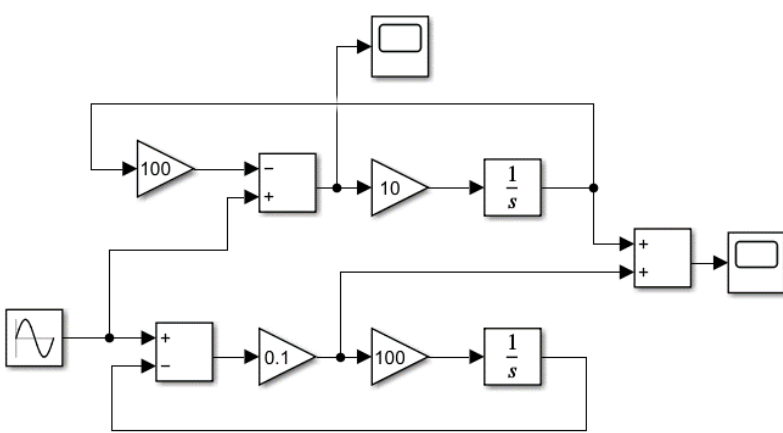
名字	值	单位	描述
Example			
R1			
☐ v		V	Voltage drop of the two ...
☐ i		A	Current flowing from pin ...
p			
☐ v		V	Potential at the pin
☐ i		A	Current flowing into the ...
n			
☐ v		V	Potential at the pin
☐ i		A	Current flowing into the ...
☐ L	0.1	H	Inductance
C			
L			
R2			
signalVoltage			
ground			
sine			

系统中任意组件中的所有变量均可查看结果

根据接口信息，电阻接口中传递是能量，因此称为**能量流连接器**。

5.4 接口机制

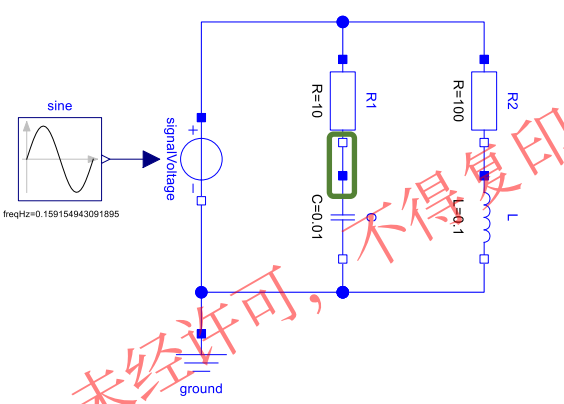
信号流连接器VS能量流连接器



信号流连接器:

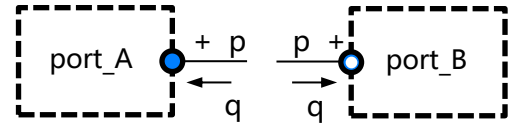
- 传递数值信号;
- 有固定流向, 输入接口→输出接口;
- 输出值→ 输入值;

能量流连接器更符合物理实际,
无需对子系统进行强行解耦,
极大减少工作量。

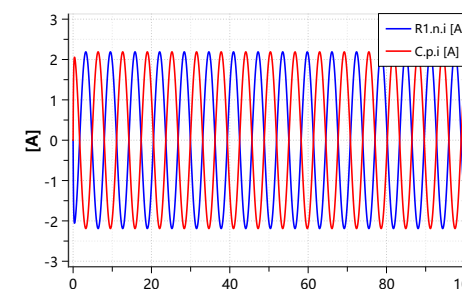
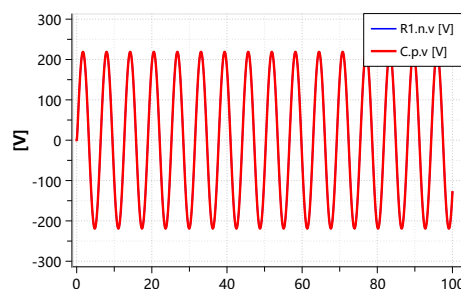


能量流连接器:

- 传递能量(根据广义基尔霍夫定律分解为势变量和流变量);
- 无固定流向, 根据边界条件和物理原理判定;
- 节点势变量相等, 流变量和为零;

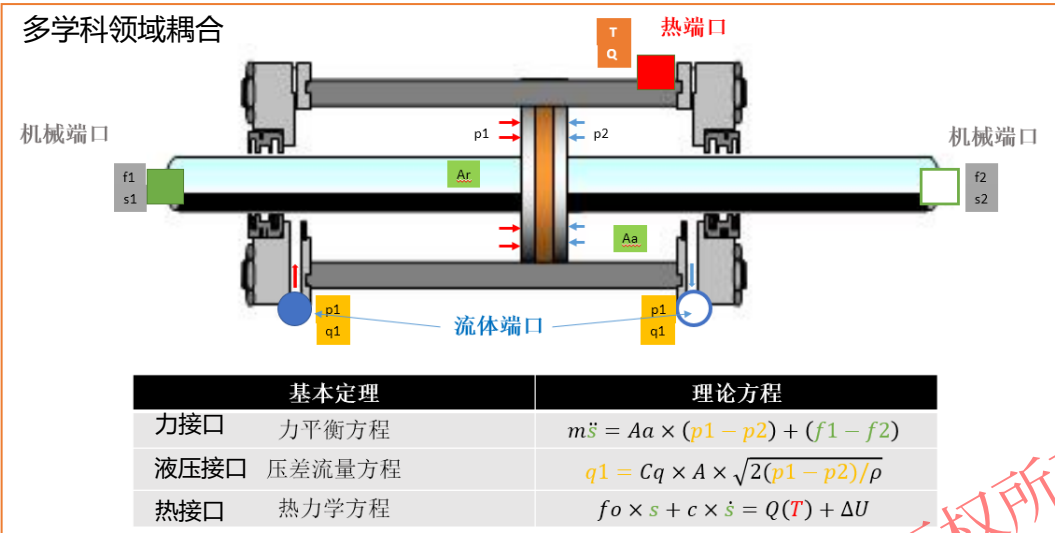


领域	势变量		流变量	
平移机械	s	位移	f	力
转动机械	φ	角度	τ	转矩
电子	v	电压	i	电流
液压	p	压力	$\dot{V}$	流速
热力学	T	温度	$\dot{Q}$	熵流
化学	μ	化学势	$\dot{N}$	粒子流



5.4 接口机制

基于能量流连接器的多领域统一建模



2. 实例化接口，定义参数和变量，根据机理方程描述模型行为，绘制图标

```
model Cylinder "液压缸"  
  parameter Modelica.SIunits.Mass m = 1 "活塞质量";  
  parameter Real Cq = 0.75 "流量系数";  
  ...  
  Modelica.SIunits.Area A "活塞面积";  
  Modelica.SIunits.Density rho "密度";  
  ...  
  Interfaces.Port_a port_a annotation (...);  
  Interfaces.Port_b port_b annotation (...);  
  ...
```

```
equation  
  //力平衡方程  
  m * a = A * a * (p1 - p2) + (f1 - f2);  
  a = der(v);  
  v = der(s);  
  //压差流量方程  
  q1 = Cq * A * sqrt(2 * (p1 - p2) / rho);  
  //热力学方程  
  f0 * s + c * der(s) = Q(T) + deltaU;  
end Cylinder;
```

基于物理原理方程构建的领域转换器可用于不同领域间的能量转换

1. 根据广义基尔霍夫原理构建不同领域接口

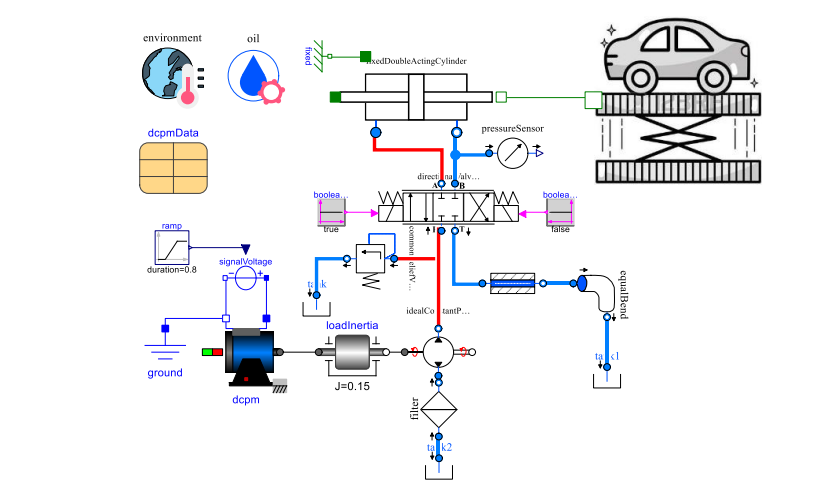
```
connector Port_A  
  Modelica.SIunits.Pressure p;  
  flow Modelica.SIunits.VolumeFlowRate q;  
end Port_A;
```

```
connector Port_B  
  Modelica.SIunits.Pressure p;  
  flow Modelica.SIunits.VolumeFlowRate q;  
end Port_B;
```

```
connector Flange_A  
  Modelica.SIunits.Position s;  
  flow Modelica.SIunits.Force f;  
end Flange_A;
```

```
connector Flange_B  
  Modelica.SIunits.Position s;  
  flow Modelica.SIunits.Force f;  
end Flange_B;
```

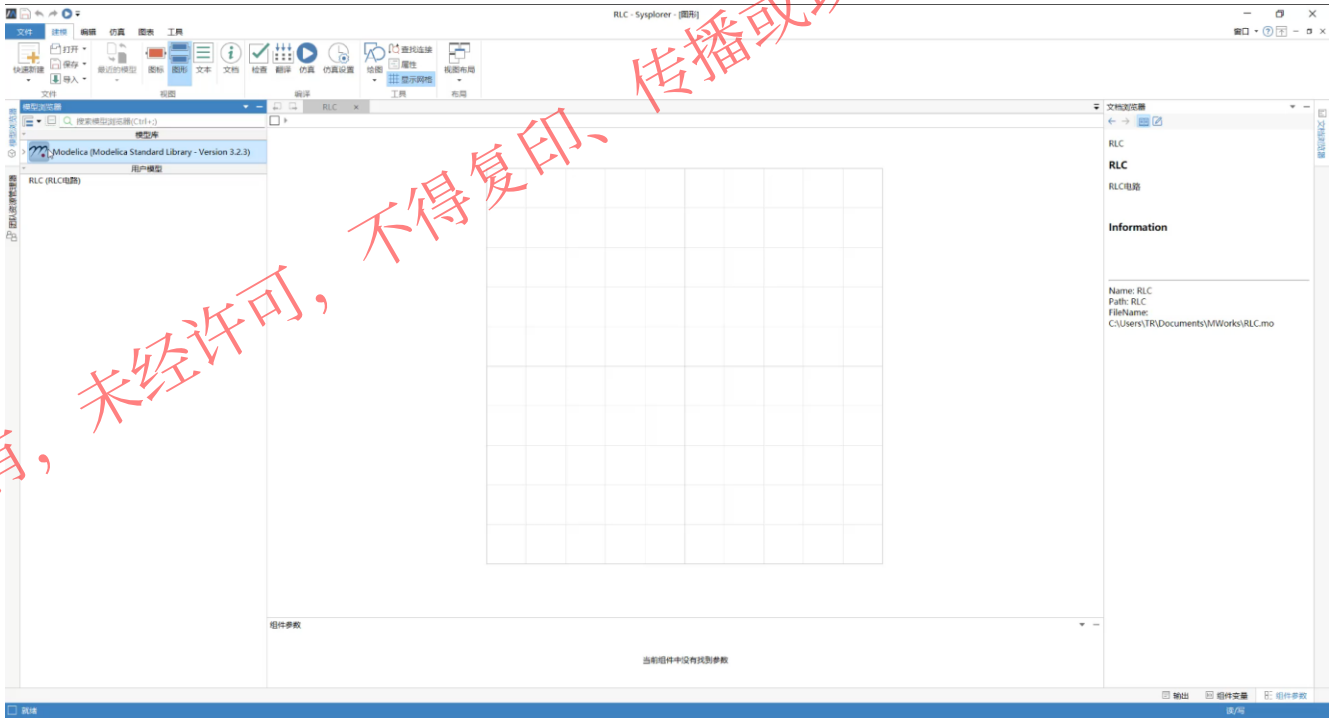
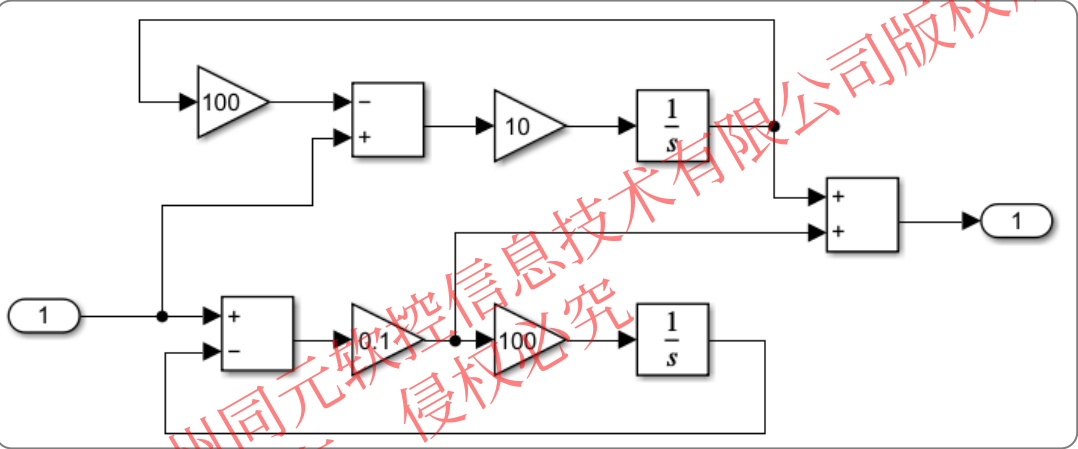
3. 根据物理拓扑关系构建多领域系统模型



5.5 模型封装与集成

问题7：当模型库中的组件不满足需求时，如何解决？

Simulink	Sysplorer
框图式模块封装	元件级(框图和物理模型)模块封装 根据机理方程直接构建
S-Function封装	C/C++封装
M脚本封装	Julia脚本封装
FMU模型封装	FMU模型封装



基于物理拓扑直接封装模型

5.5 模型封装与集成

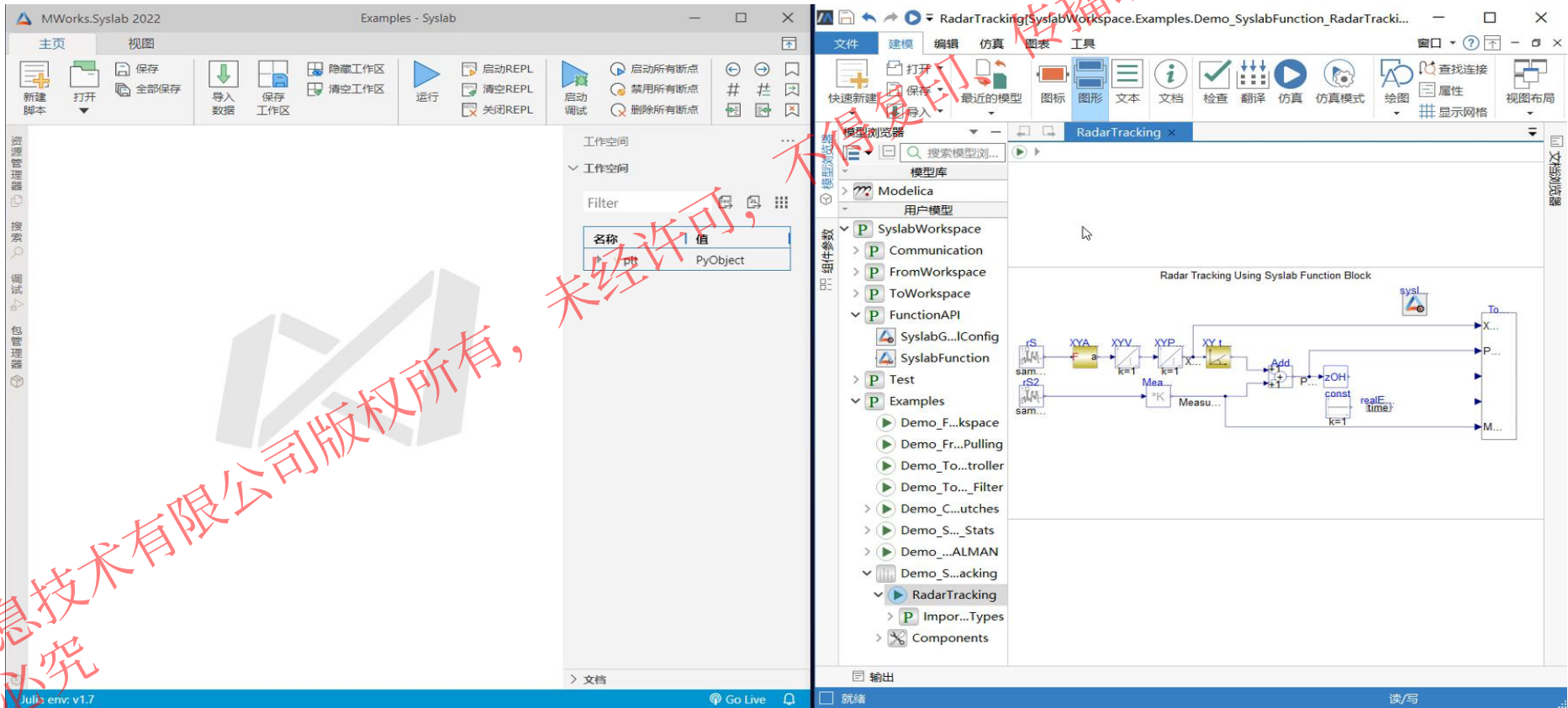
外部模型集成

数据融合

- From/To Workspace

接口融合

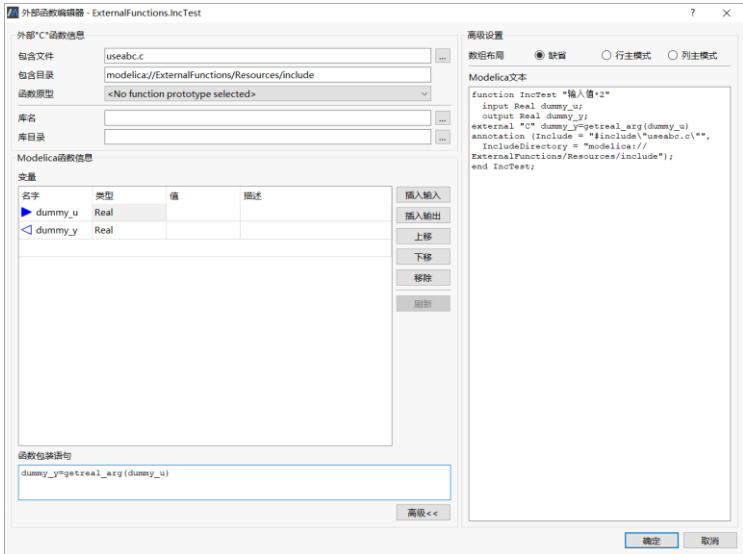
- Sysplorer API
- Syslab Function



外部模型集成

5.5 模型封装与集成

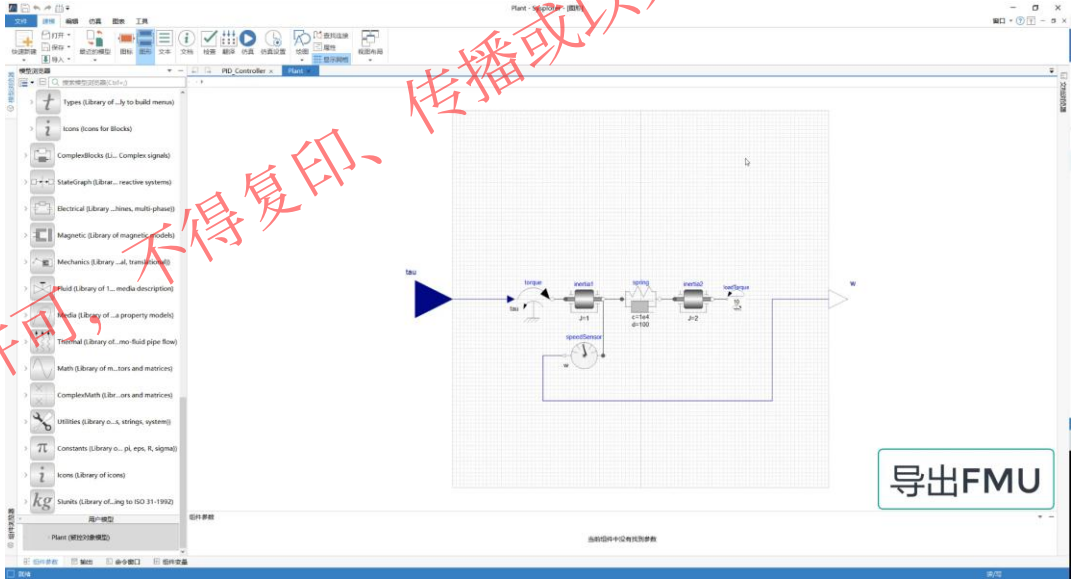
外部模型集成



```
function IncTest "输入值*2"
  input Real dummy_u;
  output Real dummy_y;
  external "C" dummy_y = getreal_arg(dummy_u)
  annotation (Include = "#include\"useabc.c\"",
    IncludeDirectory = "modelica://ExternalFunctions/Resources/include");
end IncTest;
```

模型中除了可以调用使用Modelica 语言编写的函数外，还可以调用C、C++和FORTRAN77 编写的函数。

三种方式：1. 调用c或h文件 2.直接集成C源码 3.调用动态链接库文件dll



导出FMU.mp4

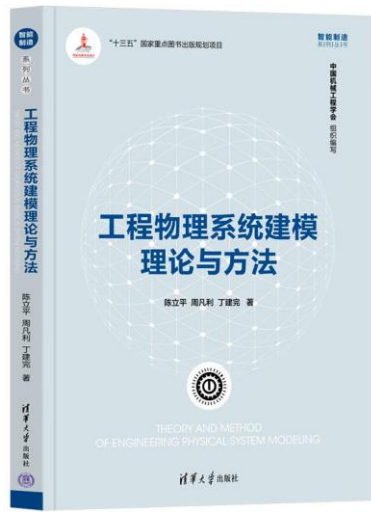
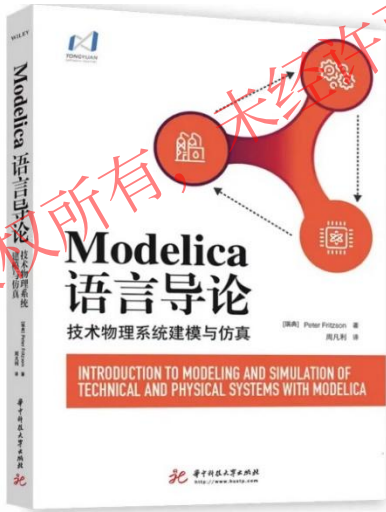
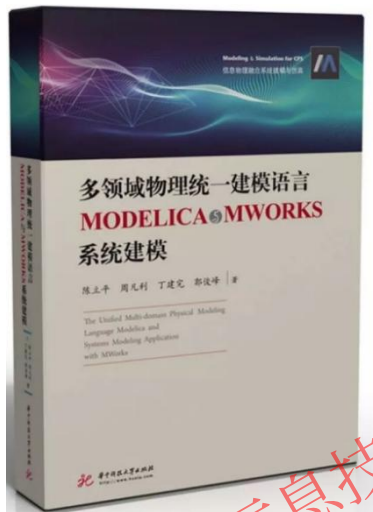
FMU Export				FMU Import			
Name	License	Platforms	Co-Simulation	Model Exchange	Co-Simulation	Model Exchange	
MWorks	\$	Windows	1.0 2.0	1.0 2.0	1.0 2.0	1.0 2.0	

同元MWORKS是中国唯一通过FMI官方测试认证的自主软件，列入FMI官网目录

5.6 Modelica知识点总览

学习材料:

- 1. Modelica语法学习视频: 《Modelica语法》系列
- 2. Modelica by Example: <http://modelicabyexample.globalcrown.com.cn/>
- 3. Modelica语法语义: <https://modelica.org/documents/MLS.pdf>
- 4. Modelica语法学习书籍:



@苏州同元软控信息技术有限公司
版权所有，侵权必究



目录

1. MWOKRS.Sysplorer软件介绍

2. 软件界面简介

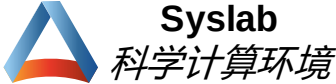
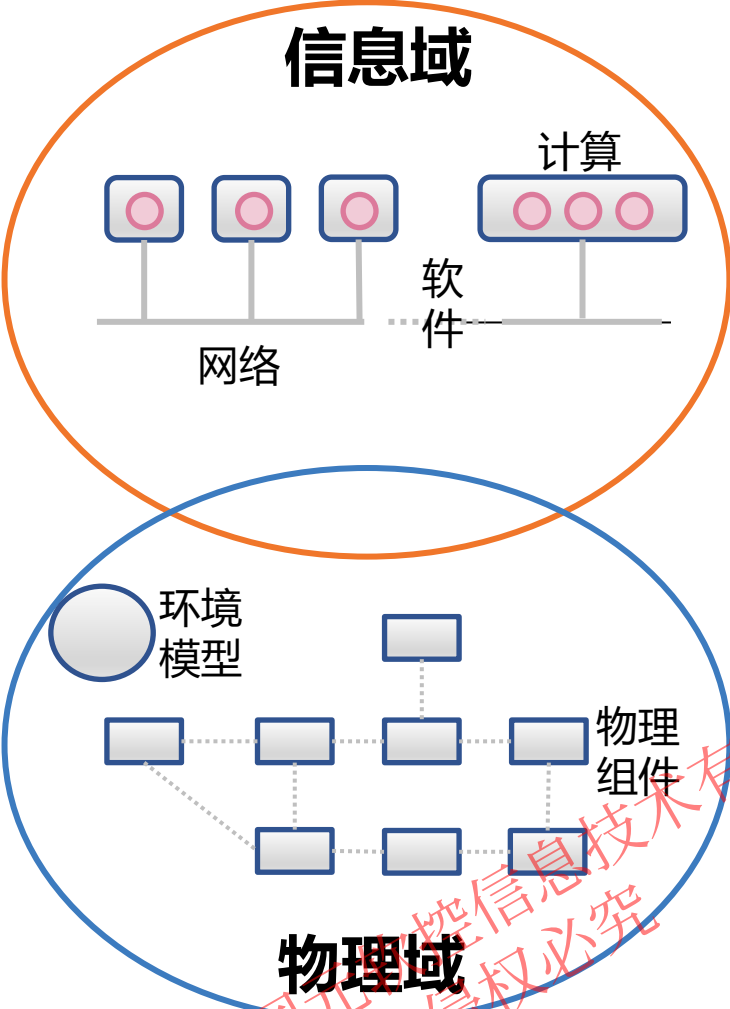
3. 图形化建模

4. Modelica文本建模

5. MWORKS.Sysplorer与Simulink之间的差异

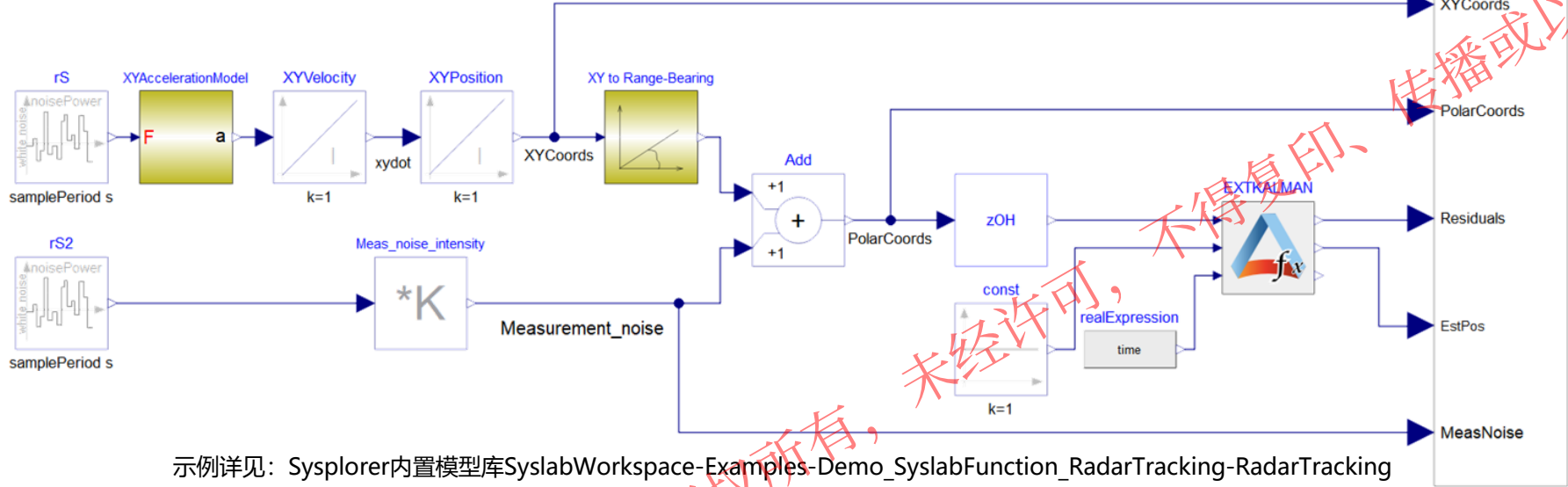
6. Sysplorer与Syslab双向融合

6.1 信息物理融合

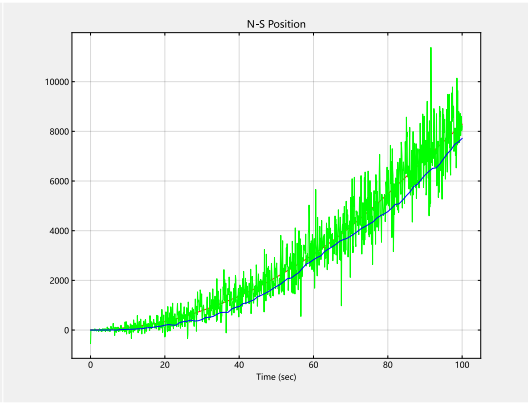
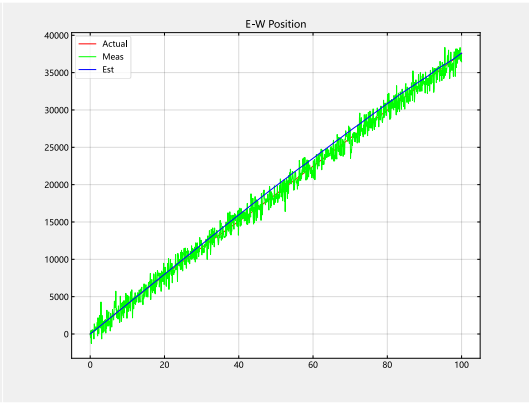
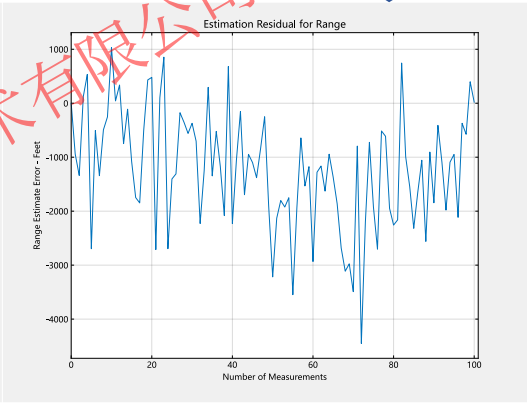
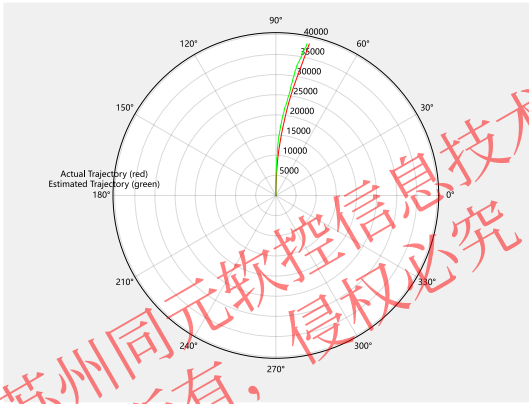


6.1 信息物理融合

syslabGlobalConfig



示例详见: Sysplorer内置模型库SyslabWorkspace-Examples-Demo_SyslabFunction_RadarTracking-RadarTracking



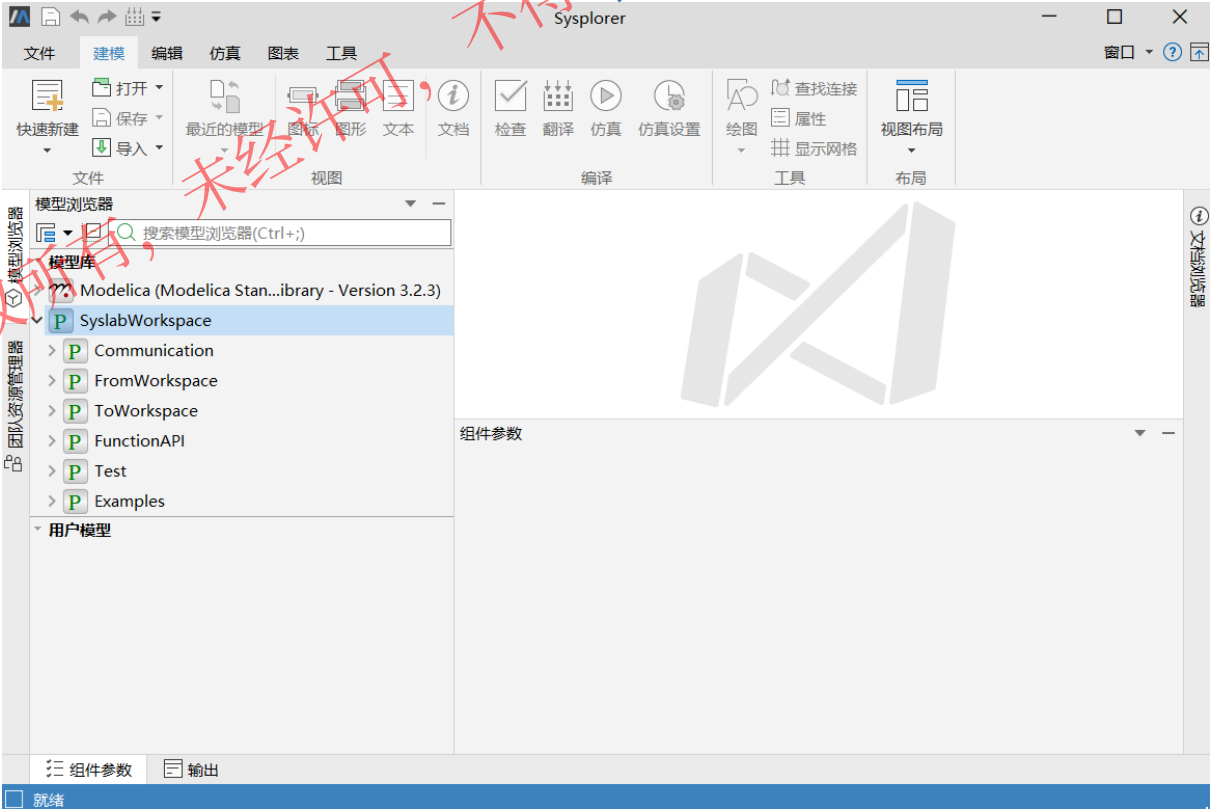
6.2 使用前准备

在Syslab菜单栏中点击Sysplorer，自动打开Sysplorer软件并加载SyslabWorkspace模型库



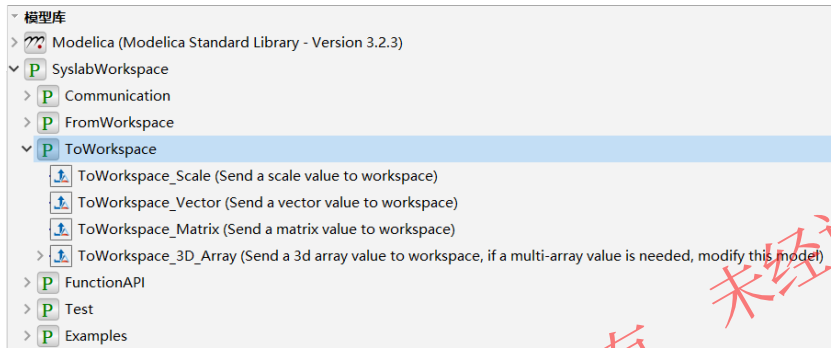
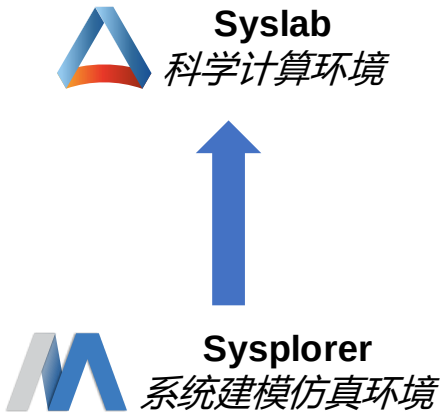
使用须知:

- 1. 如不能打开Sysplorer软件，则需要确认Syslab首选项中Sysplorer可执行文件路径是否正确
- 2. Syslab和Sysplorer均需2022版以上
- 3. Sysplorer软件编译器为64位

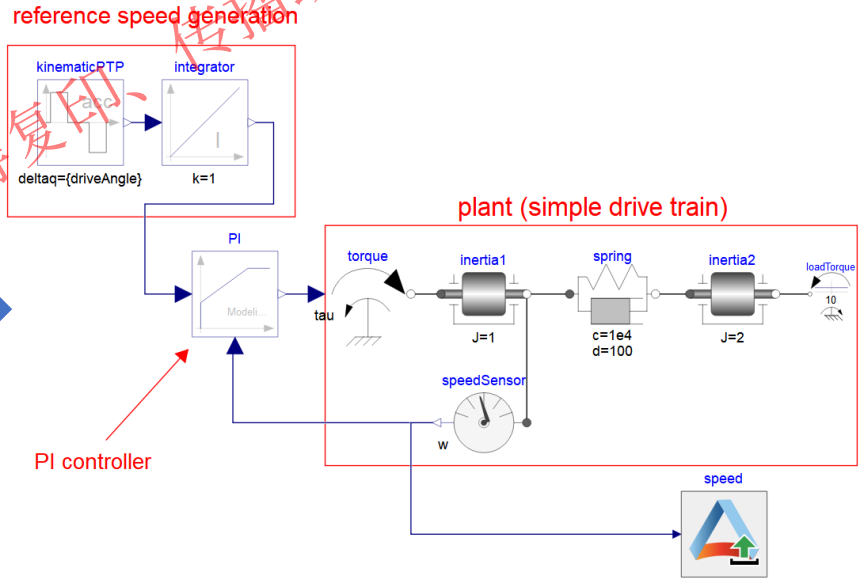


6.3 To Workspace

将Sysplorer的仿真结果发送至Syslab工作空间中



拖拽式建模

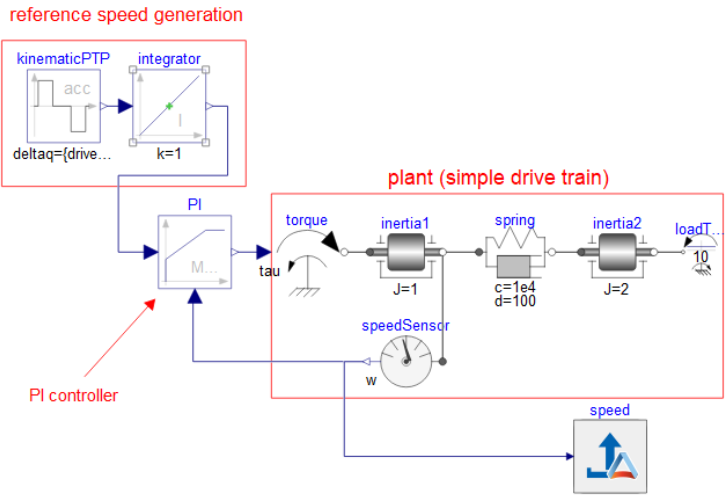


示例详见：Sysplorer内置模型库SyslabWorkspace-Examples-Demo_ToWorkspace_PID_Controller

To Workspace子库中包含4个组件，分别为：

- ToWorkspace_Scale：输出为标量数据
- ToWorkspace_Vector：输出为一维数组
- ToWorkspace_Matrix：输出为矩阵
- ToWorkspace_3D_Array：输出为三维数组

6.3 To Workspace



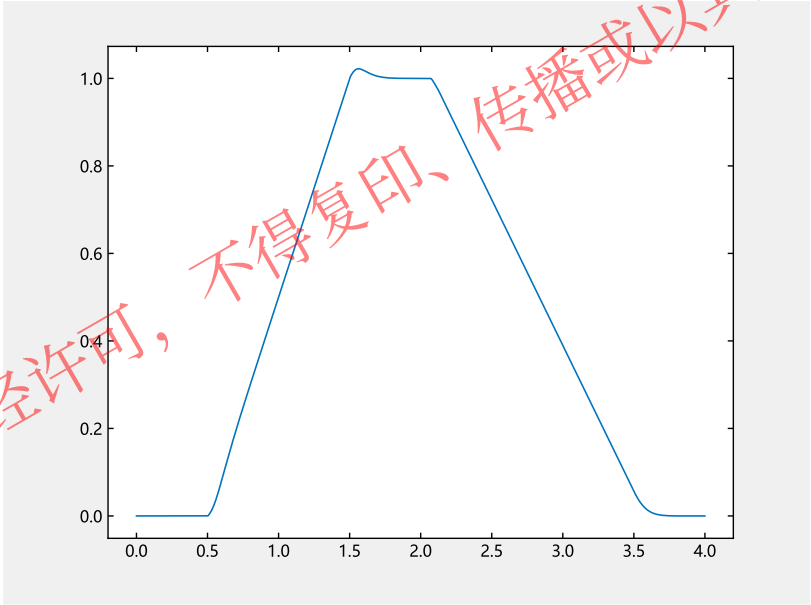
示例详见: Sysplorer内置模型库SyslabWorkspace-Examples-Demo_ToWorkspace_PID_Controller

仿真

Filter	
名称	值
ans	NamedTuple({:tout, :w}, ...)
out	NamedTuple({:tout, :w}, ...)

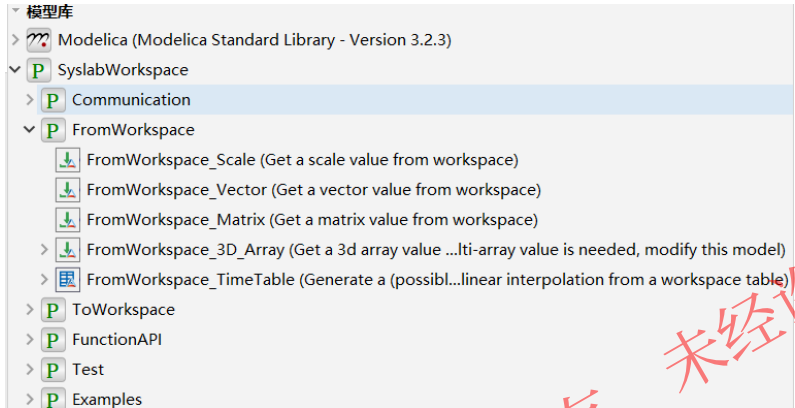
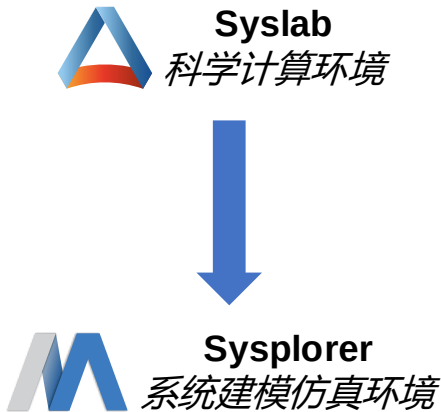
使用Syslab对仿真结果进行处理分析

```
t = out.tout
w = out.w
plot(t, w)
#在Syslab中进行输出变量的后处理
```

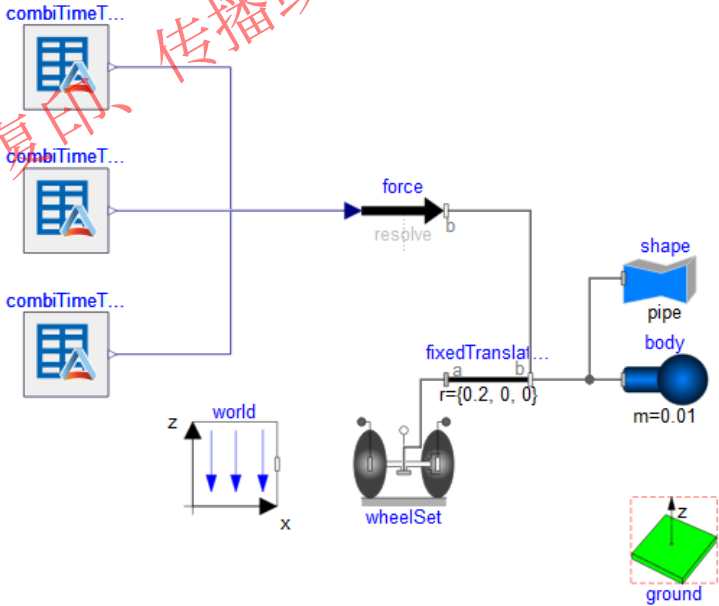


6.4 From Workspace

Sysplorer从Syslab工作空间中读取数据并作为输入



拖拽式建模



from Workspace子库中包含5个组件，分别为：

- from Workspace_Scale：获取标量数据
- from Workspace_Vector：获取一维数组
- from Workspace_Matrix：获取二维数组
- from Workspace_3D_Array：获取三维数组
- from WorkspaceTimeTable：获取表格矩阵，并通过线性插值来生成（可能是不连续的）信号

注意： from Workspace传递的量均为**变量**，不能直接作为组件参数，需将组件参数处理为输入接口或变量。

示例详见：Sysplorer内置模型库SyslabWorkspace-Examples-Demo_FromWorkspace_RollingWheelSetPulling

6.4 From Workspace

```
# Julia代码
table = [0 1 0 0
         1 1 0 0
         2 0 2 0
         3 0 2 0]
combiTimeTableX = table[:,[1,2]] #取1,2两列
combiTimeTableY = table[:,[1,3]] #取1,3两列
combiTimeTableZ = table[:,[1,4]] #取1,4两列
```

↓ 代码运行

Filter

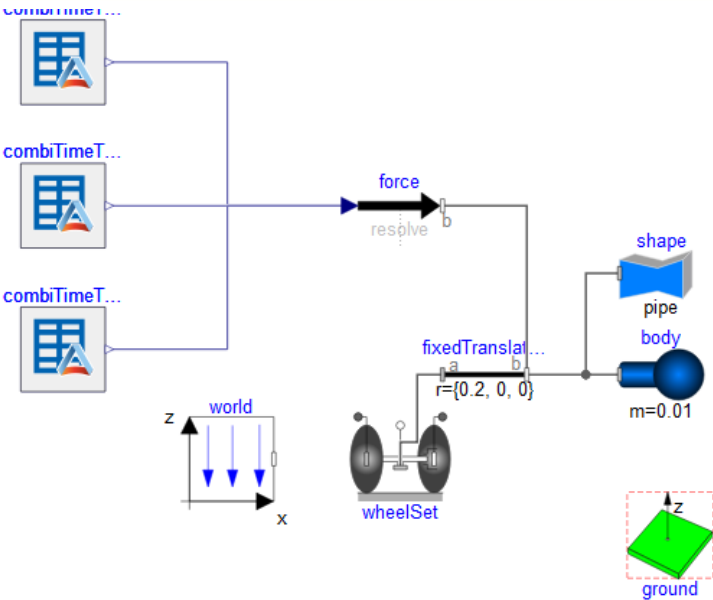
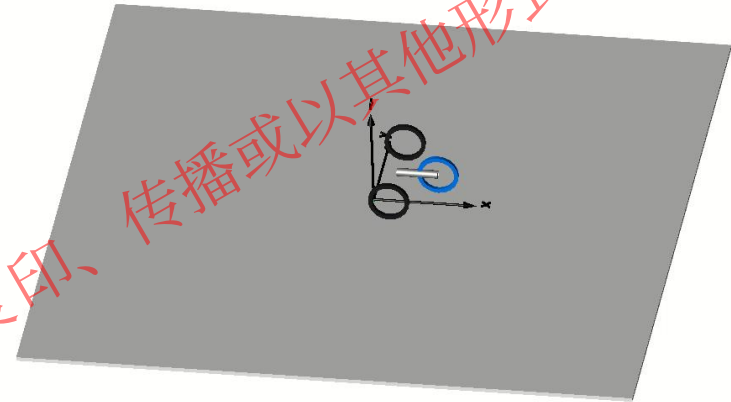
名称	值
ans	Matrix{Int64} with 4×2 e...
combiTimeTableX	Matrix{Int64} with 4×2 e...
combiTimeTableY	Matrix{Int64} with 4×2 e...
combiTimeTableZ	Matrix{Int64} with 4×2 e...
table	Matrix{Int64} with 4×4 e...

组件参数		
常规		
参数		
varName	*combiTimeTableZ*	Timetable variable name
row_dims	4	Timetable row dims
offset	0	Offset of output signal
startTime	0	Output = offset for time < startTime

使用fromworkSpace组件

设定参数

变量名与工作空间变量名需完全一致



示例详见：Sysplorer内置模型库SyslabWorkspace-Examples-Demo_FromWorkspace_RollingWheelSetPulling

6.4 From Workspace

From Workspace Functions 以函数的形式从 Syslab 工作区中读取数据，因此可以直接将获取的数据作为其他组件的**参数值**。

FromWorkspace

Functions

Real2Int

Real2Bool

FwReal (获取实型数据)

FwInt (获取整型数据)

FwBool (获取布尔型数据)

FwRealVector (获取实型向量数据)

FwIntVector (获取整型向量数据)

FwBoolVector (获取布尔型向量数据)

FwRealMatrix (获取实型矩阵数据)

FwIntMatrix (获取整型矩阵数据)

FwBoolMatrix (获取布尔型矩阵数据)

FwReal3DArray (获取实型3维数组数据)

FwInt3DArray (获取整型3维数组数据)

FwBool3DArray (获取布尔型3维数组数据)

HelpGetReal3DArray (get multi-array value)

```
#Syslab脚本:
# 标量
i_val = 5
f_val = 7.5
b_val = true

# 向量
i_vec = [1, 2, 3]
f_vec = [1, 2.5, 3.5]
b_vec = [true, false, true]

# 矩阵
i_mtx = [1 2 3; 4 5 6]
f_mtx = [1 2.5 3.5; 4 5.5 6.5]
b_mtx = [true false true; false true false]

# 三维数组
i_arr = fill(1, (2, 3, 4))
i_arr[2, 1, 3] = 17

f_arr = fill(2.5, (2, 3, 4))
f_arr[2, 1, 3] = 17

b_arr = fill(true, (2, 3, 4))
b_arr[2, 1, 3] = false
```

Syslab中计算出结果

```
model SubModel
import SyslabWorkspace.FromWorkspace.Functions.*;

//标量
parameter Integer int_x = FwInt("i_val") "整型标量";
parameter Real real_x = FwReal("f_val") "实型标量";
parameter Boolean bool_x = FwBool("b_val") "布尔型标量";

//向量
parameter Integer int_vec[:] = FwIntVector("i_vec", 3) "整型向量";
parameter Real real_vec[:] = FwRealVector("f_vec", 3) "实型向量";
parameter Boolean bool_vec[:] = FwBoolVector("b_vec", 3) "布尔型向量";

//矩阵
parameter Integer int_mtx[:,:] = FwIntMatrix("i_mtx", 2, 3) "整型矩阵";
parameter Real real_mtx[:,:] = FwRealMatrix("f_mtx", 2, 3) "实型矩阵";
parameter Boolean bool_mtx[:,:] = FwBoolMatrix("b_mtx", 2, 3) "布尔型矩阵";

//三维数组
parameter Integer int_arr[:,,:] = FwInt3DArray("i_arr", 2, 3, 4) "整型三维数组";
parameter Real real_arr[:,,:] = FwReal3DArray("f_arr", 2, 3, 4) "实型三维数组";
parameter Boolean bool_arr[:,,:] = FwBool3DArray("b_arr", 2, 3, 4) "布尔型三维数组";

annotation (...);
end SubModel;
```

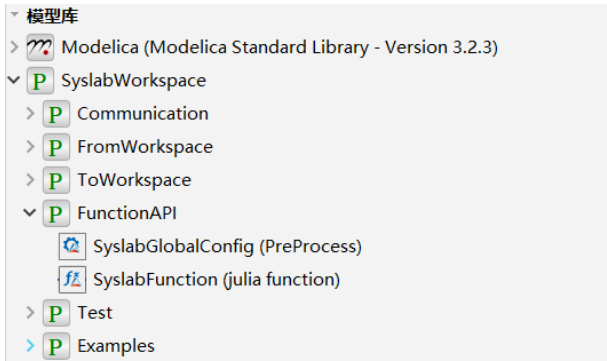
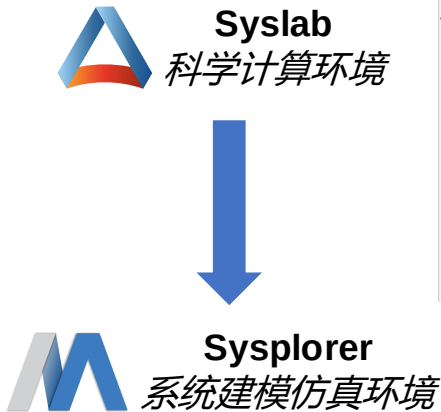
Sysplorer中模型通过函数读取工作空间值作为参数

注：函数调用见：Modelica语法-函数；参数定义见Modelica语法-类与内置类型

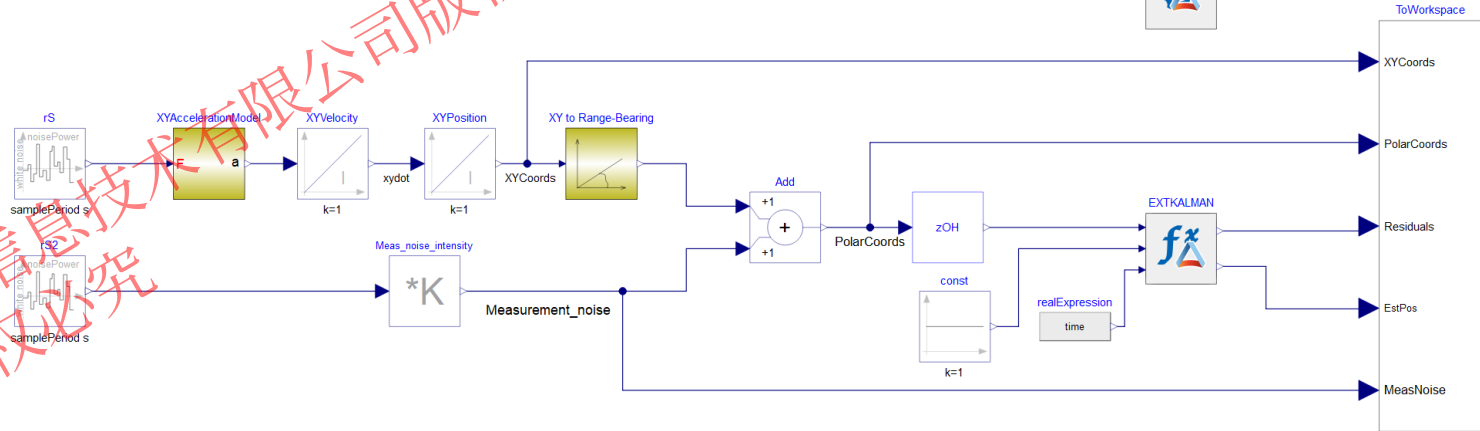


6.5 Syslab Function

将Syslab中构建的Julia复杂算法封装至Sysplorer中



拖拽式建模



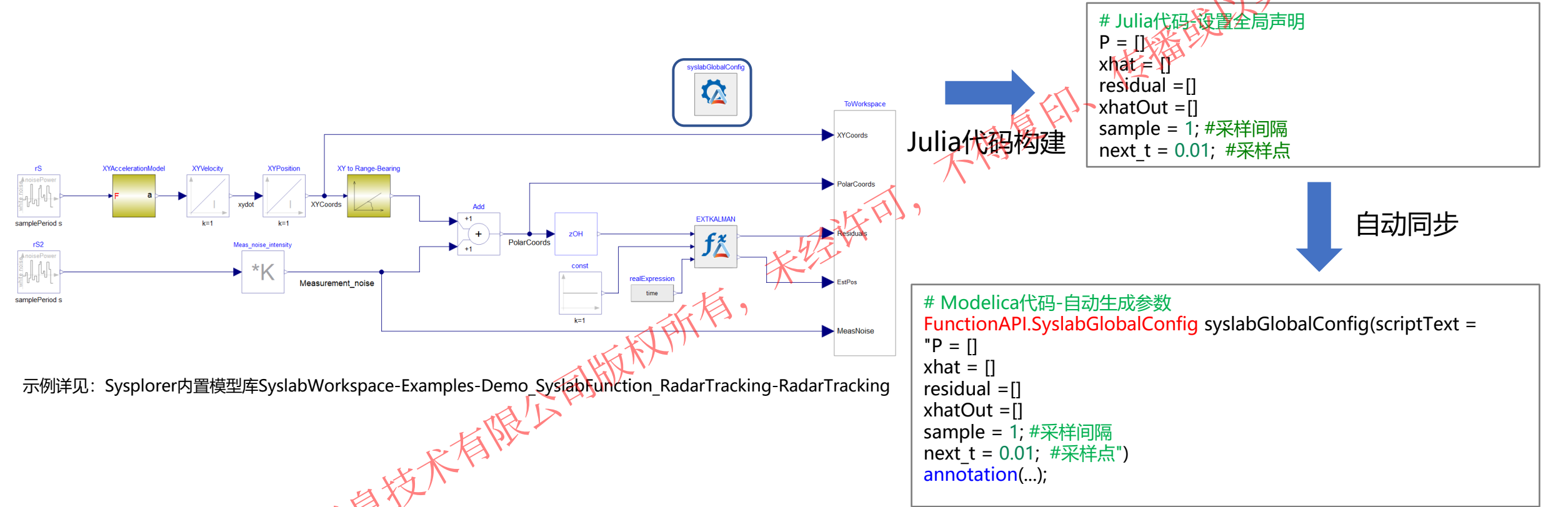
示例详见: Sysplorer内置模型库SyslabWorkspace-Examples-Demo_SyslabFunction_RadarTracking-RadarTracking

Function API中包含2个组件:

- SyslabGlobalConfig: 用于全局声明, 包括导入包及全局变量声明等。
- SyslabFunction: 用于嵌入 Julia函数, 并将Syslab Function模块的输入和输出数据指定为参数和返回值。

4.5 Syslab Function

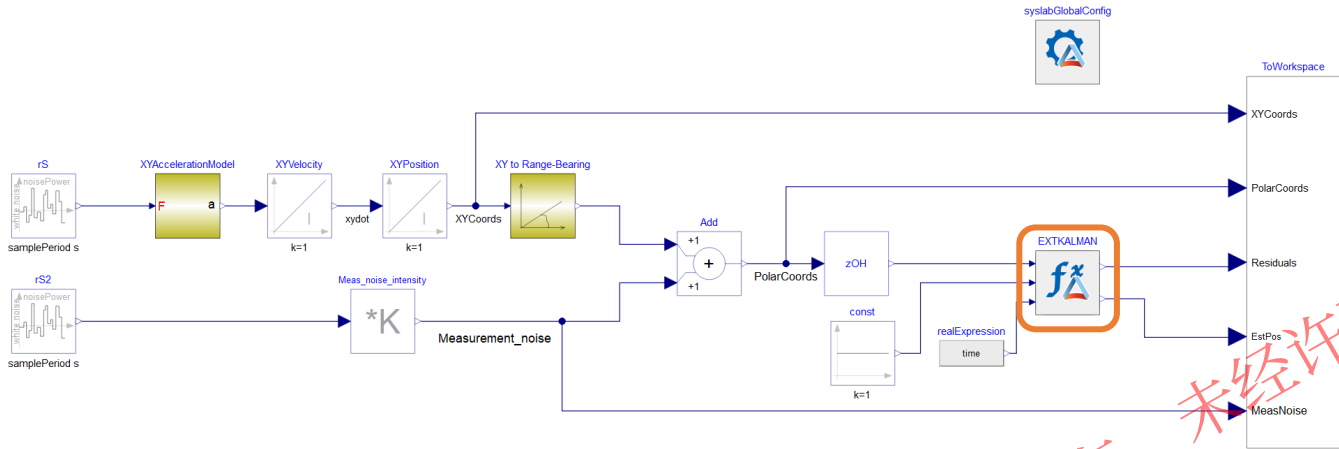
右击SyslabGlobalConfig模型，选择Syslab初始化配置，即可转到Syslab中的全局声明代码



示例详见：Sysplorer内置模型库SyslabWorkspace-Examples-Demo SyslabFunction_RadarTracking-RadarTracking

6.5 Syslab Function

右击SyslabFunction模型，选择编辑Syslab函数脚本，即可转到Syslab中的算法函数代码



Julia算法

```
function EXTKALMAN(meas, deltat, time)

# Initialization
global P;
global xhat;
global residual;
global xhatOut;
global next_t; #采样点
global sample; #采样间隔(s)

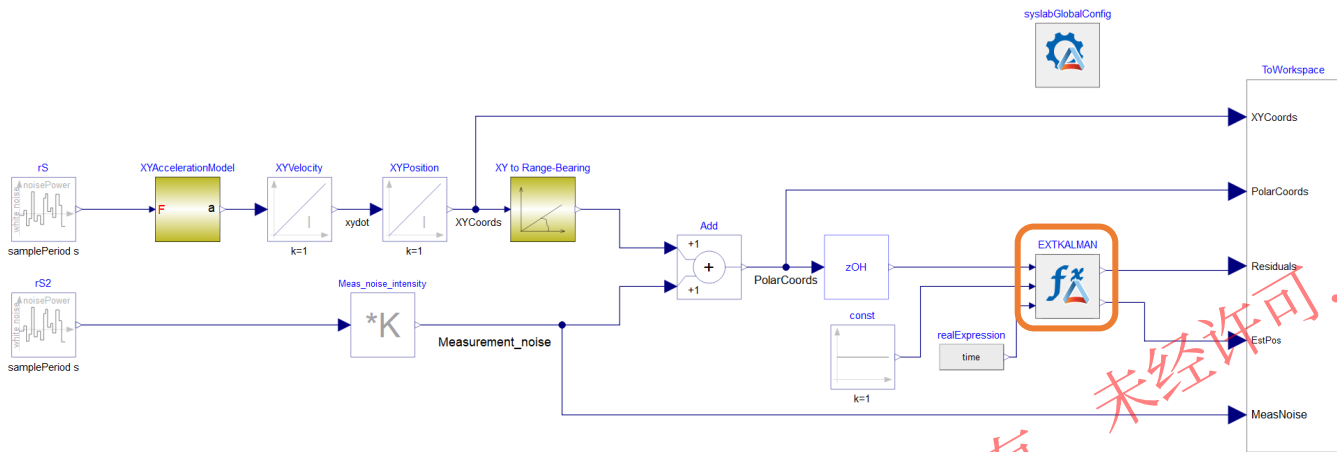
if isempty(P)
    xhat = [0.001; 0.01; 0.001; 400;]; # 4x1矩阵
    P = zeros(4, 4);
end

...
```

注意：
SyslabFunction组件认为脚本中的第一个函数为本组件的主函数，其他函数均为服务于主函数的辅助函数。

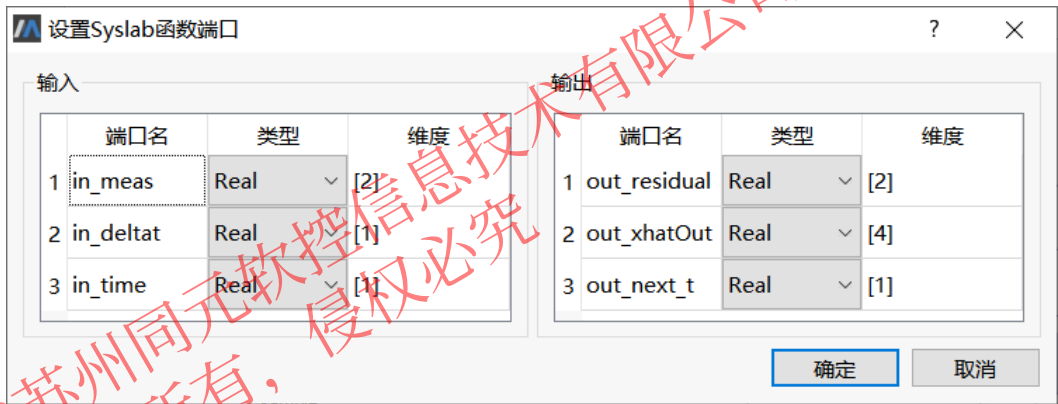
6.5 Syslab Function

SyslabFunction模型会根据第一个函数的输入参数和返回值自动生成输入输出接口；



示例详见：Sysplorer内置模型库SyslabWorkspace-Examples-Demo_SyslabFunction_RadarTracking-RadarTracking

查看输入输出接口

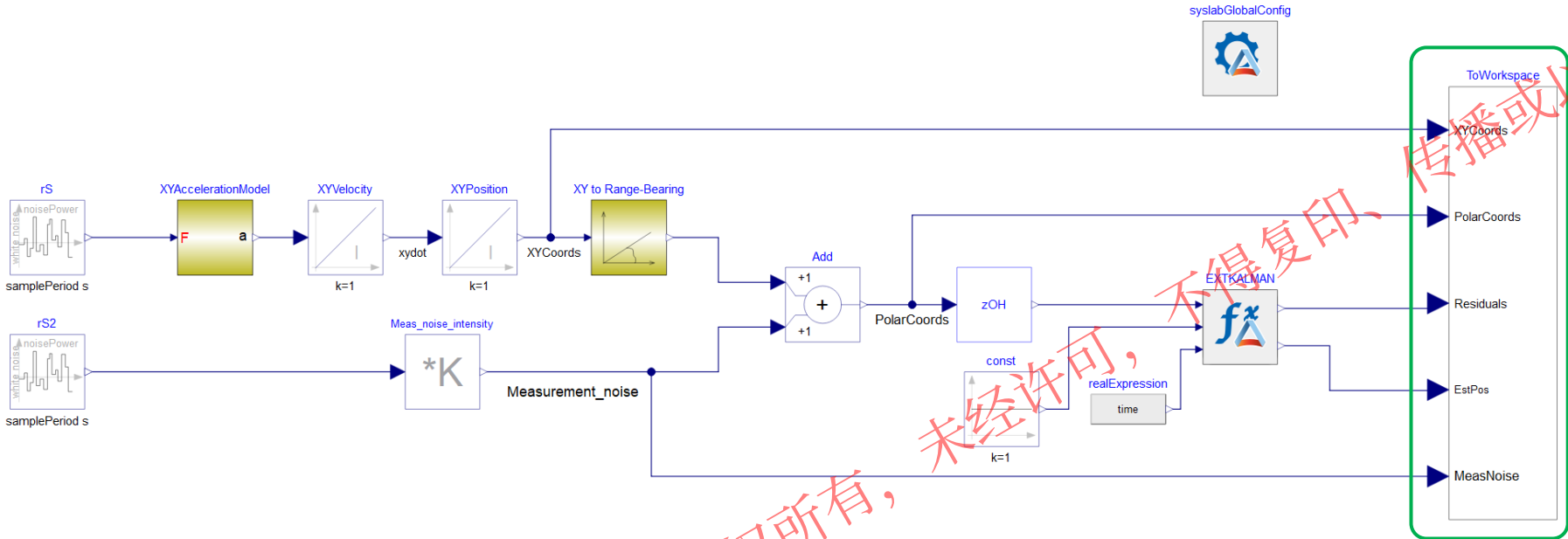


说明：

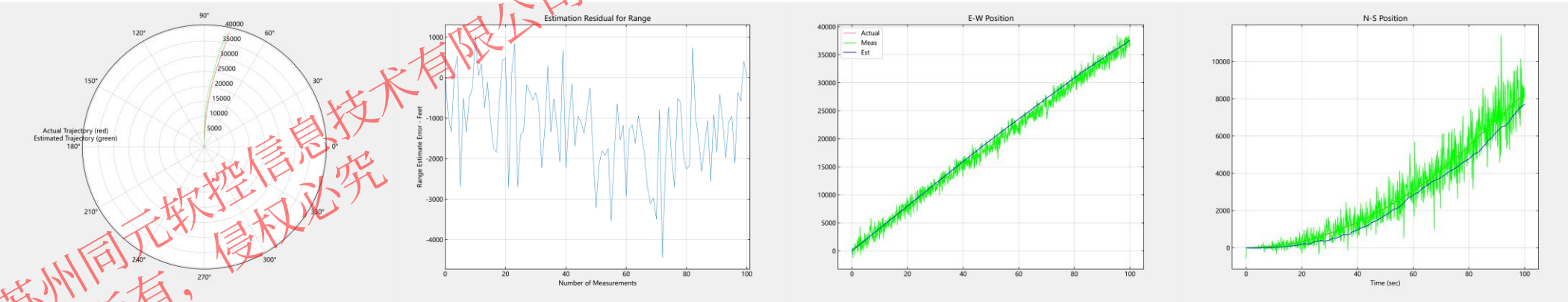
- 主函数的输入不要指定类型，不要指定具名参数；
- 主函数的输出必须使用return指定，且必须为函数体中已经出现的变量符号；

6.5 Syslab Function

使用To Workspace,
在Syslab中处理仿真结果



示例详见: Sysplorer内置模型库SyslabWorkspace-Examples-Demo_SyslabFunction_RadarTracking-RadarTracking

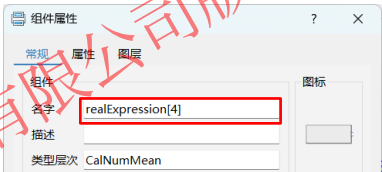


6.5 Syslab Function

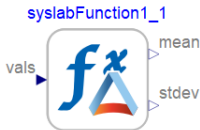
此示例主要介绍如何使用SyslabFunction对输入的向量求取平均值和标准差

操作步骤

- 新建模型，拖入realExpression和syslabFunction组件
- 点击realExpression，右键选择“属性”，修改模型名称为“realExpression[4]”；
- 修改realExpression组件参数为“{4,5,6,2}”；
- 点击syslabFunction1_1，右键选择“设置Syslab函数端口”将维度设置为[4]；



realExpression路径: Modelica.Blocks.Sources.RealExpression

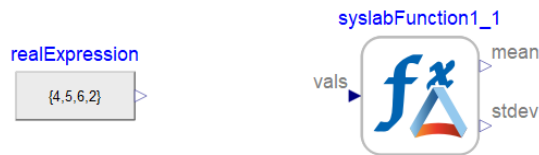


示例详见附件: CalNumMean

6.5 Syslab Function

操作步骤

- 点击syslabFunction1_1，右键选择“编辑Syslab函数脚本”
- 在Syslab脚本编辑界面输入右侧代码



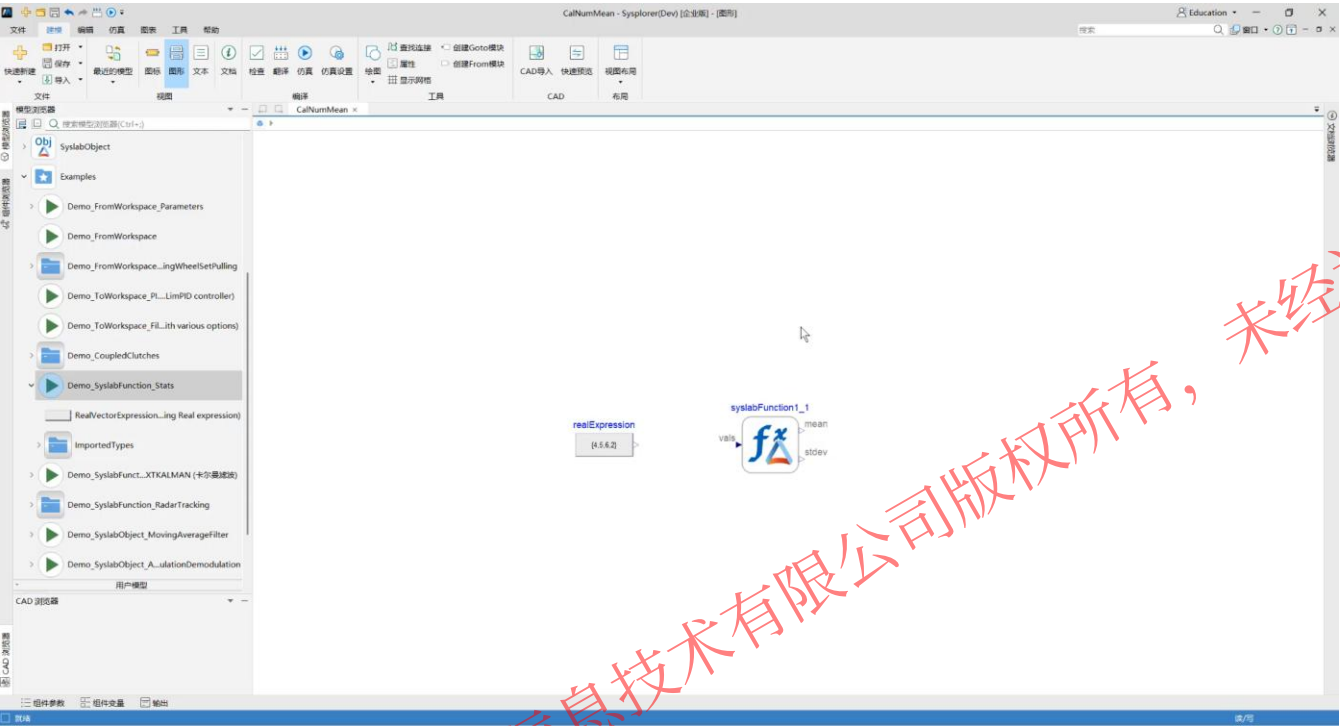
```
function stats(vals)
    # 计算平均值与标准差
    len = length(vals);
    mean = avg(vals,len);
    stdev = sqrt(sum(((vals.-avg(vals,len)).^2))/len);
    return mean,stdev
end

#求平均值
function avg(array,size)
    mean = sum(array)/size;
end
```

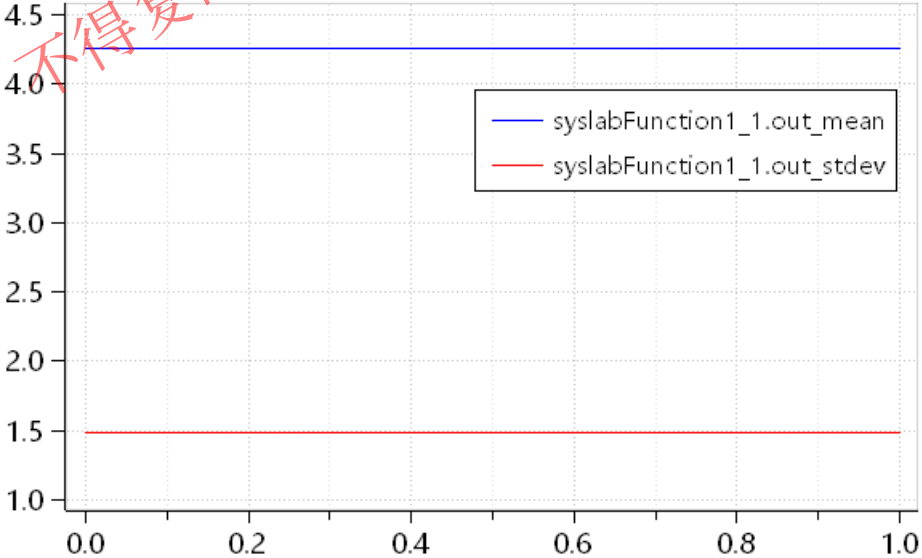


6.5 Syslab Function

操作步骤：完成连线，进行仿真，查看结果



组件连线.mp4



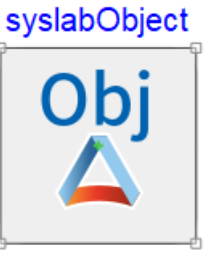
6.5 Syslab Function

将Syslab中Julia对象进行动态系统建模和处理流式数据

**Syslab**
科学计算环境

↓

**Sysplorer**
系统建模仿真环境

**syslabObject**
Obj

拖拽组件

模型库

- Modelica (Modelica Standard Library - Version 3.2.3)
- SyslabWorkspace
 - ModelicaDependence
 - Communication
 - FromWorkspace
 - ToWorkspace
 - FunctionAPI
 - SyslabObject
 - SyslabObject (Julia object block)
 - Test
 - Examples (Support for Modelica 3.2.3)

Syslab对象参数

SyslabObject

Template for syslab object block.

源代码

参数

Syslab对象文件路径:

c:/Users/Lixin Hui/Desktop/Test/SyslabObject.jl

浏览...

新建

确定

取消

新建或选择脚本路径

```
using ObjectOriented
@ooodef mutable struct SyslabObject
    # Description (用于生成Modelica组件描述)
    # Template for syslab object block.

    # Parameter (用于生成Modelica 组件参数, 包括名称、类型、描述)
    # 格式: 参数名::参数类型 = 参数值 # 参数注释
    # 例如:
    gain::Real = 1.0 # 增益

    # Private (Julia内部变量)
    # 格式: 变量名::类型 = 值 # 变量说明
    # 例如:
    _count::Integer = -1 # 计数器

    # Methods (主要调用算法, 包括setupImpl, stepImpl, releaseImpl)

    # 初始化函数: 函数名固定, 函数形参与stepImpl函数形参一致
    function setupImpl(self, u)
        self._count = 0
        # ...
        return nothing
    end #setupImpl

    # 单步计算函数: 函数名固定, 第一个函数形参必须是self,
    # 其余函数形参将作为Modelica组件的输入端口, 函数返回值作为输出端口
    function stepImpl(self, u)
        self._count += 1
        # ...
        y = u * self.gain
        return y
    end #stepImpl

    # 释放资源函数: 函数名固定, 且只能有一个函数参数self
    function releaseImpl(self)
        # ...
        return nothing
    end #releaseImpl

    # 其它自定义函数, 第一个函数形参必须是self
    # function xx(self)
    #     ...
    # end

end
```

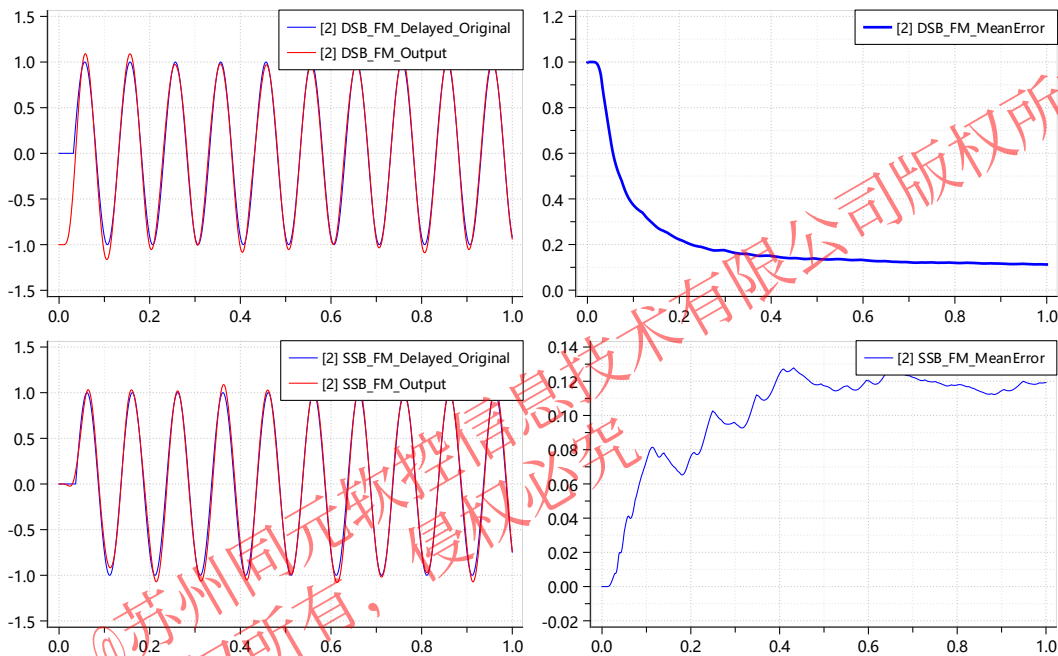
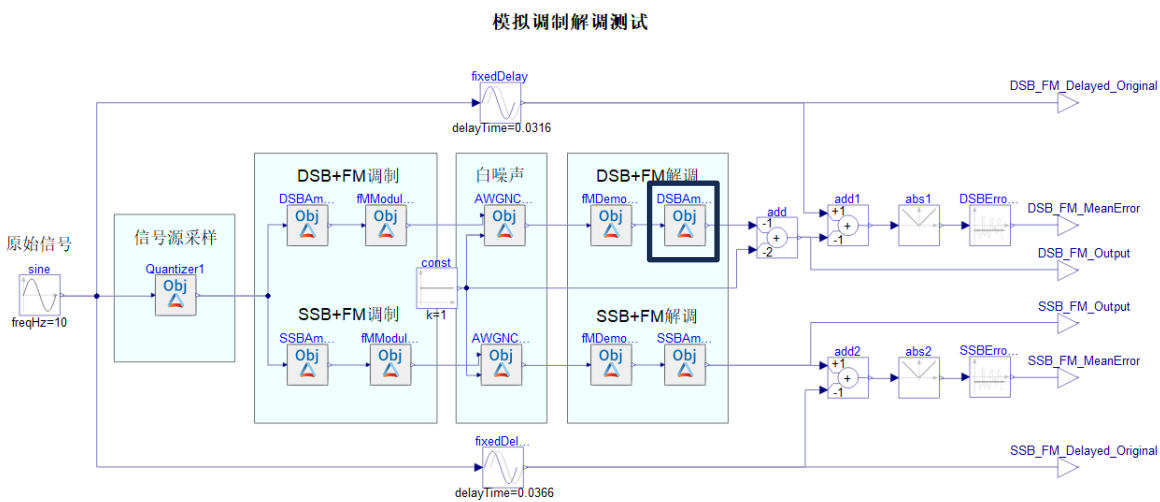
根据提供的默认模板文件配置脚本, 参数面板和输入输出接口自动生成

123

Copyright © 2025 苏州同元软控信息技术有限公司
All rights reserved

TONGYUAN
Software Control

6.5 Syslab Function



```
using TyCommunication
using TyBase
using TyMath
using TySignalProcessing
using ObjectOriented
@oodef mutable struct DSBAmplitudeDemodulator
    # Description
    # Parameter
    InputSignalOffset::Float64 = 1
    CarrierFrequency::Float64 = 100
    InitialPhase::Float64 = 0
    SampleFrequency::Float64 = 10000
    FilterOrder::Int64 = 4
    CutoffFrequency::Float64 = 100
    PassbandRipple::Float64 = 0.1
    StopbandAttenuation::Float64 = 50
    LowPassFilterMethod::String = "Butterworth"
    # Private
    Samplepoint = 0
    b = 0
    a = 0
    zi = 0
    # Methods
    function setupImpl(self, data)
        #...
        if self.SampleFrequency <= 0
            throw(ArgumentError("Fs must be a real, positive scalar."))
        end
        # check that Fs must be greater than 2*Fc
        if self.SampleFrequency < 2 * self.CarrierFrequency
            throw(ArgumentError("Fs must be at least 2*Fc."))
        end
        if self.LowPassFilterMethod == "Butterworth"
            .....
        end
        self.zi = zeros(Float64, (max(length(self.a),length(self.b)) - 1))
        return nothing
    end #setupImpl

    function stepImpl(self, data)
        #...
        temp = data .* cos(2 * pi * self.CarrierFrequency * self.Samplepoint / self.SampleFrequency + self.InitialPhase)
        .....
        return out
    end #stepImpl

    function releaseImpl(self)
        #...
        return nothing
    end #releaseImpl
end
```

组件参数

常规			
参数			
startTime	0	s	sample start time
period	0.0001	s	sample period
InputSignalOffset	1		
CarrierFrequency	100		
InitialPhase	0		
SampleFrequency	10000		
FilterOrder	4		
CutoffFrequency	20		
PassbandRipple	0.1		
StopbandAttenuati...	50		
LowPassFilterMethod	"Butterworth"		

6.6 Sysplorer API

类别	命令接口	含义
系统命令	ClearScreen	清空命令窗口
	SaveScreen	保存命令窗口内容至文件
	ChangeDirectory	更改工作目录
	ChangeSimResultDirectory	更改仿真结果目录
	RunScript	执行脚本文件
	GetLastErrors	获取上一条命令的错误信息
	ClearAll	移除所有模型
	Echo	打开或关闭命令执行状态的输出
	Exit	退出MWorks.Sysplorer
文件命令	OpenModelFile	加载指定的Modelica模型文件
	LoadLibrary	加载Modelica模型库
	ImportFMU	导入FMU文件
	EraseClasses	删除子模型或卸载顶层模型
	ExportIcon	把图标视图导出为图片
	ExportDiagram	把组件视图导出为图片
	ExportDocumentation	把模型文档信息导出到文件
	ExportFMU	模型导出为FMU
	ExportVeristand	模型导出为Veristand模型
	ExportSFunction	模型导出为Simulink的S-Function

Syslab 命令窗口(REPL)或脚本中可直接调用Sysplorer API 接口

类别	命令接口	含义
仿真命令	OpenModel	打开模型窗口
	CheckModel	检查模型
	TranslateModel	翻译模型
	SimulateModel	仿真模型
	RemoveResults	移除所有结果
	RemoveResult	移除最后一个结果
	ImportInitial	导入初值文件
	ExportInitial	导出初值文件
	GetInitialValue	获取变量初值
	SetInitialValue	设置变量初值
	ExportResult	导出结果文件
	SetCompileSolver64	设置翻译时编译器平台位数
	GetCompileSolver64	获取翻译时编译器平台位数
	SetCompileFmu64	设置fmu导出时编译器平台位数
	GetCompileFmu64	获取fmu导出时编译器平台位数

6.6 Sysplorer API

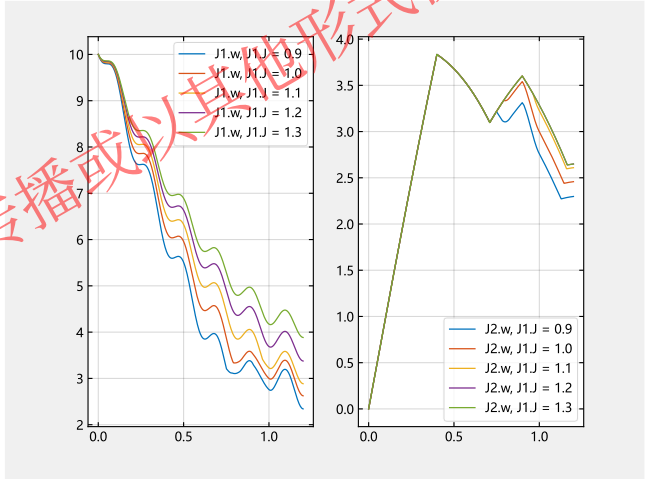
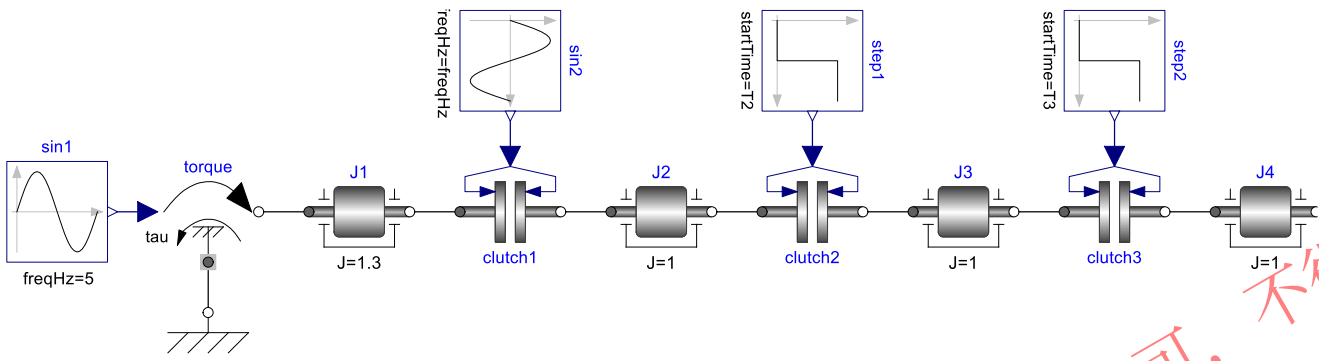
类别	命令接口	含义
曲线命令	CreatePlot	按指定的设置创建曲线窗口
	Plot	在最后一个窗口中绘制指定变量的曲线
	RemovePlots	关闭所有曲线窗口
	ClearPlot	清除曲线窗口中的所有曲线
	ExportPlot	曲线导出
文件命令	CreateAnimation	新建动画窗口
	RemoveAnimations	关闭所有动画窗口
	RunAnimation	播放动画
	StopAnimation	停止动画播放
	Animation Speed	设置动画播放速度

类别	命令接口	含义
模型对象操作命令	GetClasses	获取指定模型的嵌套类型
	GetComponents	获取指定模型的嵌套组件
	GetParamList	获取指定组件前缀层次中的参数列表
	GetModelDescription	获取指定模型的描述文字
	SetModelDescription	设置指定模型的描述文字
	GetComponentDescription	获取指定模型中组件的描述文字
	SetComponentDescription	设置指定模型中组件的描述文字
	SetParamValue	设置当前模型指定参数的值
	SetModelText	修改模型的Modelica()文本内容
	GetExperiment	获取模型仿真配置

关于Sysplorer API命令可见Syslab中文帮助文档中 **"Sysplorer API"**

6.6 Sysplorer API

在Syslab中对Sysplorer模型进行参数扫动分析



#软件恢复初始化

```
Sysplorer.ClearAll()
```

#加载Modelica3.2.1

```
Sysplorer.LoadLibrary("Modelica", "3.2.1")
```

打开模型

```
Sysplorer.OpenModel("Modelica.Mechanics.Rotational.Examples.CoupledClutches",  
Sysplorer.ModelView.Diagram)
```

扫动变量值, 扫动序列为: 0.9、1.0、1.1、1.2、1.3

```
para_sweep = [0.9, 1.0, 1.1, 1.2, 1.3]
```

结果数组

```
J1_w_list = []
```

```
J2_w_list = []
```

```
time_list = []
```

开始实验

```
for i in 1:5
```

```
println("sweep case-$i: J1.J = $(para_sweep[i])")
```

设置变量

```
Sysplorer.SetParamValue("J1.J", string(para_sweep[i]))
```

进行仿真

```
Sysplorer.SimulateModel("Modelica.Mechanics.Rotational.Examples.CoupledClutches",  
stop_time = 1.2, algo = Sysplorer.Integration.Dassl)  
println("case-$i finished")
```

记录结果

```
push!(J1_w_list, Sysplorer.GetVarValues("J1.w"))  
push!(J2_w_list, Sysplorer.GetVarValues("J2.w"))  
push!(time_list, Sysplorer.GetVarValues("time"))
```

```
end
```

结果绘图

```
println("start to plot")
```

```
subplot(1, 2, 1)
```

```
hold("on")
```

```
.....
```

示例详见: Syslab示例Examples-SyslabWorkspace-Demo_SysplorerAPI_ParameterAnalysis

建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

请各位专家指正！